APPLICATION NOTE

ATBM BLE DEMO 配网使用方法

ATBM

1x1
802.11b/g/n/ax
Wi-Fi/BLE 芯片

Table of contents

1 LINUX BLE 应用编译与生成文件说明(6461 和 RTOS 跳过该章节) 6

1.1 编译 6
1.2 WiFi 驱动加载连接共存固件与执行 BLE 应用步骤 7
1.3 BLE 配网 demo 流程 7

2 6461 编译与配网介绍（LINUX 跳过该章节） 9

2.1 编译工具链路径 9
2.2 Linux 编译注意事项 9
2.3 配置 ble 编译选项 10
2.4 Linux 生成烧录 bin 文件 10
2.5 Linux 编译路径错误需要改一下斜杆 11
2.6 BLE 配网 demo 流程 12
2.7 BLE HCI 数据导出 12

3 微信小程序配网 13

4 BLE 配网程序代码初始化与配网数据交互流程 15

4.1 主程序入口 15
4.2 BLE 线程初始化 16
4.3 BLE 常用 API 16
4.4 btshell 指令介绍用法 17
4.5 BLE 线程初始化入口与修改发射功率： 17
4.6 ATBM 广播包结构体： 17
4.7 BLE 广播包与扫描回应包配置 19
(1) 标准蓝牙广播包与扫描回应包 api 19
(2) 广播包长度计算公式 20
(3) 广播包初始化代码如下： 21
(4) 扫描回应包初始化 22
(5) 广播包长度超 31 字节排查 23
(6) 配置广播可以让手机搜索到 25
4.8 BLE 广播初始化、修改广播间隔与设备 MAC 随机地址配置 25

AltoBeam Inc. Confidential Doc

AN9310
Doc Rev: 1.4
Released:2025-04-18

4.9 BLE 设置公共地址 26
4.10 BLE 扫描 API 配置： 26
(1) 启动 BLE 扫描 API 26
(2) 停止 BLE 扫描 API 27
(3) 设置扫描固定 ble 名字才上传到应用层回调函数中 27
(4) 扫描回应包回调函数 28
(5) 显示收到广播设备 RSSI 与 Mac 地址 29
4.11 BLE 连接回调函数与连接成功后请求更新连接参数 29
4.12 请求更新 MTU 长度 30
4.13 实际发包 MTU 长度 30
4.14 在 BLE 连接回调函数中禁止发包和应用也不能在里面跑 30
5 BLE GAP 与 GATT 初始化函数入口 31
5.1 手机界面 BLE GAP 与 GATT 显示图 31
5.2 GAP 与 GATT 数组初始化 32
5.3 GAP 与 GATT 事件回调函数接口： 32
5.4 INDICATE 和 NOTIFY 介绍、配置与使用 33
5.5 手机订阅 INDICATE 34
5.6 INDICATE 初始化使用 35
(1) 收到手机订阅 indicate 事件后，创建线程发 255 包。 36
(2) 线程发包 demo 36
(3) Indicate 发包成功进该函数 36
(4) Indicate 发包 api 37
5.7 手机订阅 NOTIFY 37
5.8 NOTIFY 初始化与使用 38
5.9 Device Information READ 属性初始化与使用 39
(1) BLE READ 使用 40
5.10 WRITE 初始化与使用 41
(1) 5.6.1 WRITE 初始化 41
(2) 手机通过 WRITE 发数据给设备，设备收数据处理 41
(3) Ble 配网 demo 数据交互流程 42
(4) BLE 配网收发数据的 log 显示 43
6 移植自定义应用代码与退出 BLE 线程 43
6.1 客户应用代码建议在我们基础上修改 43
6.2 退出 ble 线程方式有两种方式(6461 暂未支持) 43
(1) 指令退出：atbm_cli quit。 43
(2) 函数退出：lib_ble_thread_exit() 43
在不能在协议栈里面调用，要在其他线程调用该 lib_ble_thread_exit()退出，需要用 atbm_ble_exit()函数替代 hif_ioctl_loop() 43
6.3 缩减 ble 代码大小 44
(1) 关闭 bt_shell 测试代码 44
把该宏置 MYNEWT_VAL_SHELL_CMD (0) 44
(2) 用不到 SM 加密可以去掉，能省十几 k(6461 暂未支持) 44

AltoBeam Inc. Confidential Document

6.4 缩小驱动大小，单独使用 WIFI+BLE 固件，去掉完整 WiFi 固件。(linux 专用) 45
7 导出 HCI 数据分析 45
8 错误码信息 47
8.1 以下是 API 返回值错误码原因 47
8.2 BLE 连接断开错误码解析 47
9 单独 BLE 广播与扫描（无法连接） 55
9.1 启动广播共存宏 55
9.2 启动 ble 广播共存 56
9.3 设置 ble 广播数据 57
9.4 ble 扫描到数据进入回调函数 57
9.5 过滤 BLE 接收到广播数据 58
10 库方式编译 DEMO (6461 不需要看) 58
10.1 启动编译成静态库宏 58
10.2 动态库编译 59
10.3 调用库编译应用 59
10.4 协议栈初始化函数 60
10.5 切换省内存版本 BLE 60
11 BLE MASTER 指令使用介绍 61
11.1 Nrf connect 软件启动 ble slave 广播 61
11.2 扫描附近所有 BLE 设备 61
11.3 扫描指定 BLE 名字设备 63
11.4 停止扫描 63
11.5 连接 slave 设备 63
11.6 显示连接设备的 GAP 与 GATT 65
11.7 BLE master 通过 WRITE 属性发数据 66
11.8 断开连接 slave 设备 66
12 BLE MASTER DEMO 使用介绍 66
12.1 根据名字连接 slave 设备 66
12.2 根据 BLE 设备地址连接 slave 设备 66
12.3 初始化 peer 属性队列 67
12.4 连接成功后获取 slave 端 Service 和 Characteristic、Descriptors 67
12.5 断开连接释放该连接内存占用 68
12.6 释放 peer 属性队列 68
12.7 获取完所有属性 69
12.8 打印已获取属性列表 69
12.9 获取对应服务与特征值 uuid 是否存在，并发起 read 请求获取数据 70
12.10 获取对应 Write 属性并发数据给从机 71
12.11 获取对于描叙符特征值后，并订阅 71
12.12 NOTIFY 订阅 API 使用 71
12.13 INDICATE 订阅 API 使用 72

AltoBeam Inc. Confidential Document

12.14 断开 slave 设备 72
13 BLE 主从机 SMP 加密使用 72
13.1 BLE Slave SMP 初始化 72
13.2 手机 nrf connect 软件建立加密连接： 75
13.3 BLE Slave 建立连接 log 76
13.4 BLE MASTER 连接成功进行加密绑定 api 76
13.5 BLE 主从机指令连接设备并建立加密方式 76
14 支持安卓 BLUEDROID(6461 不需要看) 77
15 支持 LINUX BLUEZ(6461 不需要看) 77
15.1 修改 BLE 公共地址 79
16 编译 OS 层报错修改汇总(6461 不需要看) 79
16.1 Os 平台适配报错 79
16.2 atbm_cli 输入 btshell 命令无法识别 80

AltoBeam Inc. Confidential Document

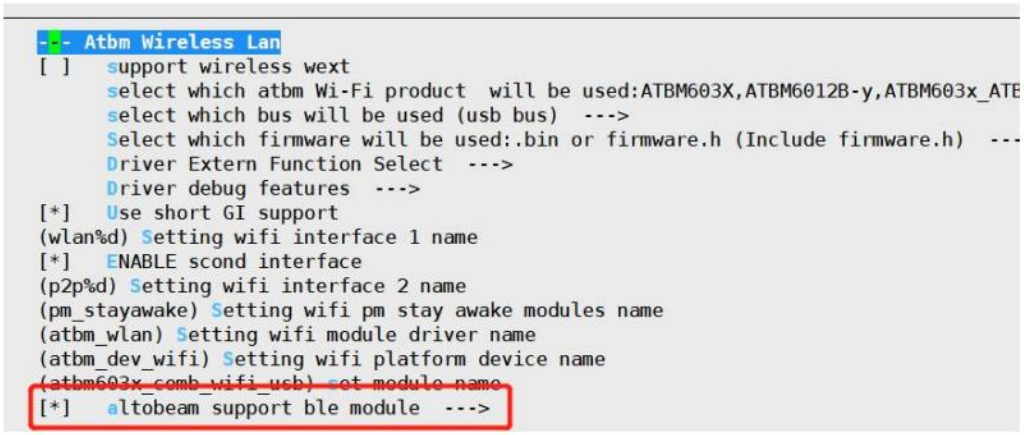
作者	说明	版本	日期
Fuzhenzhou	初始版本	v1.0	20250208
Fuzhenzhou	增加 hci 数据导出	v1.1	20250225
Wangxixi	修改格式	v1.2	20250312
Fuzhenzhou	新增打印已获取属性列表	v1.3	20250328
Fuzhenzhou	新增广播包超长排查方法	v1.4	20250418

AltoBeam Inc. Confidential Document

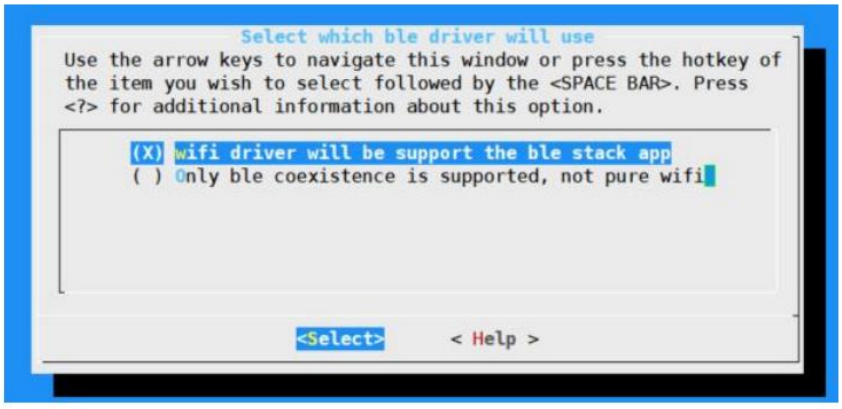
1 LINUX BLE 应用编译与生成文件说明(6461 和 rtos 跳过该章节)

1.1 编译

(10) BLE 支持



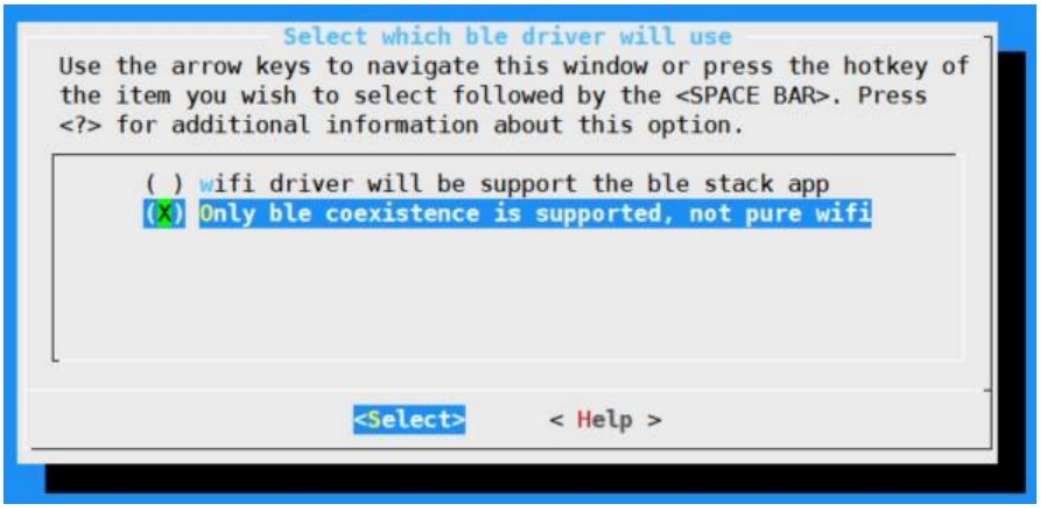
10.1 WIFI + BLE 连接共存配置



需要将带有 WiFiBLEComb 名称的固件改名为 fiwmare_<bus>_wifi_bt_comb.h 放到 hal_apollo 目录下。

使用该配置需要放置多个固件，支持纯 WIFI 固件和 BLE 共存固件

10.2 只支持 BLE+WIFI，不支持纯 WIFI



ble_smart_cfg 源码在驱动根目录的 ble_host\user_app\ble_smart_cfg 目录下
编译 WiFi 驱动：make
编译 BLE 应用：make ble_stack
或者使用我们推荐这种方式：修改 build_driver.sh，一次性 BLE 和 WiFi 两个同时编译。
编译 335 平台：./build_driver.sh 335
完整的编译完驱动会生成以下文件：

```
ubuntu_sz_2ndfuzhenzhou~/6062/atbm_wifi6_40M/driver_install\ $ls
ATBM606x_wifi_usb.ko  atbm_cli  atbm_tool  cfg80211.ko  nimble_linux  t
```

ATBM606x_wifi_usb.ko //WiFi 驱动
cfg80211.ko //WiFi6 建议不用内核自带 cfg80211，WiFi4 可以直接使用内核自带。
nimble_linux //BLE 应用，使用 BLE 时拷到板子上。
atbm_cli //跟 BLE 通讯，用于 AT 指令透传控制；不用指令控制 BLE 时可以不用。

1.2 WiFi 驱动加载连接共存固件与执行 BLE 应用步骤

在 Linux 系统里面直接执行：
insmod cfg80211.ko
insmod ATBM606x_wifi_usb.ko wifi_bt_comb=1 //1：加载 WiFi+BLE 固件 0：加载完整版 WiFi 固件

需要查询以下路径 wifi 加载驱动成功才能启动 ble，不然会启动失败。
/sys/module/<module name>/parameters/insmod_stat 驱动加载完成置为 1

./nimble_linux & //运行 BLE 协议栈

1.3 BLE 配网 demo 流程

连接脚本需要在根目录\ble_host\user_app\目录下把 connect_ap.sh 复制到 linux 系统 tmp 目录下执行。

脚本代码如下：

connect_ap. sh

```
else
    echo -e "
ctrl_interface=/tmp/wpa_supplicant\n
ap_scan=1\n
network={\n
\tssid=\"$1\"\n
\tpsk=\"$2\"\n
\tscan_ssid=1\n
    }\n
    " > /tmp/wpa_supplicant.conf
fi
}



connect_ap()
{

    write_wpa_conf $1 $2

    wpa_supplicant -Dnl80211 -iwlan0 -c /tmp/wpa_supplicant.conf -B
    udhcpc -iwlan0 &
}


connect_ap $1 $2
```

通过BLE配网流程拿到SSID和PWD写成wpa_supplicant.conf文件再启动连接

ble_smart_demo.c

Symbol Name (Alt+L)

- u16
- s32
- u32
- s64
- u64
- MAX_SYNC_EVE
- ATBM_SMT_STA
- ATBM_SMT_STA
- cfg_done_conne
- connect_ap
- status_async
  - type
  - driver_mode
  - list_empty
  - event_buffer
- at_cmd_direct_i
  - len
  - cmd
- ble_smt
- gCmdStr
- if 0
- endif
- if 0
- endif
- if 0
- endif
- cmd_system

```
    printf("wpa_success %d\n", wpa_success);
    return wpa_success;

} « end smt_check_wpa_status »

void connect_wifi_ap(u8* ssid, u8 ssidLen, u8* pwd, u8 pwdLen)
{
    u8 ssidStr[33] = { 0 };
    u8 pwdStr[65] = { 0 };
    int sret, i;
    int conn_ap_len;
    if (ssidLen > 32 || pwdLen > 64) {
        fprintf(stdout, "ssidLen(%d) pwdLen(%d) error!\n", ssidLen, pwdLen);
        return;
    }

    memcpy(ssidStr, ssid, ssidLen);
    memcpy(pwdStr, pwd, pwdLen);
    conn_ap_len = strlen(connect_ap);
    if(conn_ap_len){
        if(pwdLen)
            sprintf(gCmdStr, "%s %s %s",connect_ap,ssid,pwd);
        else
            sprintf(gCmdStr, "%s %s",connect_ap,ssid);
    }else{
        if(pwdLen)
            sprintf(gCmdStr, "/tmp/connect_ap.sh %s %s",ssid,pwd);
        else
            sprintf(gCmdStr, "/tmp/connect_ap.sh %s",ssid);
    }
    printf("start CMD:%s \n",gCmdStr);
    system(gCmdStr);
//  cmd_system(gCmdStr,NULL,0);
#if 0
    sprintf(gCmdStr, "ifconfig wlan0 down");
```

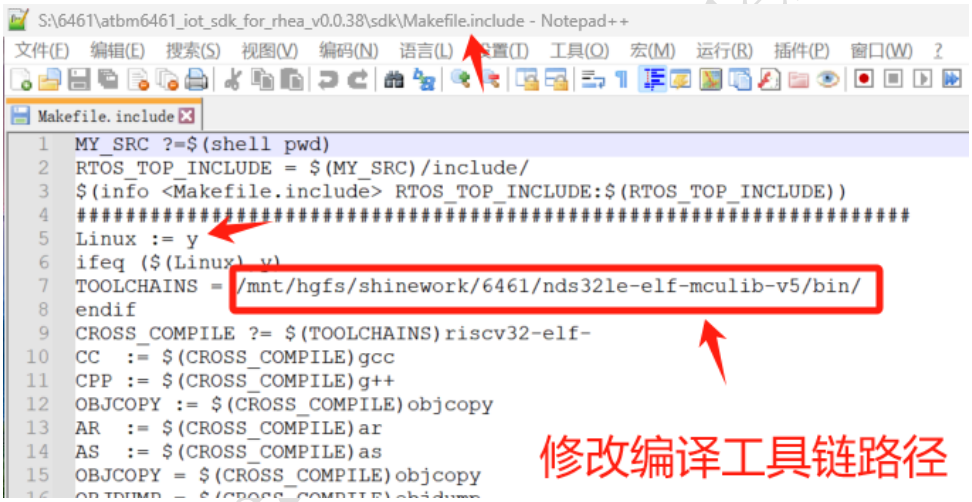
2 6461 编译与配网介绍（linux 跳过该章节）

2.1 编译工具链路径



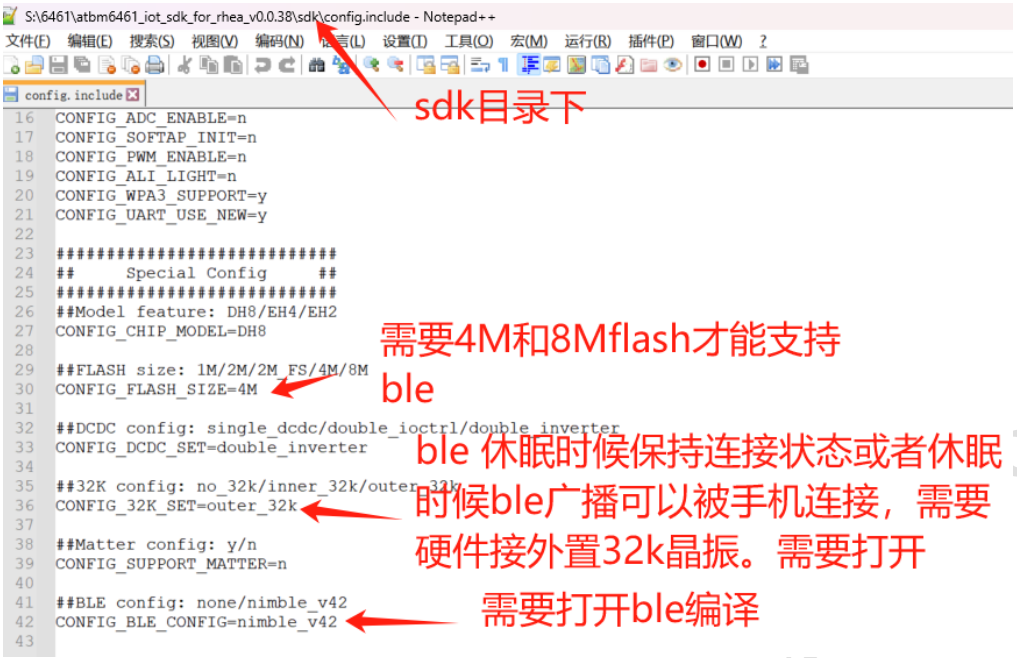
2.2 Linux 编译注意事项

打开编译 linux



AltoBeam Confidential

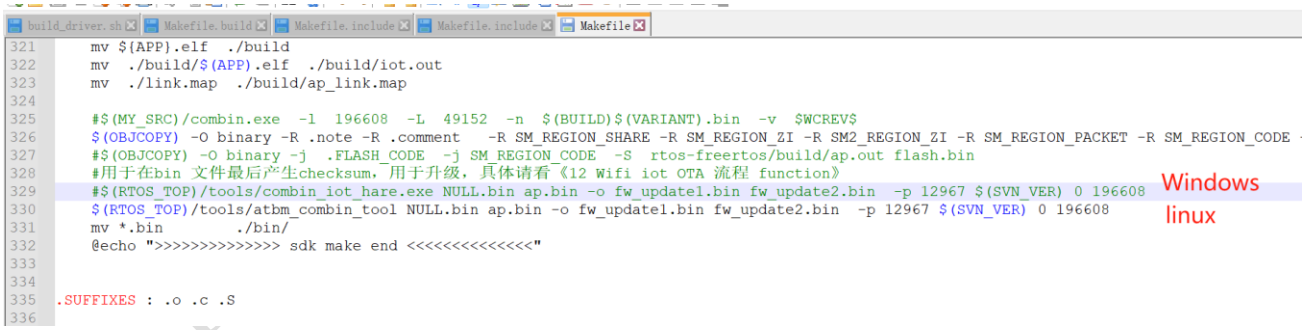
2.3 配置 ble 编译选项



2.4 Linux 生成烧录 bin 文件

需要改成 linux 生成方式

$(RTOS_TOP)/tools/atbm_combin_tool NULL.bin ap.bin -o fw_update1.bin fw_update2.bin -p 12967 $(SVN_VER) 0 196608





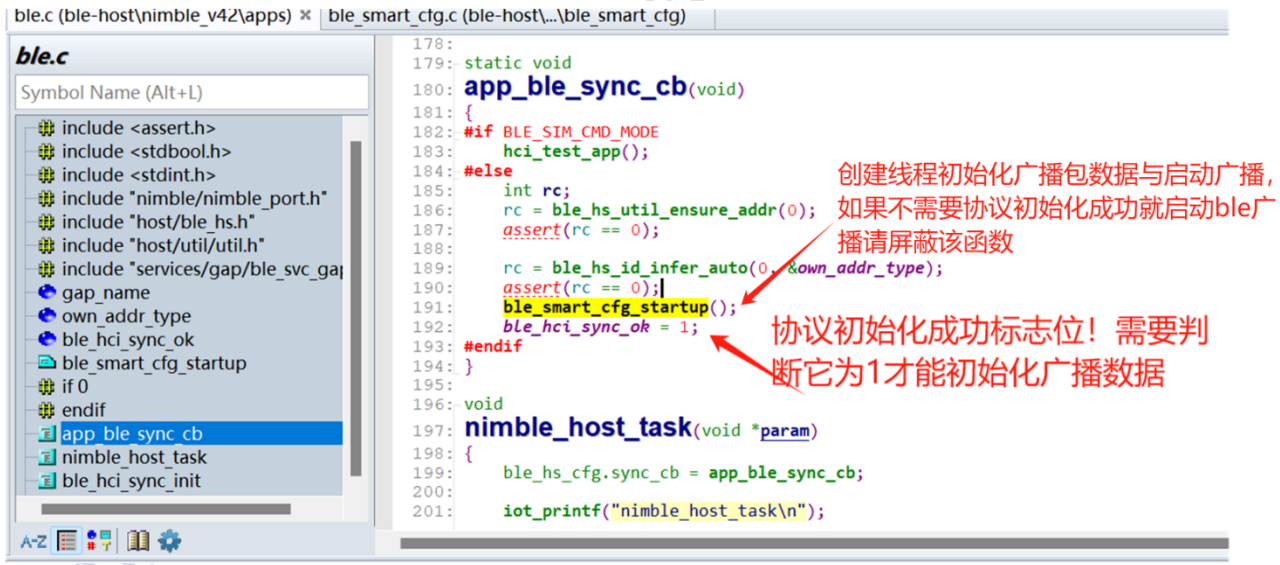
Confidential Document

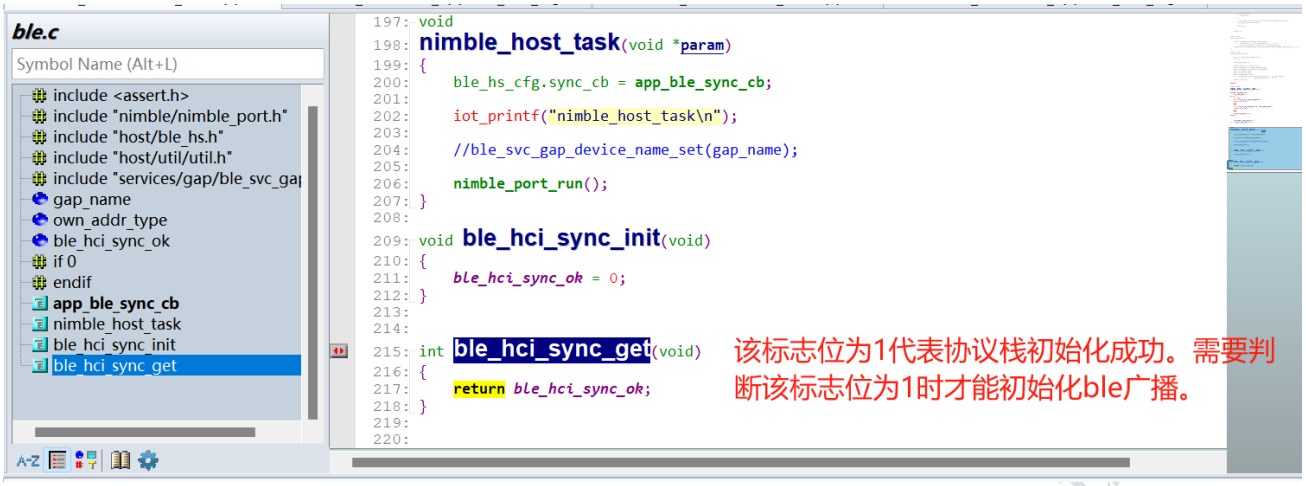
2.5 Linux 编译路径错误需要改一下斜杆

“\”改成“/” #include “transport\ram\ble_hci_ram.h”改成#include “transport/ram/ble_hci_ram.h”



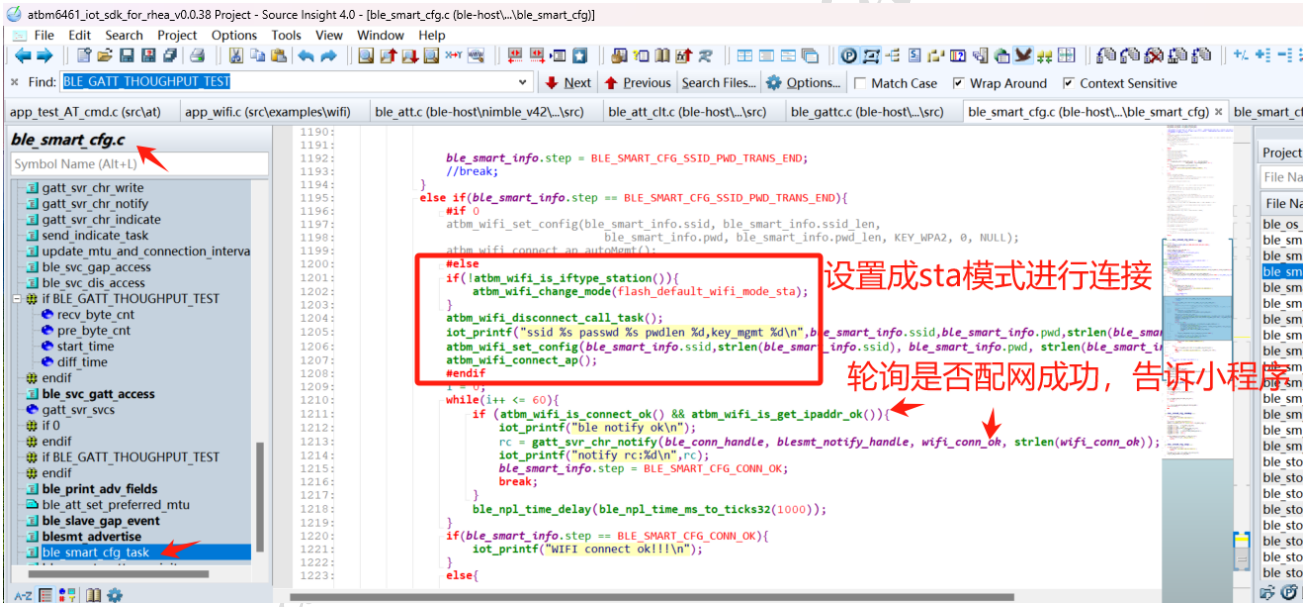
最后 make all 编译即可(编译出来 demo 默认启动 ble 广播，请根据自己的需求是否要默认启动不需要默认启动请屏蔽)





2.6 BLE 配网 demo 流程

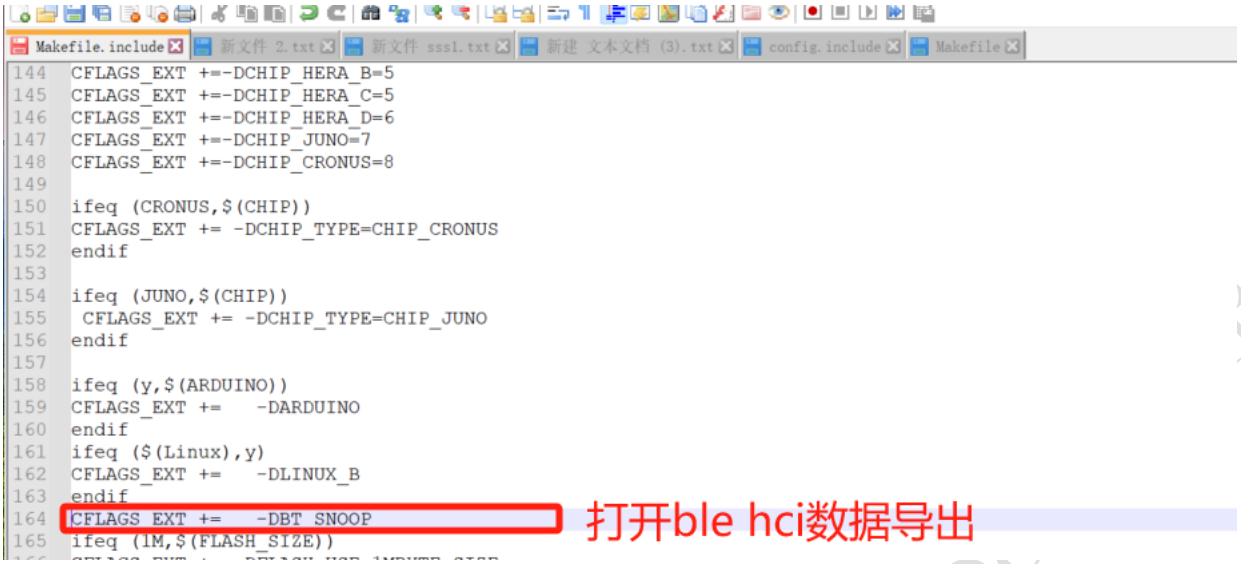
需要按照第 3 章节微信小程序进行配网,收到 ssid 后进入下面调用 api 进行连接并反馈给小程序配网成功



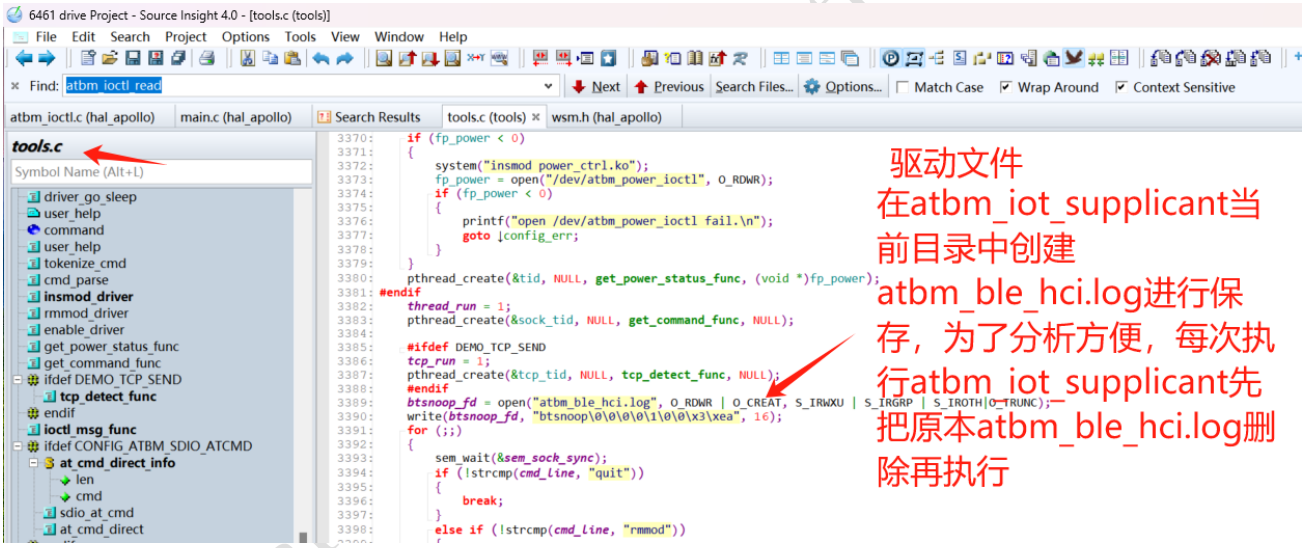
2.7 BLE HCI 数据导出

BLE 应用异常问题调试需要打开宏抓取 hcilog 分析问题

CFLAGS_EXT += -DBT_SNOOP



需要手机未连接 ble 前启动 atbm_iot_supplicant &，不然没办法解析 ble 数据。复现问题后，导出 atbm_ble_hci.log 给我们用软件打开分析



3 微信小程序配网

微信小程序测试需要跟我们申请：





点击蓝牙配网



输入要连接的路由器的 ssid & password 点击 WIFI 连接

Confidential Document

4 BLE 配网程序代码初始化与配网数据交互流程

4.1 主程序入口

目录 ble_host\nimble_v42\apps\main.c



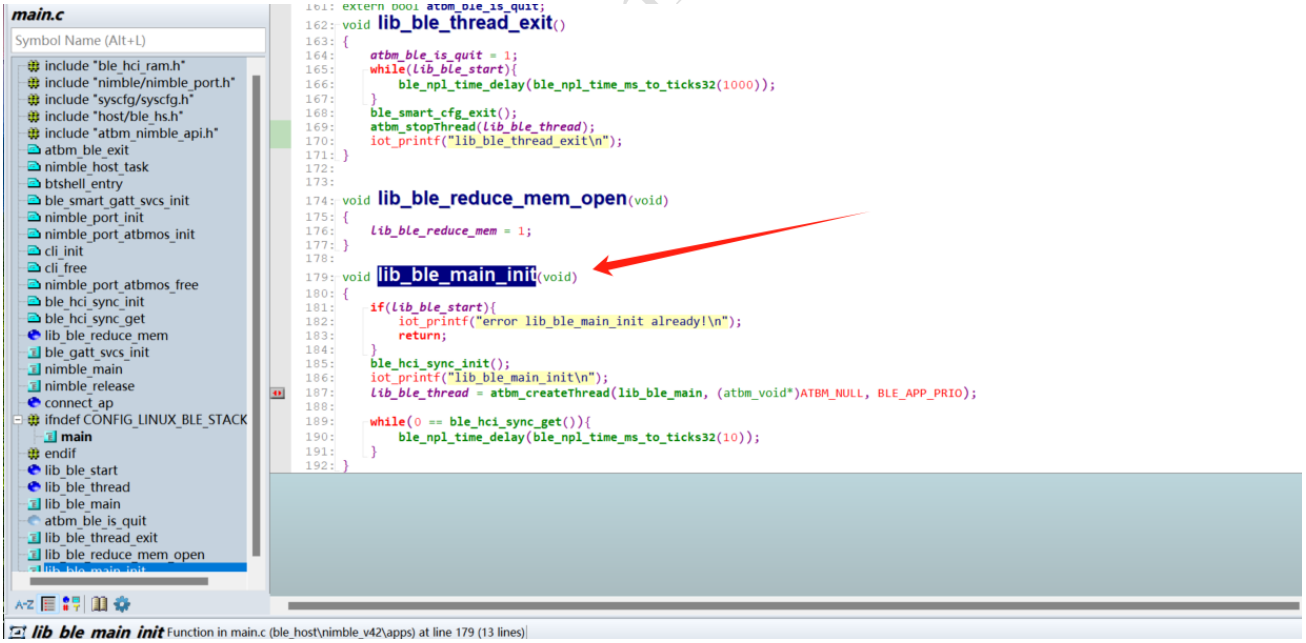

```
extern int atbm_ble_exit(void);

#ifndef CONFIG_LINUX_BLE_STACK_LIB
int main(int argc,char * argv[])
//int atbm_ble_start(void)
{

    if(argc > 1 && argv[1]){
        memcpy(connect_ap,argv[1],strlen(argv[1]));
        printf("connect_ap file: %s ,%s,%d\n",connect_ap,argv[1],strlen(argv[1]));
    }
    iot_printf("======>>atbm_ble_start>>>\n");
    nimble_main();
    //atbm_startconfig cmd
    atcmd_init_ble();
    //
    hif_ioctl_init();
    //
    ble_hs_sched_start();
    iot_printf("hif_ioctl_loop\n");
    //start ioctl to wifi driver
    //hif_ioctl_loop();
    atbm_ble_exit();
    iot_printf("ble_hs_hci_cmd_reset\n");
    nimble_release();
    //reset LL BLE when quit
    //ble_hs_hci_cmd_reset();
    return 0;
} « end main »
#endif

int lib_ble_start = 0;
```

库方式初始化函数：




```
extern bool atbm_ble_is_quit;
void lib_ble_thread_exit()
{
    atbm_ble_is_quit = 1;
    while(lib_ble_start){
        ble_npl_time_delay(ble_npl_time_ms_to_ticks32(1000));
    }
    ble_smart_cfg_exit();
    atbm_stopThread(lib_ble_thread);
    iot_printf("lib_ble_thread_exit\n");
}

void lib_ble_reduce_mem_open(void)
{
    lib_ble_reduce_mem = 1;
}

void lib_ble_main_init(void)
{
    if(lib_ble_start){
        iot_printf("error lib_ble_main_init already!\n");
        return;
    }
    ble_hci_sync_init();
    iot_printf("lib_ble_main_init\n");
    lib_ble_thread = atbm_createThread(lib_ble_main, (atbm_void*)ATBM_NULL, BLE_APP_PRIO);

    while(0 == ble_hci_sync_get()){
        ble_npl_time_delay(ble_npl_time_ms_to_ticks32(10));
    }
}
```

4.2 BLE 线程初始化



4.3 BLE 常用 API

全部 api 请进入该网址查看：

https://mynewt.apache.org/latest/network/ble_hs/ble_gatts.html

BLE 广播主要对 ble_hs_adv_fields 初始化，最长 31 个字节。如果需要 2 个广播包就需要配置扫描回应包。

Api：

ble_gap_adv_set_fields(const struct ble_hs_adv_fields *adv_fields) 配置广播包

ble_gap_adv_rsp_set_fields(const struct ble_hs_adv_fields *rsp_fields) 配置扫描回应包

int ble_att_set_preferred_mtu(uint16_t mtu) //MTU 设置连接成功设置最大 256 字节

Int ble_gap_adv_start(uint8_t own_addr_type, const ble_addr_t *direct_addr,
int32_t duration_ms,
const struct ble_gap_adv_params *adv_params,
ble_gap_event_fn *cb, void *cb_arg) //启动广播

int ble_gap_adv_stop(void) //停止广播

Int ble_gap_adv_active(void) //返回为 1 时处于广播状态

Int ble_gap_set_default_tx_power(int8_t dbm) //范围值(0 ~ 15)dbm 必须广播开启前设置

Int ble_gap_terminate(uint16_t conn_handle, uint8_t hci_reason) //断开手机连接

Int ble_gap_update_params(uint16_t conn_handle,
const struct ble_gap_upd_params *params) //申请更改连接间隔

void get_ble_addr(uint8_t* ble_addr) //获取 BLE 的默认 mac 地址，BLE MAC 地址默认是 WIFI MAC 地址最后一位+1

AltoBeam Inc. Confidential

4.4 btshell 指令介绍用法

https://mynewt.apache.org/latest/network/btshell/btshell_GAP.html

4.5 BLE 线程初始化入口与修改发射功率：

BLE 发射功率范围值为 0-15db



4.6 ATBM 广播包结构体：

AD Type	Value	描述
Flags	0x01	广播出自己蓝牙某些特性
Service UUIDs	0x02~0x07	广播出自己服务的 UUID
Local Name	0x08/0x09	广播出自己的蓝牙名字
TX Power Level	0x0A	广播出自己的射频发射功率
Simple Pairing Option OOB Tags	0x0D~0x0F	广播出安全管理带外标签(本文忽略)
Security Manager TK Value	0x10	广播出带外方式配对绑定时的 TK(本文忽略)
Security Manager OOB Flags	0x11	广播出带外特性标志(本文忽略)
Slave Connection Interval Range	0x12	广播出自己希望的连接参数范围
Service Solicitation	0x14/0x15	广播出自己希望来接自己的主机有特定的服务
Service Data	0x16	服务数据
Manufacturer Specific Data	0xFF	广播出厂商信息（用户可以放自定义数据）

广播包 31 字节和扫描回应包 31 字节配置结构体都是一样。如下：

```
struct ble_hs_adv_fields {
    /*** 0x01 - Flags. */
    uint8_t flags;

    /*** 0x02,0x03 - 16-bit service class UUIDs. */
    ble_uuid16_t *uuids16;
    uint8_t num_uuids16;
    unsigned uuids16_is_complete:1;

    /*** 0x04,0x05 - 32-bit service class UUIDs. */
    ble_uuid32_t *uuids32;
    uint8_t num_uuids32;
    unsigned uuids32_is_complete:1;

    /*** 0x06,0x07 - 128-bit service class UUIDs. */
    ble_uuid128_t *uuids128;
    uint8_t num_uuids128;
    unsigned uuids128_is_complete:1;

    /*** 0x08,0x09 - Local name. */
    uint8_t *name;
    uint8_t name_len;
    unsigned name_is_complete:1;

    /*** 0x0a - Tx power level. */
    int8_t tx_pwr_lvl;
    unsigned tx_pwr_lvl_is_present:1;

    /*** 0x0d - Slave connection interval range. */
    uint8_t *slave_itvl_range;

    /*** 0x16 - Service data - 16-bit UUID. */
    uint8_t *svc_data_uuid16;
    uint8_t svc_data_uuid16_len;

    /*** 0x17 - Public target address. */
    uint8_t *public_tgt_addr;
    uint8_t num_public_tgt_addrs;

    /*** 0x19 - Appearance. */
    uint16_t appearance;
    unsigned appearance_is_present:1;

    /*** 0x1a - Advertising interval. */
```

AltoBeam Inc. Confidential Document

```
    uint16_t adv_itvl;
    unsigned adv_itvl_is_present:1;

    /*** 0x20 - Service data - 32-bit UUID. */
    uint8_t *svc_data_uuid32;
    uint8_t svc_data_uuid32_len;

    /*** 0x21 - Service data - 128-bit UUID. */
    uint8_t *svc_data_uuid128;
    uint8_t svc_data_uuid128_len;

    /*** 0x24 - URI. */
    uint8_t *uri;
    uint8_t uri_len;

    /*** 0xff - Manufacturer specific data. */
    uint8_t *mfg_data;
    uint8_t mfg_data_len;
};
```

4.7 BLE 广播包与扫描回应包配置

(1) 标准蓝牙广播包与扫描回应包 api

类似涂鸦配网适配传进来广播包数据和扫描回应包数据，调用下面两个数据传进去

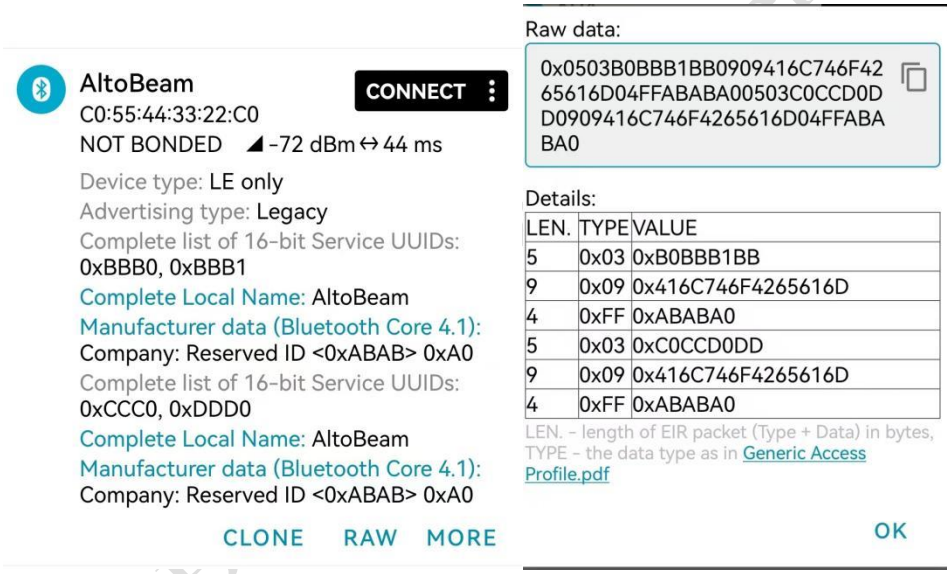


Confidential Document

int ble_gap_adv_set_data(const uint8_t *data, int data_len) //广播包
int ble_gap_adv_rsp_set_data(const uint8_t *data, int data_len) //扫描回应包



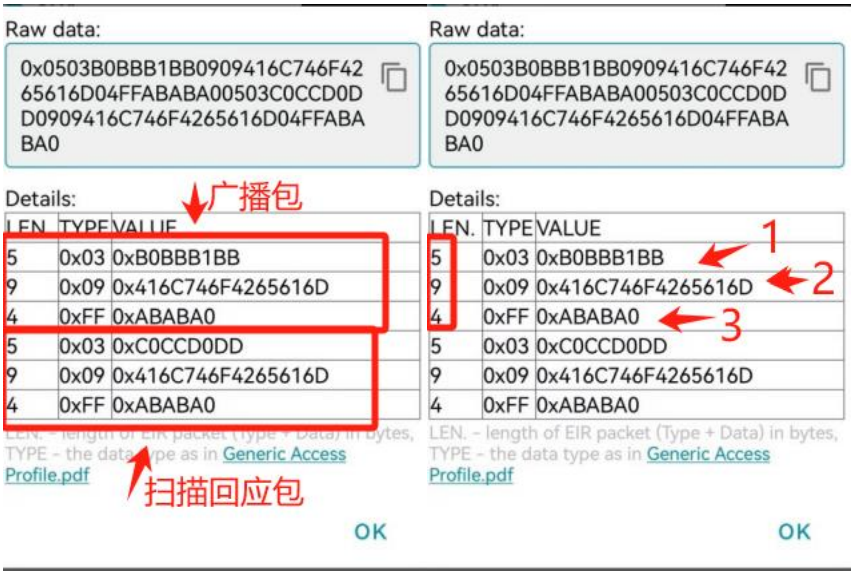
BLE 程序运行后，在手机搜到 BLE 广播数据显示。注意广播包长度最大 31 字节！



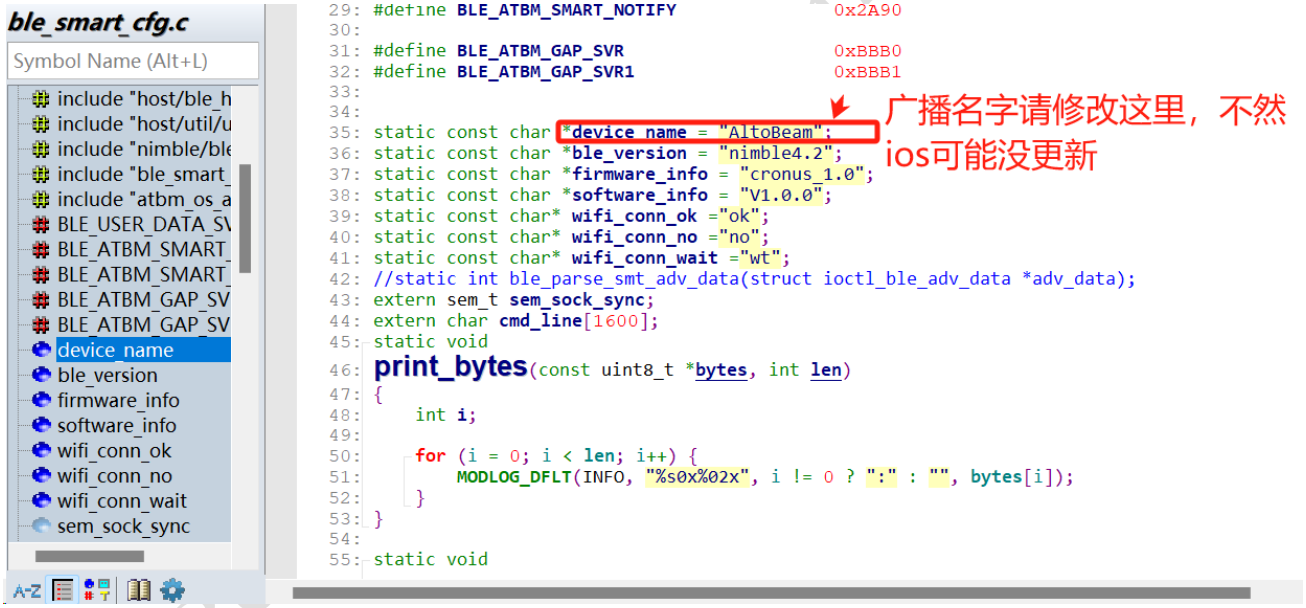
(2) 广播包长度计算公式

广播数据的 LTV 格式
广播数据采用长度-类型-值（LTV）结构组织，每个字段的组成如下：
Length（长度）：1 字节，表示后续 Type + Value 的总字节数。
Type（类型）：1 字节，定义数据的用途（如设备名称、UUID 等）。
Value（值）：可变长度，具体数据内容。
上面广播包长度计算方式，每个 TYPE 总长度 LEN 值全部相加(5+9+4)=18
有三个 type 类型还要加上 3 18+3=21 总长度

Confidential Document



(3) 广播包初始化代码如下：



AltoBeam Confidential Document

```
static void blesmt_advertise(void)
{
    struct ble_gap_adv_params adv_params;
    struct ble_hs_adv_fields fields;
    ble_uuid16_t uuids16[2];
    uint8_t mfg_data[3] = {0xAB, 0xAB, 0xA0};//自 定 义 广 播 自 己 私 有 数 据 ，
    int rc;
    memset(&fields, 0, sizeof(fields));
    //广 播 数 据
    fields.name = (uint8_t *)device_name;
    fields.name_len = strlen(device_name);
    fields.name_is_complete = 1;//选 这 0x09  Complete local name
    //#define GAP_ADTYPE_LOCAL_NAME_SHORT          0x08 //!< Shortened local name

    //#define GAP_ADTYPE_LOCAL_NAME_COMPLETE       0x09 //!< Complete local name


    uuids16[0].u.type = BLE_UUID_TYPE_16;
    uuids16[0].value = BLE_ATBM_GAP_SVR;//声 明 存 在 GAP服 务
    uuids16[1].u.type = BLE_UUID_TYPE_16;
    uuids16[1].value = BLE_ATBM_GAP_SVR1;//声 明 存 在 GAP服 务
    fields.uuids16 = &uuids16[0];
    fields.num_uuids16 = 2;//声 明 了 几 个 uuid
    fields.uuids16_is_complete = 1;//咨 询 gap格 式 是 完 整 还 是 非 完 整 ,上 面 填 的 是 完 整 所 以 选 择 广 播 数 据
    //#define GAP_ADTYPE_16BIT_MORE                0x02 //!< Service: More 16-bit UUIDs available
    //#define GAP_ADTYPE_16BIT_COMPLETE            0x03 //!< Service: Complete list of 16-bit UUIDs

    //uint8_t public_tgt_addr[6] = {0x01, 0x02, 0x03,0x04,0x05,0x06};
    //fields.public_tgt_addr=public_tgt_addr;
    //fields.num_public_tgt_addrs=1;

    fields.mfg_data = mfg_data;//厂 商 私 有 协 议
    fields.mfg_data_len = 3;//注 意 必 须 跟 mfg_data数 组 大 小 一 样

    rc = ble_gap_adv_set_fields(&fields);//广 播 包 配 置
    if (rc != 0) {
        iot_printf("error setting advertisement data; rc=%d\n", rc);
        return;
    }
```



如果超过广播包超过 31 字节，请把扫描回应包所有 demo 删了。存放自己广播包数据。参考下面

(4) 扫描回应包初始化

跟广播包初始化一样。只是换成了 ble_gap_adv_rsp_set_fields 进行初始化。代码如下：

```
//以 下 是 扫 描 回 应 包 配 置
struct ble_hs_adv_fields adv_fields;
ble_uuid16_t uuids16_rsp[2];
uint8_t mfg_rsp_data[3] = {0xff, 0xcc, 0xdd};//自 定 义 广 播 自 己 私 有 数 据 ，
memset(&adv_fields, 0, sizeof(adv_fields));
//广 播 数 据 https://blog.csdn.net/m0_57781768/article/details/127739862
adv_fields.name = (uint8_t *)device_name;
adv_fields.name_len = strlen(device_name);
adv_fields.name_is_complete = 1;//选 这 0x09 Complete local name
//#define GAP_ADTYPE_LOCAL_NAME_SHORT          0x08 //!< Shortened local name
//#define GAP_ADTYPE_LOCAL_NAME_COMPLETE       0x09 //!< Complete local name
uuids16_rsp[0].u.type = BLE_UUID_TYPE_16;
uuids16_rsp[0].value = 0xccc0;//声 明 存 在 GAP服 务
uuids16_rsp[1].u.type = BLE_UUID_TYPE_16;
uuids16_rsp[1].value = 0xddd0;//声 明 存 在 GAP服 务
adv_fields.uuids16 = &uuids16_rsp[0];
adv_fields.num_uuids16 = 2;//声 明 了 几 个 uuid
adv_fields.uuids16_is_complete = 1;//咨 询 gap格 式 是 完 整 还 是 非 完 整 ,上 面 填 的 是 完 整 所 以 选 择 广 播 数
//#define GAP_ADTYPE_16BIT_MORE                0x02 //!< Service: More 16-bit UUIDs available
//#define GAP_ADTYPE_16BIT_COMPLETE            0x03 //!< Service: Complete list of 16-bit UUIDs

//fields.public_tgt_addr=&mac_addr[0];
//fields.num_public_tgt_addrs=1;
adv_fields.mfg_data = mfg_data;
adv_fields.mfg_data_len = 3;//注 意 必 须 跟 mfg_data数 组 大 小 一 样
rc=ble_gap_adv_rsp_set_fields(&adv_fields);
 if (rc != 0) {
    iot_printf("error setting advertisement rsp data; rc=%d\n", rc);
    return;
}
```

(5) 广播包长度超 31 字节排查

遇到设置广播包或者扫描回应包失败码报错 rc=4，参考 api 返回错误码 8.1 章节是 BLE_HS_EMSGSIZE 4

```
rc = ble_gap_adv_set_fields(&fields);
if (rc != 0) {
    iot_printf("error setting advertisement data; rc=%d\n", rc);
    return;
}
rc=ble_gap_adv_rsp_set_fields(&adv_fields);
if (rc != 0) {
    iot_printf("error setting advertisement rsp data; rc=%d\n", rc);
return;
}
```

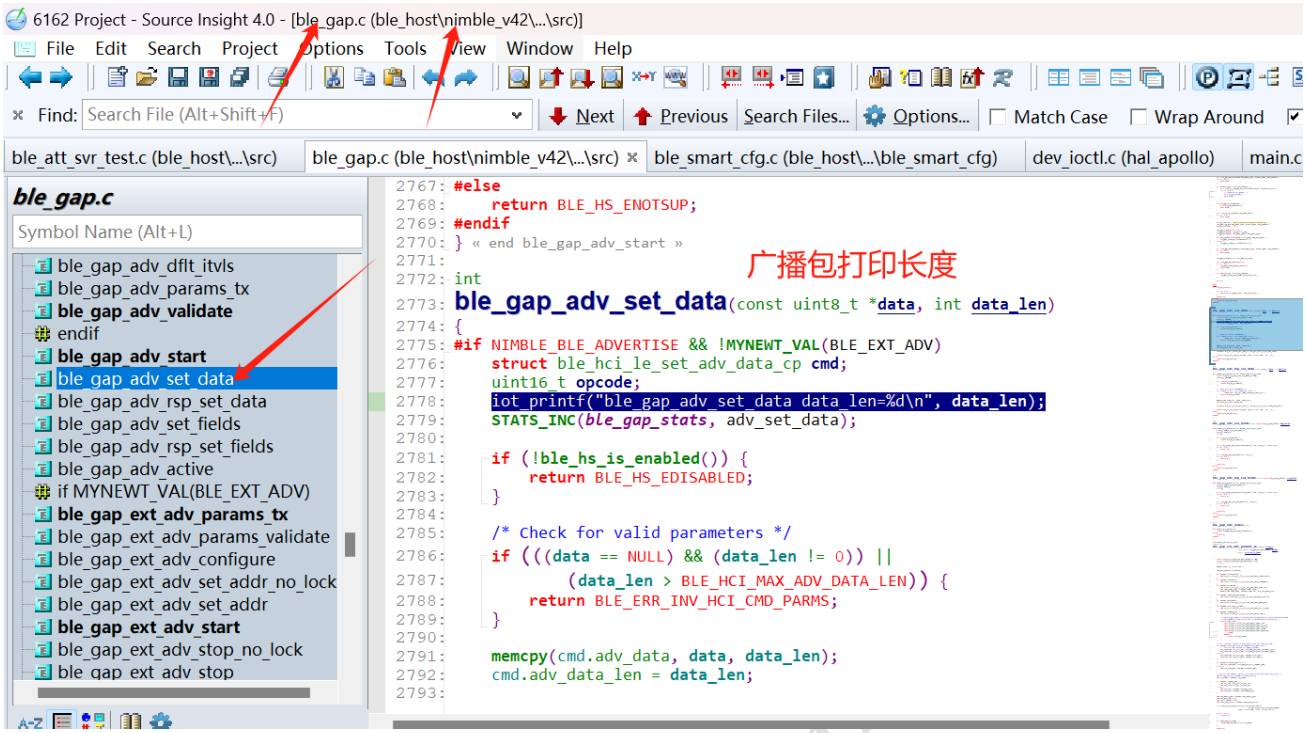
8.1 以下是 API 返回值错误码原因

```
#define BLE_HS_EAGAIN               1
#define BLE_HS_EALREADY             2
#define BLE_HS_EINVAL               3
#define BLE_HS_EMSGSIZE             4
#define BLE_HS_ENOENT               5
#define BLE_HS_ENOMEM               6
#define BLE_HS_ENOTCONN             7
#define BLE_HS_ENOTSUP              8
#define BLE_HS_EAPP                 9
#define BLE_HS_EBADDATA             10
#define BLE_HS_EOS                  11
#define BLE_HS_ECONTROLLER          12
#define BLE_HS_ETIMEOUT             13
```

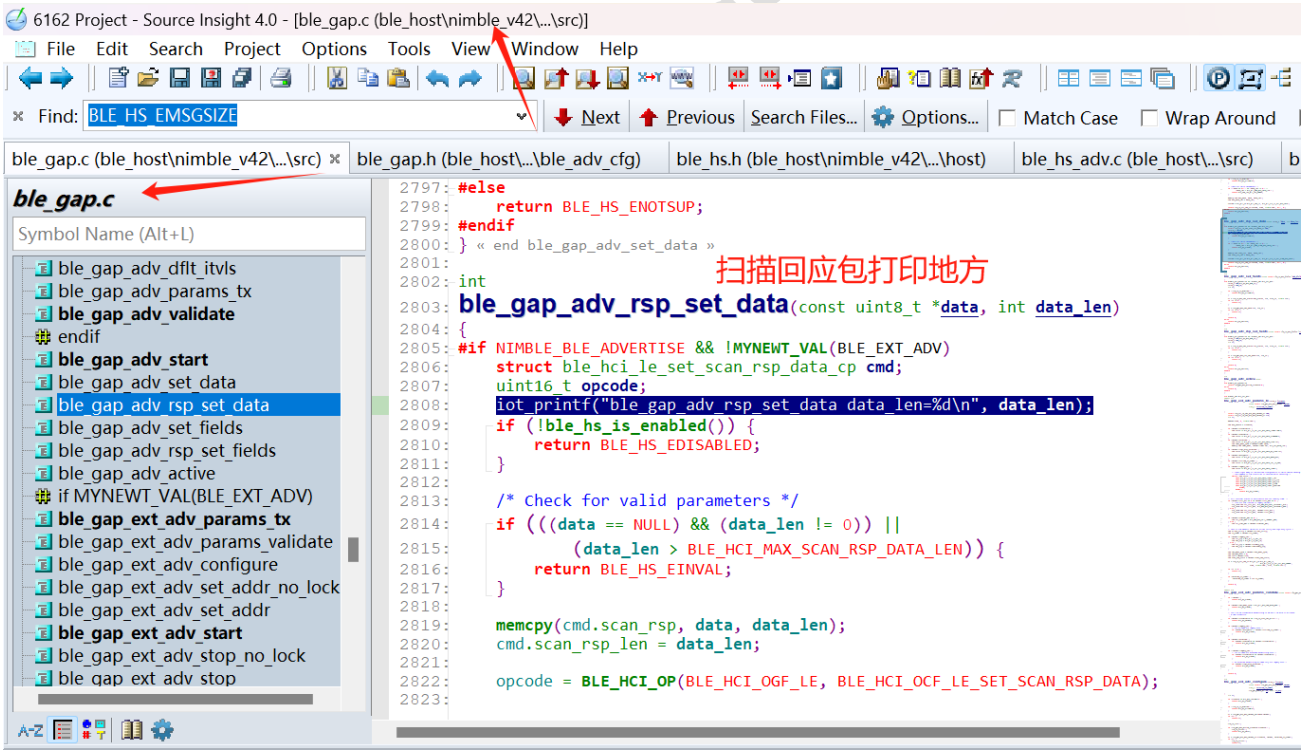
增加打印查看设置广播长度为多少

```
iot_printf("ble_gap_adv_set_data data_len=%d\n", data_len);
```

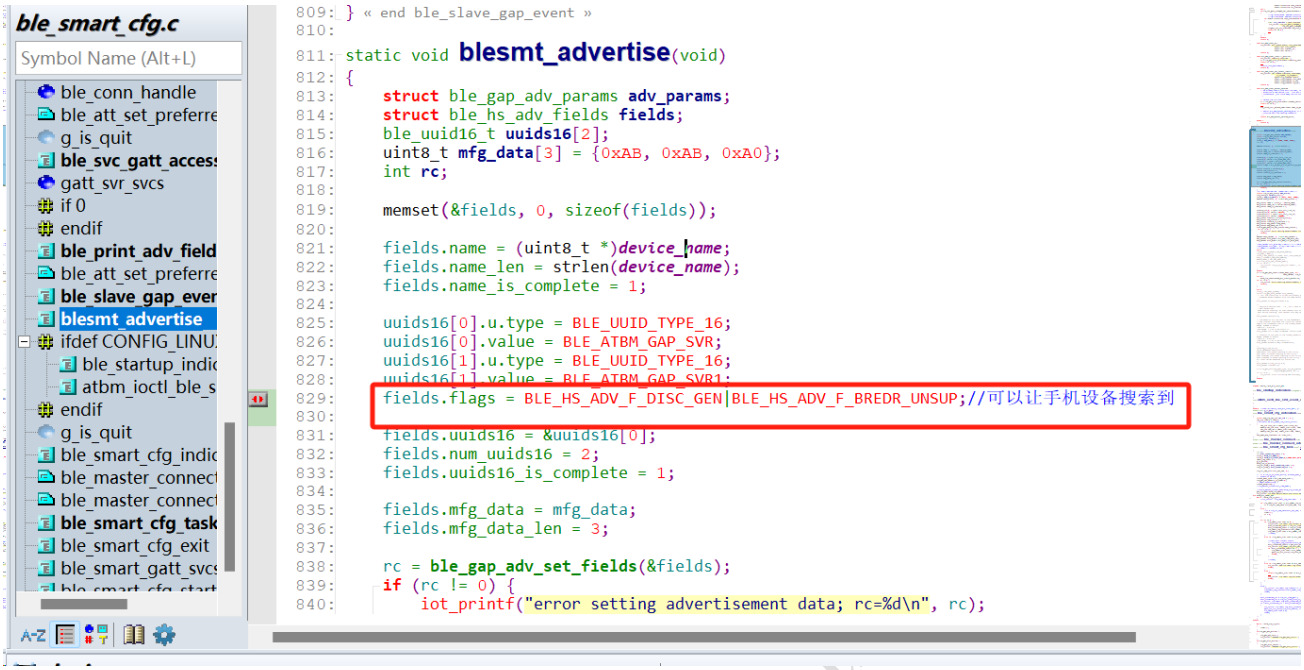
Confidential



iot_printf("ble_gap_adv_rsp_set_data data_len=%d\n", data_len);



(6) 配置广播可以让手机搜索到



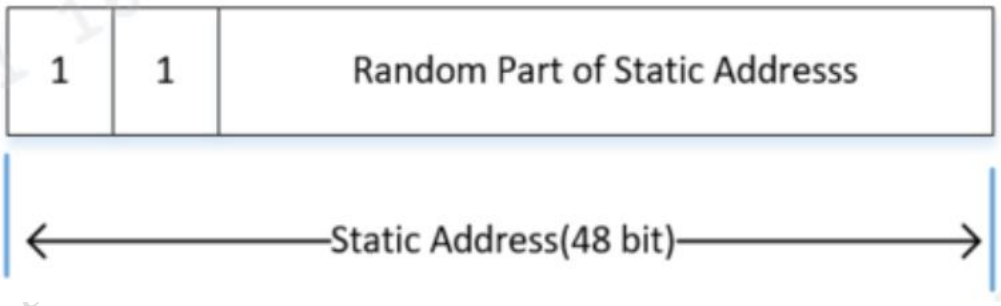
注意：这个只能让手机发现，如果想要手机系统直接连接还需要参考第十章节启动 smp 加密方式。

4.8 BLE 广播初始化、修改广播间隔与设备 MAC 随机地址配置

Random Device Address(随机设备地址):

该地址是 BLE 协议新增的一种地址类型，该类型的设备地址不是固定分配的，而是设备启动后随机生成的。最高两位为“11”，剩下的 46bit 是一个随机数，不能全部为 0 或者 1,。这 46 位随机数在一个上电周期内保持不变，下一次上电时可以改变，也可以保持不变。

uint8_t mac_addr[6] = {0x00, 0x22, 0x33,0x44,0x55,0xC0};//小端格式



其中 ble_slave_gap_event(struct ble_gap_event *event, void *arg)是连接状态回调函数。

```
//广 播 参 数
//不 可 连 接 模 式 ;
//定 向 连 接 模 式 ;
//无 向 连 接 模 式 ;  BLE_GAP_CONN_MODE_UND
memset(&adv_params, 0, sizeof(adv_params));
adv_params.conn_mode = BLE_GAP_CONN_MODE_UND;

//不 可 发 现 模 式 ;
//有 限 可 发 现 模 式 ;
//一 般 可 发 现 模 式 ;  BLE_GAP_DISC_MODE_GEN
adv_params.disc_mode = BLE_GAP_DISC_MODE_GEN;
adv_params.itvl_min= 60;//最 小 广 告 间 隔 , 0表 示 使 用 默 认 值 , 单 位 : 0.625ms */
adv_params.itvl_max  = 70;//最 大 广 告 间 隔 , 0表 示 使 用 默 认 值 ,单 位 : 0.625ms */
//广 播 间 隔 值 20ms-10.24s

blesmt_addr_type=BLE_OWN_ADDR_RANDOM;//BLE_OWN_ADDR_RANDOM;
ble_addr_t addr_t;
uint8_t mac_addr[6] = {0xC0, 0x22, 0x33,0x44,0x55,0xC0};//小 端 格 式  自 定 义 mac地 址
addr_t.type=BLE_OWN_ADDR_RANDOM;
memcpy(addr_t.val,mac_addr,6);
// ble_hs_id_set_rnd(mac_addr);
// ble_gap_ext_adv_set_addr(BLE_ADV_INSTANCES, &addr_t);

rc = ble_hs_id_set_rnd(mac_addr);
if (rc != 0) {
    iot_printf("ble_hs_id_set_rnd rc=%d\n", rc);
    return;
}

rc = ble_gap_adv_start(blesmt_addr_type, NULL, BLE_HS_FOREVER,
                       &adv_params, ble_slave_gap_event, NULL);//启 动 广 播
if (rc != 0) {
    iot_printf("error ble_gap_adv_start; rc=%d\n", rc);
    return;
}
```

4.9 BLE 设置公共地址

BLE MAC 地址是小端格式，启动广播前设置。

```
84  vendor interface for set controller pub addr
85  SRAM_CODE int ble_hs_vs_set_pub_addr(uint8_t *pub_addr)
86  {
87      int rc;
88
89      rc = ble_hs_hci_set_pub_addr(pub_addr);
90      if(rc){
91          return rc;
92      }
93
94      ble_hs_id_set_pub(pub_addr);
95
96      return 0;
97  }
```

4.10 BLE 扫描 API 配置：

(1) 启动 BLE 扫描 API

SRAM_CODE int ble_gap_disc(uint8_t own_addr_type, int32_t duration_ms,
const struct ble_gap_disc_params *disc_params,
ble_gap_event_fn *cb, void *cb_arg)



(2) 停止 BLE 扫描 API

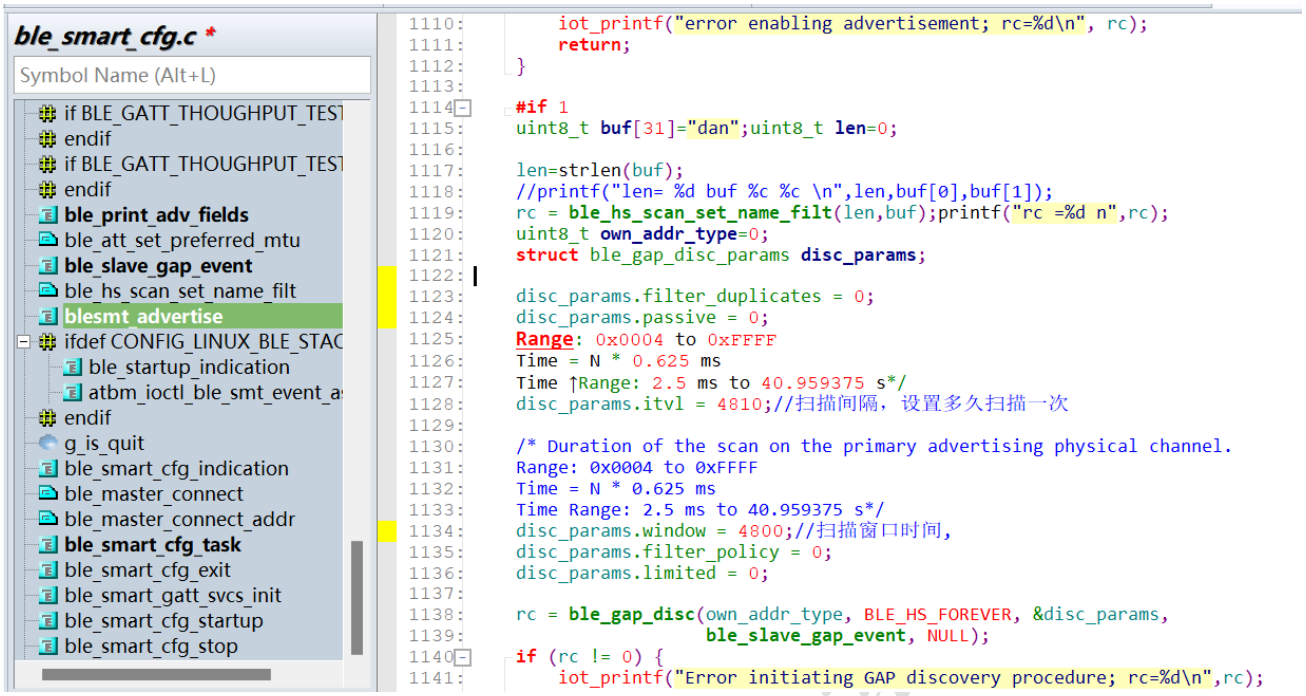
SRAM_CODE int ble_gap_disc_cancel(void)

(3) 设置扫描固定 ble 名字才上传到应用层回调函数中

需要在启动扫描前设置

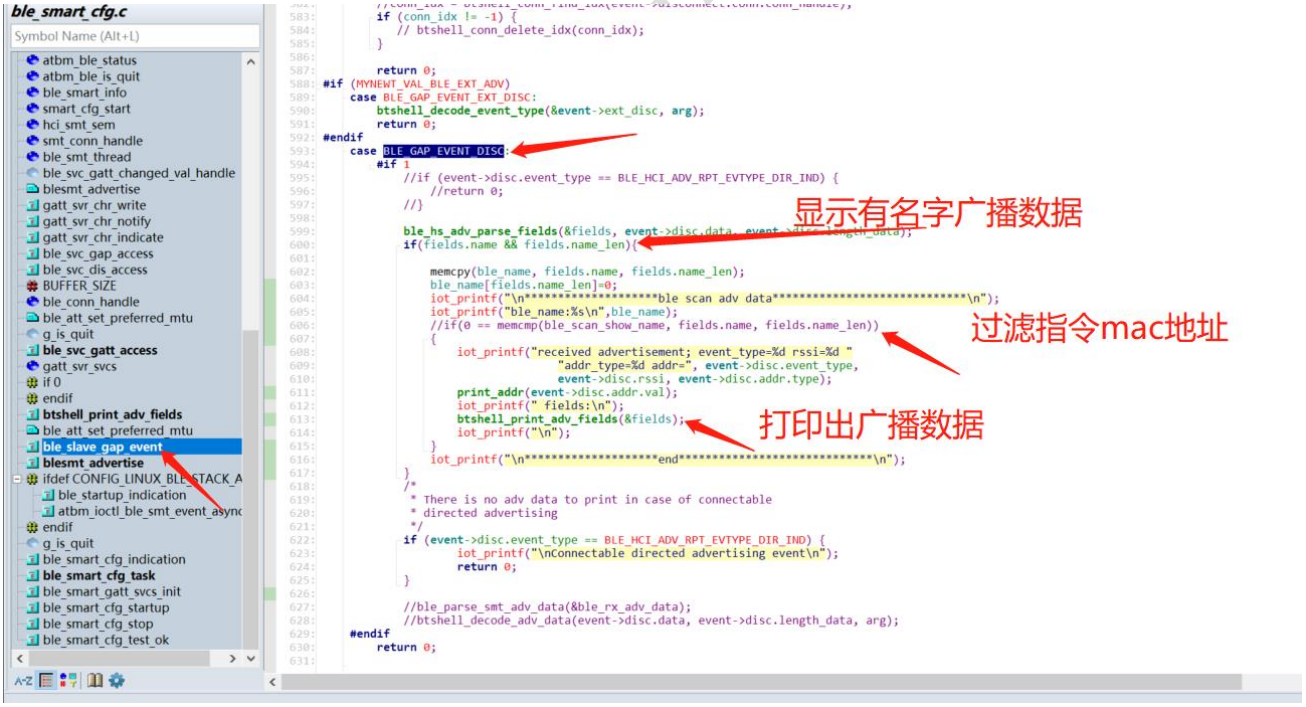
SRAM_CODE int ble_hs_scan_set_name_filt(uint8_t name_len, uint8_t *name)

AltoBeam Inc. Confidential



(4) 扫描回应包回调函数

BLE 扫描事件回调函数设置为 ble_slave_gap_event,事件回调为 BLE_GAP_EVENT_DISC。



(5) 显示收到广播设备 RSSI 与 Mac 地址

```
case BLE_GAP_EVENT_DISC:
    #if 1
        ble_hs_adv_parse_fields(&fields, event->disc.data, event->disc.length_data);
        if(fields.name && fields.name_len)
        {

            memcpy(ble_name, fields.name, fields.name_len);
            ble_name[fields.name_len]=0;
            iot_printf("\n**********************ble scan adv data*****************************\n");
            iot_printf("ble_name:%s\n",ble_name);
            iot_printf("rssi:%d\n",event->disc.rssi);//rssi信号强度
            //mac 地址是小端模式
            iot_printf("mac:%x:%x:%x:%x:%x:%x:\n",event->disc.addr.val[5],event->disc.addr.val[4],event->disc.addr.val[3],event->disc.addr.val[2],
            //if(0 == memcmp(ble_scan_show_name, fields.name, fields.name_len))
            {
                iot_printf("received advertisement; event_type=%d rssi=%d "
                           "addr_type=%d addr=", event->disc.event_type,
                           event->disc.rssi, event->disc.addr.type);
                print_addr(event->disc.addr.val);
                iot_printf(" fields:\n");
                btshell_print_adv_fields(&fields);
                iot_printf("\n");
            }
            iot_printf("\n**********************end*****************************\n");
        }
        /*
         * There is no adv data to print in case of connectable
         * directed advertising
         */
```

4.11 BLE 连接回调函数与连接成功后请求更新连接参数

ble_gap_update_params 使用

struct ble_gap_upd_params params;

params.itvl_min=60;////Range: 0x0006 to 0x0C80 Time = N * 1.25 msec 1*1.25=7.5ms

params.itvl_max=70;

params.latency=0;//减少或者设置为 0，那么每次连接事件中都需要回复 Master 的包，当然功耗会上升，但数据发送速度也会提高。

params.supervision_timeout=3000;//断线超时时间 Supervision timeout for the LE Link.Range: 0x000A to 0x0C80Time = connTimeout* 10 msecTime Range: 100 msec to 32 seconds

ble_gap_update_params(conn_handle, &params);//连接成功后才能请求。同不同意看 master 回应



4.12 请求更新 MTU 长度

在 4.10 章节上图有调用 api 地方，MTU 最大设置成 255。此 api 只能跟手机申请，实际手机支持 MTU 多少会发 MTU 更新事件过来（下图有事件回调地方）。有个别手机设置 MTU 不一定有 255 字节。BLE 4.2 版本理论上允许的最大 MTU 设置可以达到 255 字节，但在实际应用中，考虑到协议开销、设备兼容性以及可能的限制因素，有效用户数据的传输可能受限于一个较低的数值，如 238 字节（基于您提供的特定信息）或通过数据长度扩展达到的大约 251 字节。为了确定具体的最大 MTU 值，需要在实际设备间进行 MTU 协商，并考虑设备的实际支持能力以及是否启用数据长度扩展功能。在实际编程或应用开发过程中，应使用蓝牙栈提供的 API 或方法来进行 MTU 协商以获取最佳的传输效率。

4.13 实际发包 MTU 长度

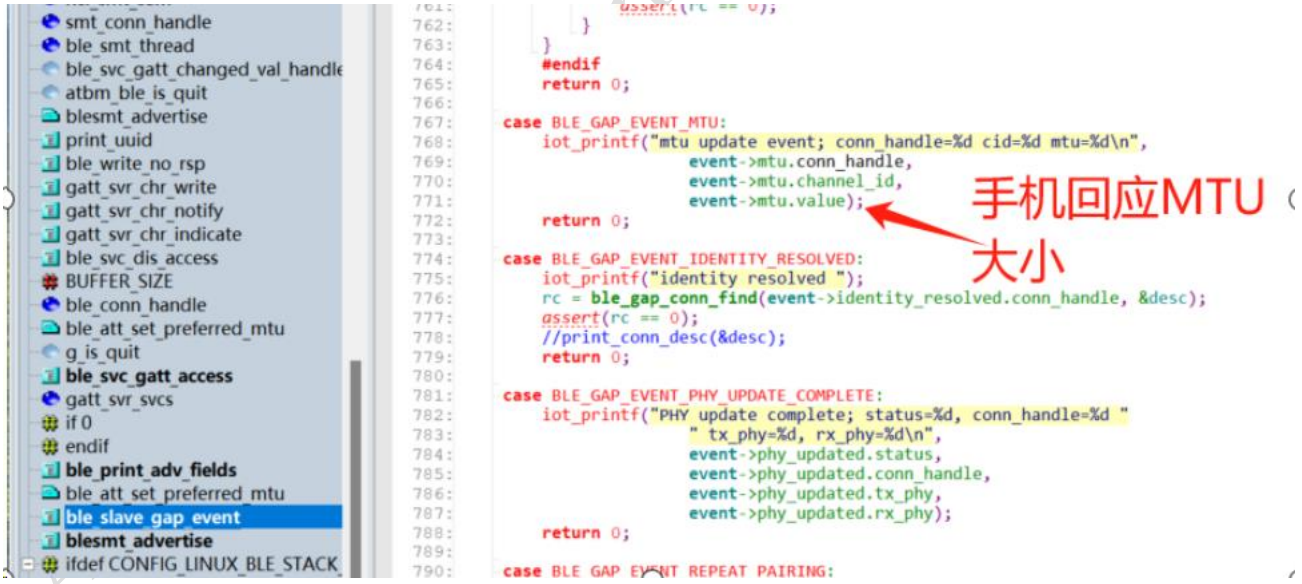
在 BLE 4.2 中，尽管 MTU（Maximum Transmission Unit，最大传输单元）可以被设置到一个较大的值，如您设置 255 字节，但实际用户数据的长度需要从 MTU 中减去协议 overhead（开销）。根据 BLE 的数据包结构，每个 ATT（Attribute Protocol）协议数据单元（PDU）都会包含一些固定的开销：

ATT Opcode: 占用 1 个字节，用于指示操作类型，比如读、写请求或响应等。

ATT Handle: 占用 2 个字节，用于标识属性的句柄。

因此，如果 MTU 设置为 255 字节，实际可用的用户数据长度为 MTU - (Opcode + Handle)，即 255 - (1 + 2)，计算得出为 252 字节。

请注意，这里没有考虑 ATT 协议可能引入的其他潜在开销，例如 ATT 协议中某些操作可能还需要额外的协议头或属性长度字段等。但在大多数基本通信场景下，上述计算方法提供了用户数据有效长度的一个大致估算。此外，实际应用中还应考虑任何特定实现或平台可能施加的额外限制。



注意：连接成功后必须从手机获取到 mtu 来决定最长发包长度，MTU 不能为固件值！因为个别手机 MTU 是 180 多或者其他值，如果发送单包最长超过手机回应 MTU 值大小会导致发送数据丢失各种问题。关系到不同手机兼容性问题。

4.14 在 BLE 连接回调函数中禁止发包和应用也不能在里面跑

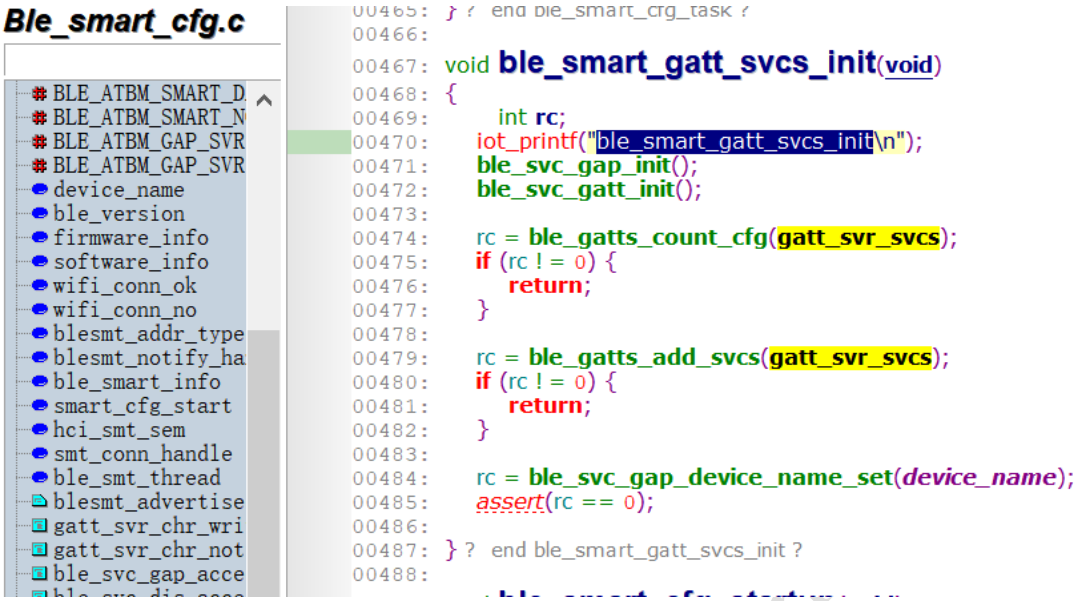
在连接回调函数里面发包会导致 BLE 事件没办法退出，导致 BLE 有断连接或者应用崩溃风险。

Confidential Document

请创建线程或者信号量传递到另个线程执行。

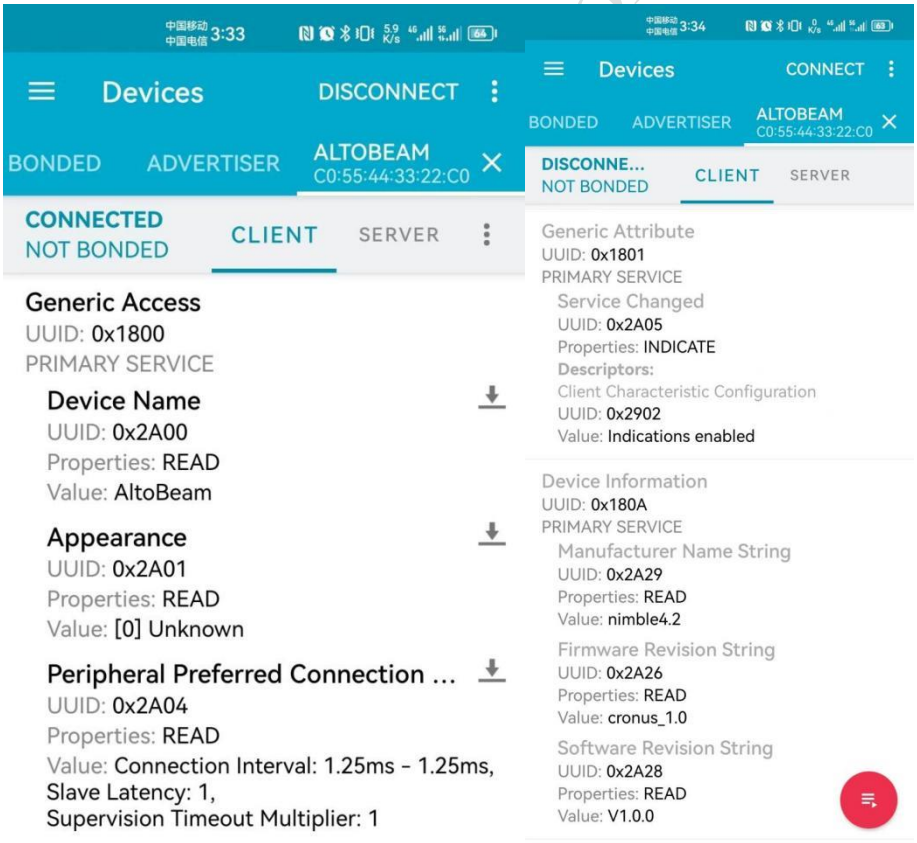
5 BLE GAP 与 GATT 初始化函数入口

Ble_smart_cfg.c



5.1 手机界面 BLE GAP 与 GATT 显示图

手机通过 nrf connect 连接上 BLE 后读取到属性表



5.2 GAP 与 GATT 数组初始化

Ble_svc_gap.c

```
00040: static char ble_svc_gap_name[BLE_SVC_GAP_NAME_MAX_LEN + 1] =
00041:     MYNEWT_VAL(BLE_SVC_GAP_DEVICE_NAME);
00042: static uint16_t ble_svc_gap_appearance = MYNEWT_VAL(BLE_SVC_GAP_APPEARANCE);
00043:
00044: static int
00045: ble_svc_gap_access(uint16_t conn_handle, uint16_t attr_handle,
00046:                    struct ble_gatt_access_ctxt *ctxt, void *arg);
00047:
00048: static const struct ble_gatt_svc_def ble_svc_gap_defs[] = {
00049:     {
00050:         /*** Service: GAP. */
00051:         .type = BLE_GATT_SVC_TYPE_PRIMARY,//Primary Service（首要服务项）
00052:         .uuid = BLE_UUID16_DECLARE(BLE_SVC_GAP_UUID16),//Service uuid
00053:         .characteristics = (struct ble_gatt_chr_def[]) { {
00054:             /*** Characteristic: Device Name. */
00055:             .uuid = BLE_UUID16_DECLARE(BLE_SVC_GAP_CHR_UUID16_DEVICE_NAME),//Characteristic uuid
00056:             .access_cb = ble_svc_gap_access,//处理Characteristic回调函数
00057:             .flags = BLE_GATT_CHR_F_READ |//权限只能手机端读
00058: #if MYNEWT_VAL(BLE_SVC_GAP_DEVICE_NAME_WRITE_PERM) >= 0
00059:                     BLE_GATT_CHR_F_WRITE |
00060:                     MYNEWT_VAL(BLE_SVC_GAP_DEVICE_NAME_WRITE_PERM) |
00061: #endif
00062:                     0,
00063:         }, {
00064:             /*** Characteristic: Appearance. */
00065:             .uuid = BLE_UUID16_DECLARE(BLE_SVC_GAP_CHR_UUID16_APPEARANCE),
00066:             .access_cb = ble_svc_gap_access,
00067:             .flags = BLE_GATT_CHR_F_READ |
00068: #if MYNEWT_VAL(BLE_SVC_GAP_APPEARANCE_WRITE_PERM) >= 0
00069:                     BLE_GATT_CHR_F_WRITE |
00070:                     MYNEWT_VAL(BLE_SVC_GAP_APPEARANCE_WRITE_PERM) |
00071: #endif
00072:                     0,
00073:         }, {
00074: #if PPCP_ENABLED
00075:             /*** Characteristic: Peripheral Preferred Connection Parameters. */
00076:             .uuid =
00077:                 BLE_UUID16_DECLARE(BLE_SVC_GAP_CHR_UUID16_PERIPH_PREF_CONN_PARAMS),
00078:             .access_cb = ble_svc_gap_access,
00079:             .flags = BLE_GATT_CHR_F_READ,
00080:         }, {
00081: #endif
00082: #if MYNEWT_VAL(BLE_SVC_GAP_CENTRAL_ADDRESS_RESOLUTION) >= 0
00083:             /*** Characteristic: Central Address Resolution. */
00084:             .uuid = BLE_UUID16_DECLARE(BLE_SVC_GAP_CHR_UUID16_CENTRAL_ADDRESS_RESOLUTION),
00085:             .access_cb = ble_svc_gap_access,
```

5.3 GAP 与 GATT 事件回调函数接口：

Ble_svc_gap.c

```
00178:
00179: static int
00180: ble_svc_gap_access(uint16_t conn_handle, uint16_t attr_handle,
00181:                    struct ble_gatt_access_ctxt *ctxt, void *arg)
00182: {
00183:     uint16_t uuid16;
00184: #if MYNEWT_VAL(BLE_SVC_GAP_CENTRAL_ADDRESS_RESOLUTION) >= 0
00185:     uint8_t central_ar = MYNEWT_VAL(BLE_SVC_GAP_CENTRAL_ADDRESS_RESOLUTION);
00186: #endif
00187: #if PPCP_ENABLED
00188:     uint16_t ppcp[4] = {
00189:         htole16(MYNEWT_VAL(BLE_SVC_GAP_PPCP_MIN_CONN_INTERVAL)),
00190:         htole16(MYNEWT_VAL(BLE_SVC_GAP_PPCP_MAX_CONN_INTERVAL)),
00191:         htole16(MYNEWT_VAL(BLE_SVC_GAP_PPCP_SLAVE_LATENCY)),
00192:         htole16(MYNEWT_VAL(BLE_SVC_GAP_PPCP_SUPERVISION_TMO))
00193:     };
00194: #endif
00195:     int rc;
00196:
00197:     uuid16 = ble_uuid_u16(ctxt->chr->uuid);
00198:     assert(uuid16 != 0);
00199:     iot_printf("ble_svc_gap_access uuid%x\n", uuid16);
00200:     switch (uuid16) {
00201:     case BLE_SVC_GAP_CHR_UUID16_DEVICE_NAME:
00202:         if (ctxt->op == BLE_GATT_ACCESS_OP_READ_CHR) {
00203:             rc = ble_svc_gap_device_name_read_access(ctxt);
00204:         } else if (ctxt->op == BLE_GATT_ACCESS_OP_WRITE_CHR) {
00205:             rc = ble_svc_gap_device_name_write_access(ctxt);
00206:         } else {
00207:             assert(0);
00208:         }
00209:         return rc;
00210:
00211:     case BLE_SVC_GAP_CHR_UUID16_APPEARANCE:
00212:         if (ctxt->op == BLE_GATT_ACCESS_OP_READ_CHR) {
```

5.4 INDICATE 和 NOTIFY 介绍、配置与使用

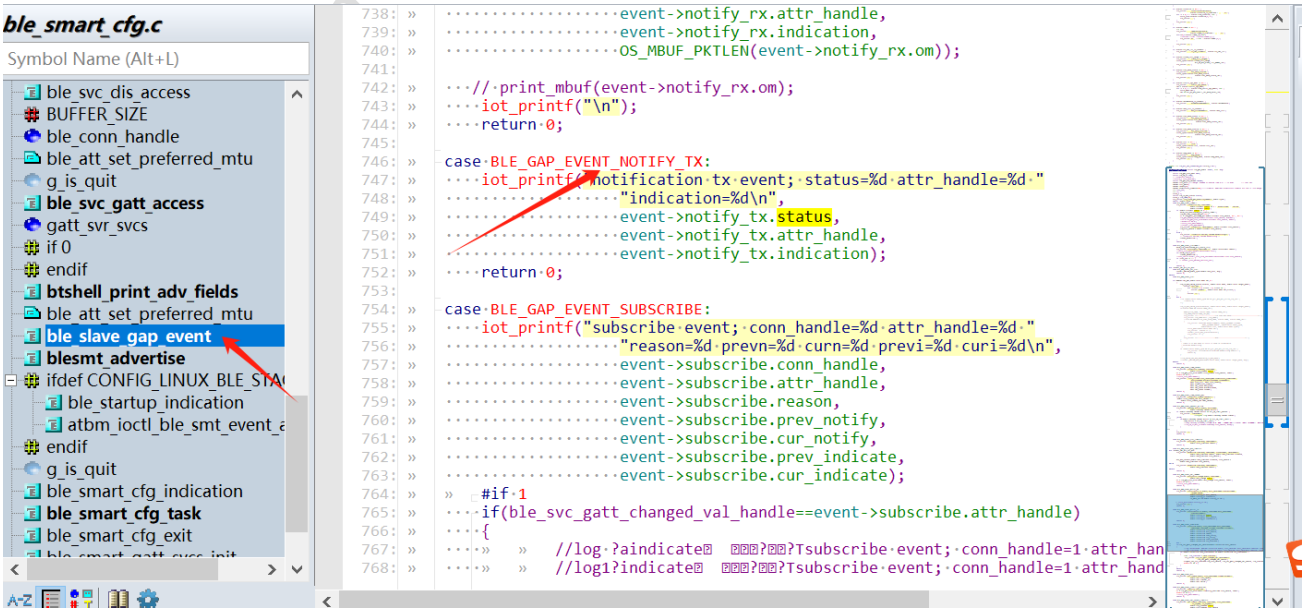
BLE 的 Notification 和 Indication 是 GATT 协议中用于订阅 GATT Server 提供的数据的方式。它们在传输数据时有一些区别。Notification 是一种未经确认的消息或更新，而 Indication 是一种经过确认的消息或更新。当 GATT Server 上的相关数据更新时，会发送这些 Notification 和 Indication。你需要"订阅"你想要接收通知或指示的数据。

从某种意义上说，Notification 和 Indication 类似于 TCP 和 UDP 数据包。TCP 要求在发送数据时，接收方通过发送确认数据包来确认已接收到数据。而 UDP 则只是发送数据，不关心是否实际上已确认接收。在这个意义上，Indication 类似于 TCP，而 Notification 类似于 UDP。

使用 Notification 时，你可以在发送之前将一定数量的数据包排队，一旦缓冲区满了，你会从 ble_gattc_notify_custom()函数中收到一个 BLE_HS_ENOMEM 或者-1 错误代表 buf 已满，需要延时 100ms 重发一次。



当一个数据包通过链路层发送后，你会收到一个 ble_slave_gap_event 事件回调函数 BLE_GAP_EVENT_NOTIFY_TX 事件，此时你可以继续排队更多的数据包。



Indication 比 Notification 慢，因为你发送一个 Indication（你的应用层将对端收到后才回复 ACK）。如果你需要更高的传输速率，建议使用 Notification。

至于为什么有人说 Notification 不可靠，可能是因为 Notification 没有接收方的确认机制，所以无法保证数据是否被成功接收。而 Indication 则需要接收方发送应用层 ACK 来确认数据的接收。

总结一下，Notification 和 Indication 是用于订阅 GATT Server 数据的方式，它们在传输数据时有一些区别。Notification 是未经确认的消息或更新，而 Indication 是经过确认的消息或更新。使用 Notification 可以实现较高的传输速率，而 Indication 则需要接收方发送应用层 ACK 来确认数据的接收。

Notify 数据频繁发包重传 demo：

```
187: static int ble_test_task(void* param)
188: {
189:     int rc,i,cnt;
190:     iot_printf(" send %d\n",sizeof(test_f));
191:     rc=gatt_svr_chr_notify(ble_conn_handle, blesmt_notify_handle, test_f, sizeof(test_f));
192:     cnt=0;
193:     if(atbm_ble_status == 0){
194:         break;
195:     }
196:     while(rc && (cnt<50)&&(atbm_ble_status))
197:     {
198:         cnt++;
199:         iot_printf(" wait rc=%d\n",rc);
200:         usleep(50*1000);
201:         rc=gatt_svr_chr_notify(ble_conn_handle, blesmt_notify_handle, test_f, sizeof(test_f));
202:         if(rc)
203:         {
204:             iot_printf(" error %d \n",cnt);
205:         }
206:     }
207:     return 0;
208: } « end ble_test_task »
209:
```

5.5 手机订阅 INDICATE



ble 收到事件通知：

```
s_event14
subscribe event; conn_handle=1 attr_handle=10 reason=1 prevn=0 curn=0 previ=0 curi=1
```

attr_handle=10 对应下面初始化 ble_svc_gatt_changed_val_handle 值。初始化后 ble_svc_gatt_changed_val_handle 值会等于 10。

当 previ=0 与 curi=1 时，手机订阅该 indicate

当 previ=1 与 curi=0 时，手机取消订阅该 indicate

5.6 INDICATE 初始化使用

ble_svc_gatt.c

```
#include "services/gatt/ble_svc_gatt.h"

static uint16_t ble_svc_gatt_changed_val_handle;
static uint16_t ble_svc_gatt_start_handle;
static uint16_t ble_svc_gatt_end_handle;

static int
ble_svc_gatt_access(uint16_t conn_handle, uint16_t attr_handle,
                    struct ble_gatt_access_ctxt *ctxt, void *arg);

static const struct ble_gatt_svc_def ble_svc_gatt_defs[] = {
    {
        /*** Service: GATT */
        .type = BLE_GATT_SVC_TYPE_PRIMARY,//首 要 服 务 项
        .uuid = BLE_UUID16_DECLARE(BLE_GATT_SVC_UUID16),
        .characteristics = (struct ble_gatt_chr_def[]) { {
            .uuid = BLE_UUID16_DECLARE(BLE_SVC_GATT_CHR_SERVICE_CHANGED_UUID16),
            .access_cb = ble_svc_gatt_access,//接 收 到 数 据 回 调 函 数
            .val_handle = &ble_svc_gatt_changed_val_handle,
            .flags = BLE_GATT_CHR_F_INDICATE,//权 限
        }, {
            0, /* No more characteristics in this service. */
        } },
    },

    {
        0, /* No more services. */
    },
};
```

GATT 0x2a05 打开 indication enable 事件回调时进行数据传输示范



(1) 收到手机订阅 indicate 事件后，创建线程发 255 包。

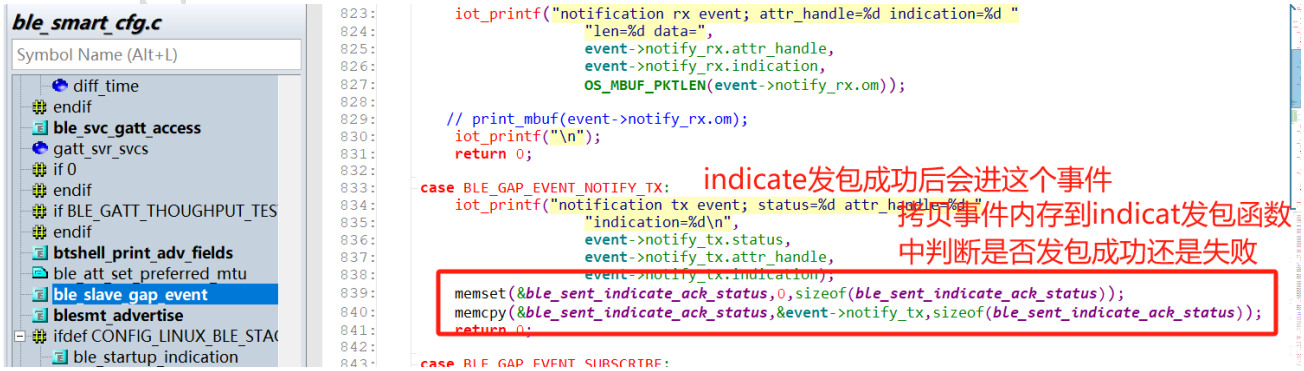


(2) 线程发包 demo



(3) Indicate 发包成功进该函数

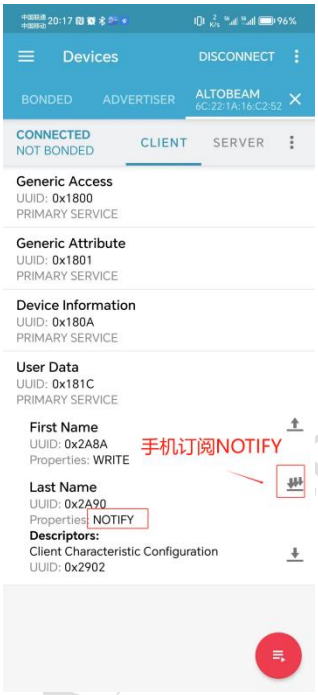
拷贝发包状态。status 值为 14（BLE_HS_EDONE）就是发包成功



(4) Indicate 发包 api



5.7 手机订阅 NOTIFY



BLE 收到事件：

```
msh />s_event14
subscribe event; conn_handle=1 attr_handle=23 reason=1 prevn=0 curn=1 previ=0 curi=0
```

AltoBeam Inc. Confidential

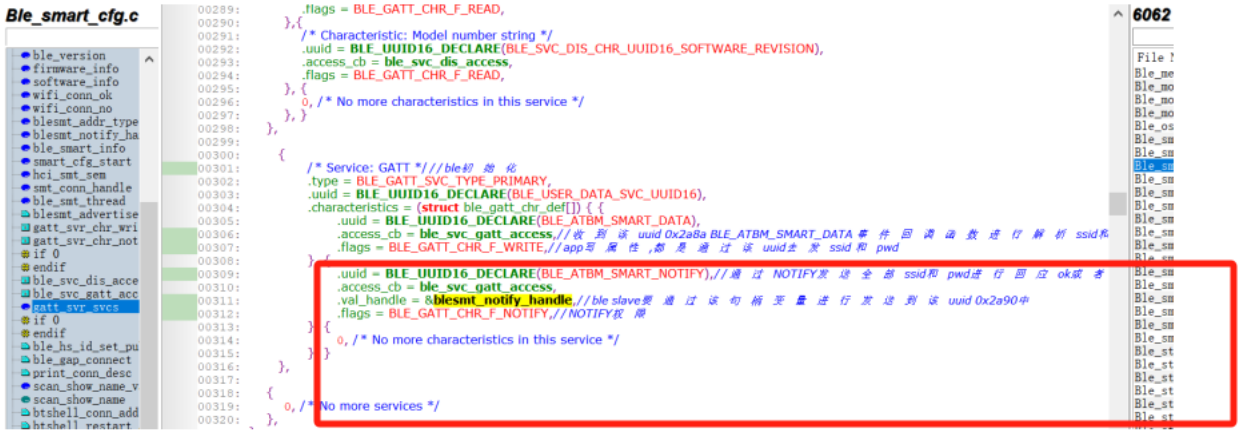
attr_handle=23 对应初始化 blesmt_notify_handle 值。初始化后 blesmt_notify_handle 值会等于 23。

当 prevn=0 与 curn=1 时，手机订阅该 notify

当 prevn=1 与 curn=0 时，手机取消订阅该 notify

5.8 NOTIFY 初始化与使用

代码初始化：



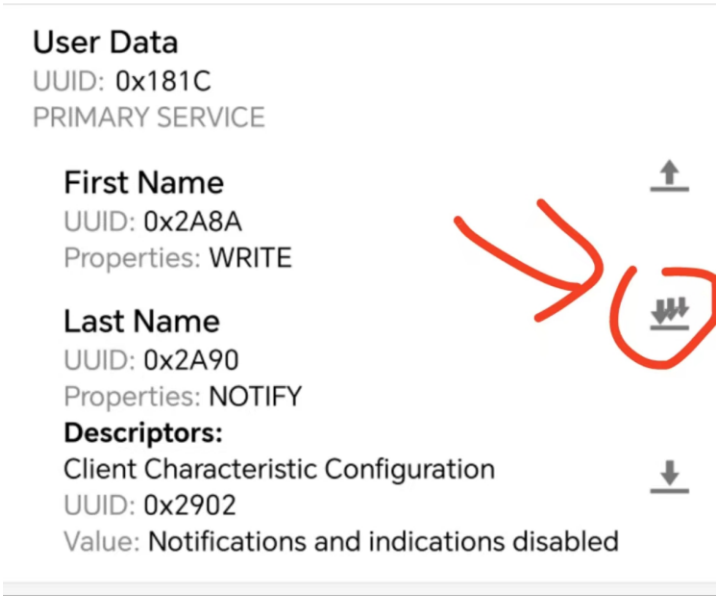
BLE 收到 ssid 和 pwd 后进行 notify 发数据手机端的 uuid 0x2A90 中



如果需要发送大数据包请参考 cmd send-notify 命令写法，建立一个线程发包；

不能卡住 event 回调函数中卡住导致 ble 使用异常！

使用前请将手机 notify 订阅打开：



atbm_cli btshell send-notify interval=50 number=250 len=254

Interval：单位是 ms

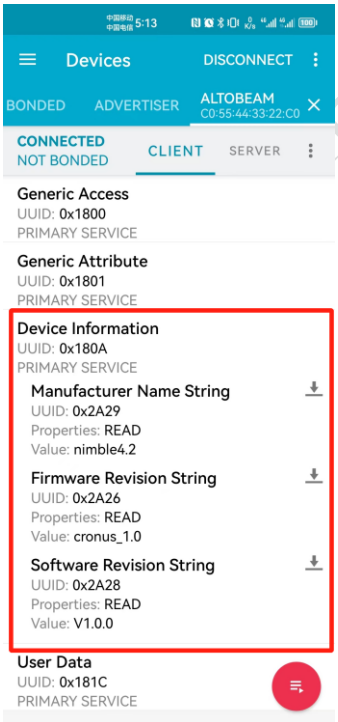
Number：发送多少包

Len：一包发送多少字节

(6461 指令输入不需要 atbm_cli，请直接输入 btshell send-notify interval=50 number=250 len=254)

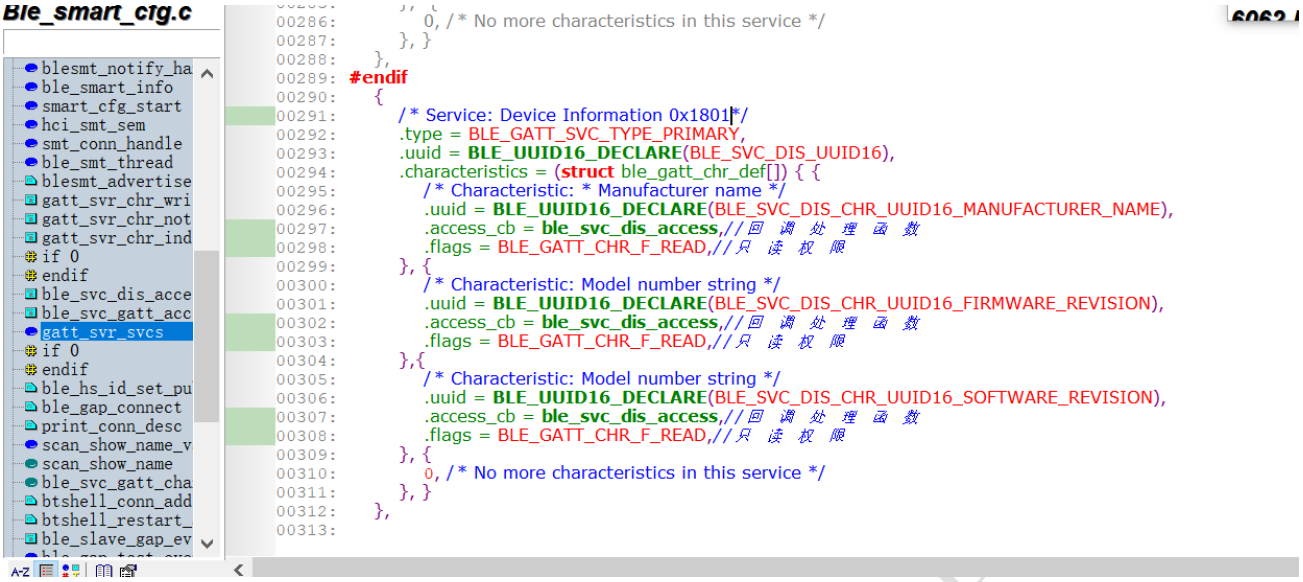
5.9 Device Information READ 属性初始化与使用

Uuid 0x180A Device Information Service: Device Information 手机读取图:



Confidential Document

代码配置如图:



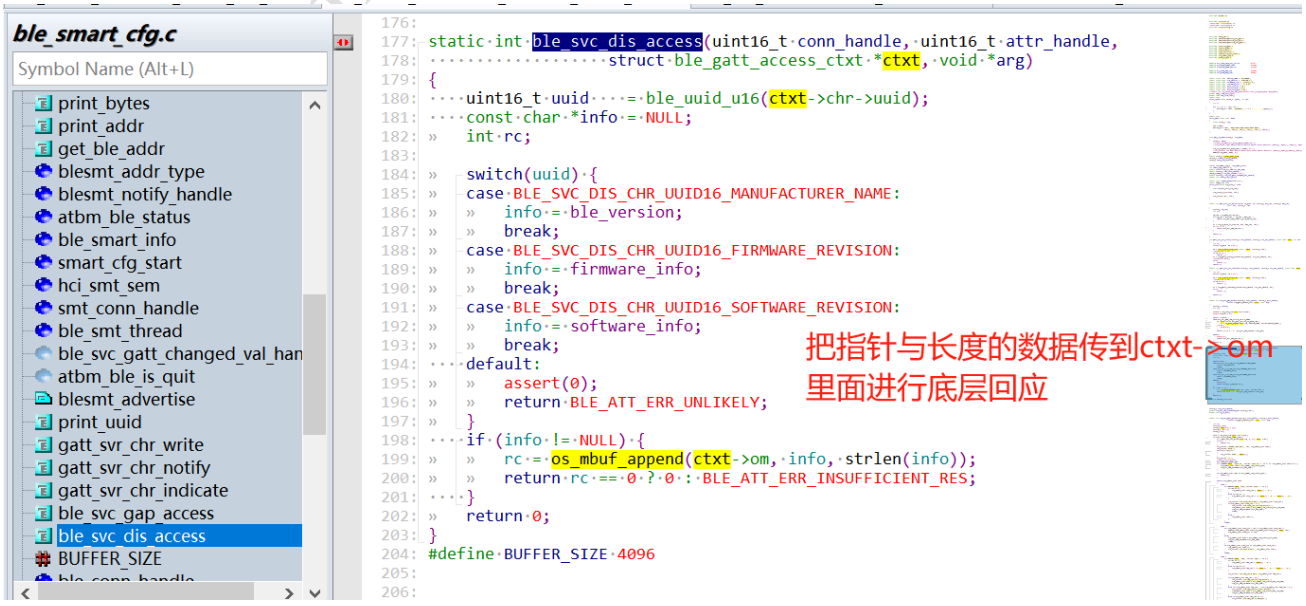
Uuid 0x180A Device Information 下面 Characteristic:

Manufacturer name Model number string Model number string 回应

(1) BLE READ 使用

可以参考以下事件回调函数实现

```
34:
35: static const char *device_name = "AltoBeam";
36: static const char *ble_version = "nimble4.2";
37: static const char *firmware_info = "cronus_1.0";
38: static const char *software_info = "V1.0.0";
39: static const char *wifi_conn_ok = "ok";
40: static const char* wifi_conn_no = "no";
41: static const char* wifi_conn_wait ="wt";
42: //static int ble_parse_smt_adv_data(struct ioctl_ble_adv_data *adv_data);
43: extern sem_t sem_sock_sync;
44: extern char cmd_line[1600];
45: static void
46: print_bytes(const uint8_t *bytes, int len)
47: {
48: int i;
```



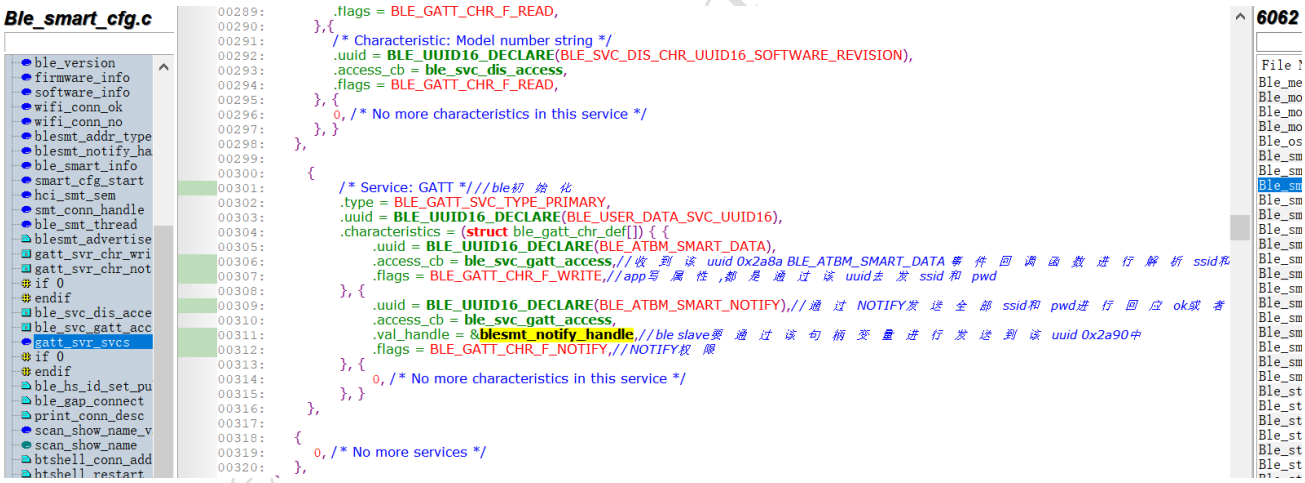

```
176:
177: static int ble_svc_dis_access(uint16_t conn_handle, uint16_t attr_handle,
178:                              struct ble_gatt_access_ctxt *ctxt, void *arg)
179: {
180:     uint16_t uuid    = ble_uuid_u16(ctxt->chr->uuid);
181:     const char *info = NULL;
182:     int rc;
183:
184:     switch(uuid) {
185:     case BLE_SVC_DIS_CHR_UUID16_MANUFACTURER_NAME:
186:         info = ble_version;
187:         break;
188:     case BLE_SVC_DIS_CHR_UUID16_FIRMWARE_REVISION:
189:         info = firmware_info;
190:         break;
191:     case BLE_SVC_DIS_CHR_UUID16_SOFTWARE_REVISION:
192:         info = software_info;
193:         break;
194:     default:
195:         assert(0);
196:         return BLE_ATT_ERR_UNLIKELY;
197:     }
198:     if (info != NULL) {
199:         rc = os_mbuf_append(ctxt->om, info, strlen(info));
200:         return rc == 0 ? 0 : BLE_ATT_ERR_INSUFFICIENT_RES;
201:     }
202:     return 0;
203: }
204: #define BUFFER_SIZE 4096
205:
206:
```

5.10 WRITE 初始化与使用

蓝牙配网 demo 手机界面读取图



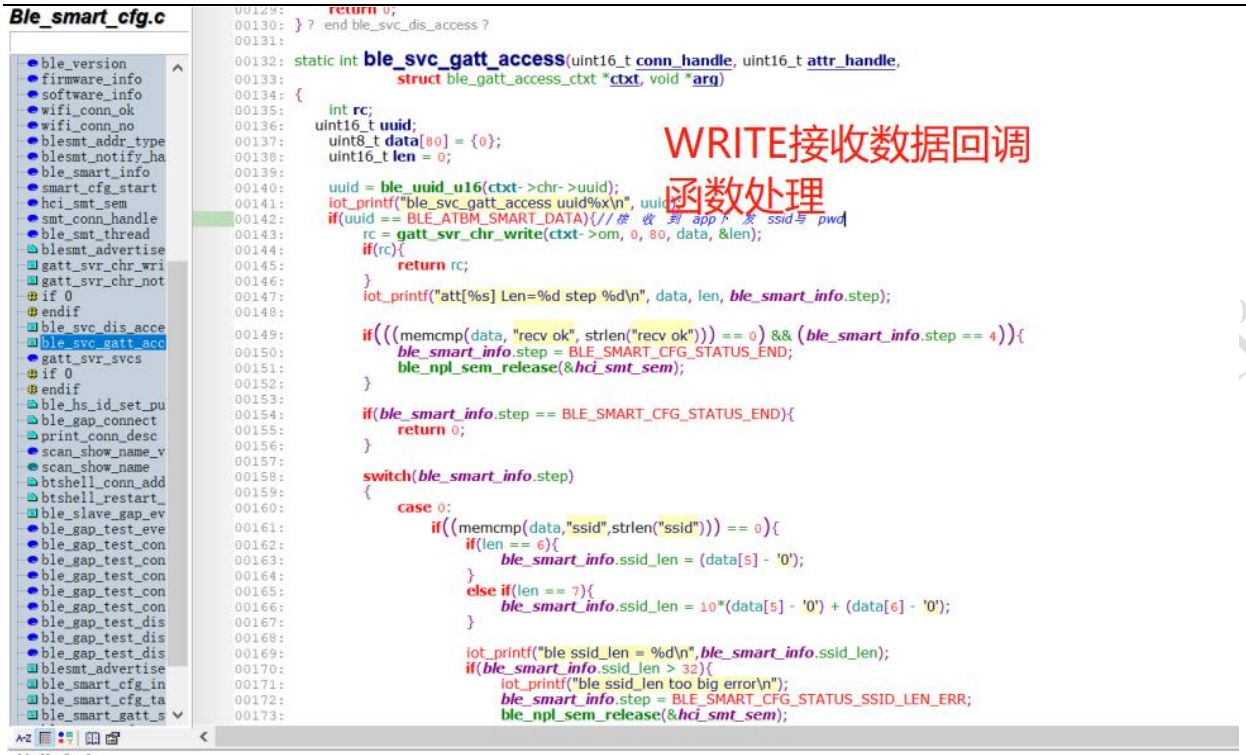
(1) 5.6.1 WRITE 初始化



(2) 手机通过 WRITE 发数据给设备，设备收数据处理

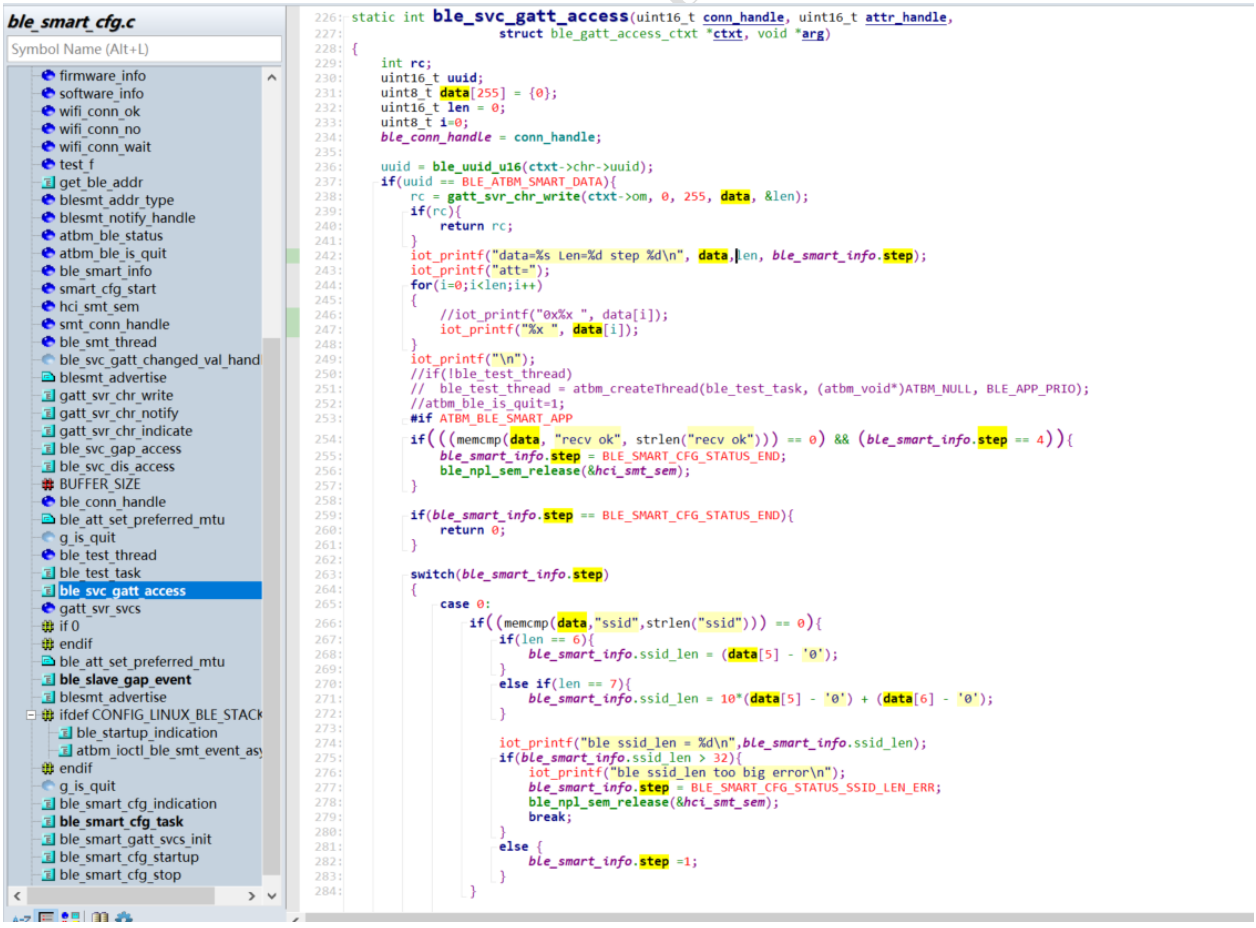
BLE 配网解析收到 APP 下发 SSID 与 PWD 代码事件回调函数

AltoBeam Confidential Document

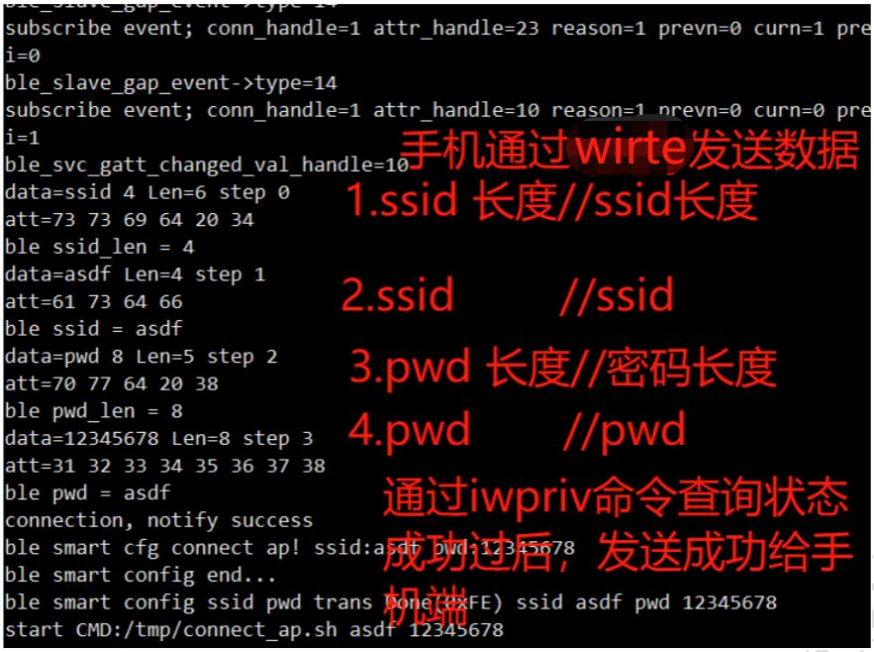


(3) Ble 配网 demo 数据交互流程

手机端通过 uuid 0x2A8A write 数据过来，在下面函数中解析。



(4) BLE 配网收发数据的 log 显示



6 移植自定义应用代码与退出 ble 线程

6.1 客户应用代码建议在我们基础上修改

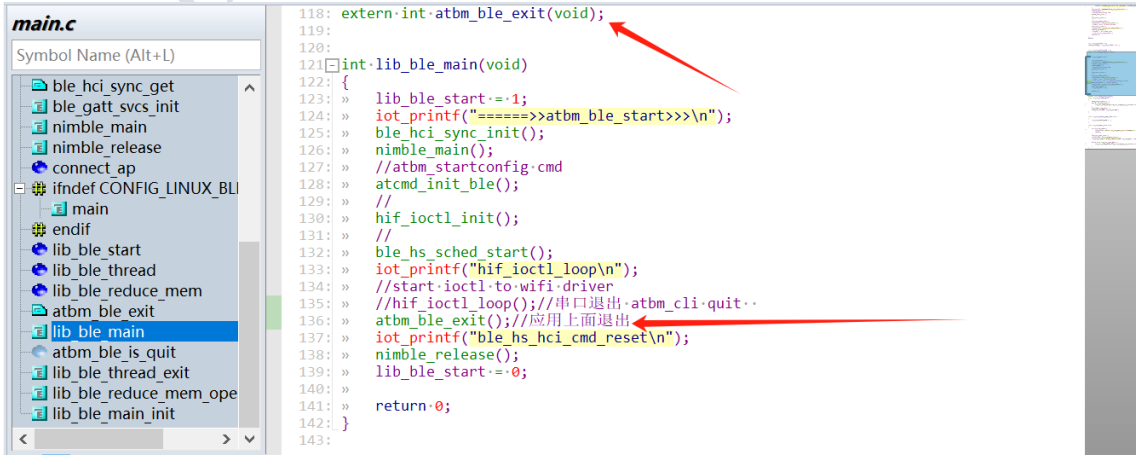
在 ble_smart_cfg.h 头文件中屏蔽该宏#define ATBM_BLE_SMART_APP 1

6.2 退出 ble 线程方式有两种方式(6461 暂未支持)

(1) 指令退出：atbm_cli quit。

(2) 函数退出：lib_ble_thread_exit()

在不能在协议栈里面调用，要在其他线程调用该 lib_ble_thread_exit()退出，需要用 atbm_ble_exit()函数替代 hif_ioctl_loop()



Beam Inc Confidential Document

```
int main()
{
    while(1)
    {
        lib_ble_main_init();
        iot_printf("lib_ble_main\n");
        sleep(10);
        lib_ble_thread_exit();
        iot_printf("lib_ble_thread_exit\n");
        sleep(10);

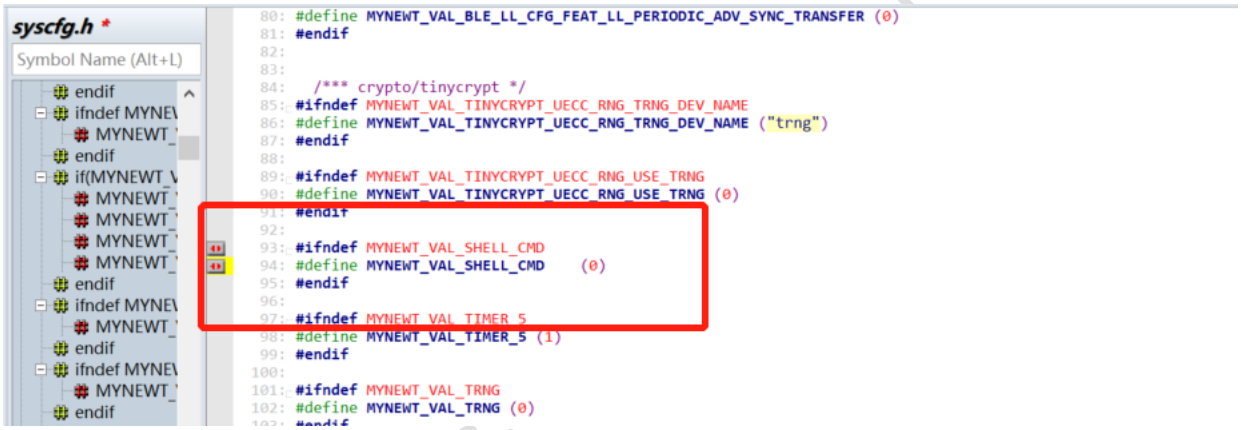
    }
}
```

6.3 缩减 ble 代码大小

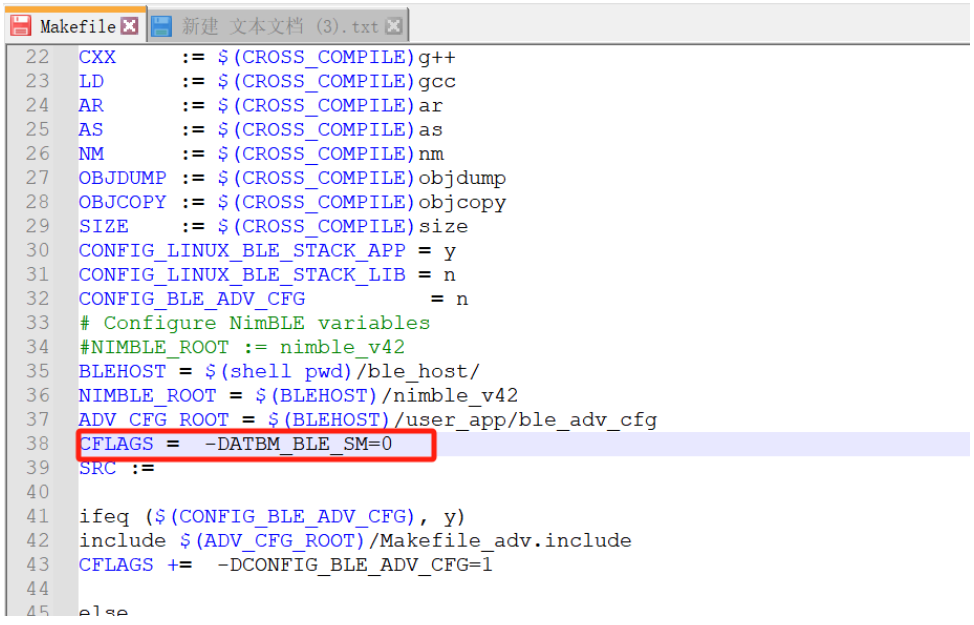
配置选项在\ble_host\os\linux_app\include\syscfg\syscfg.h

(1) 关闭 bt_shell 测试代码

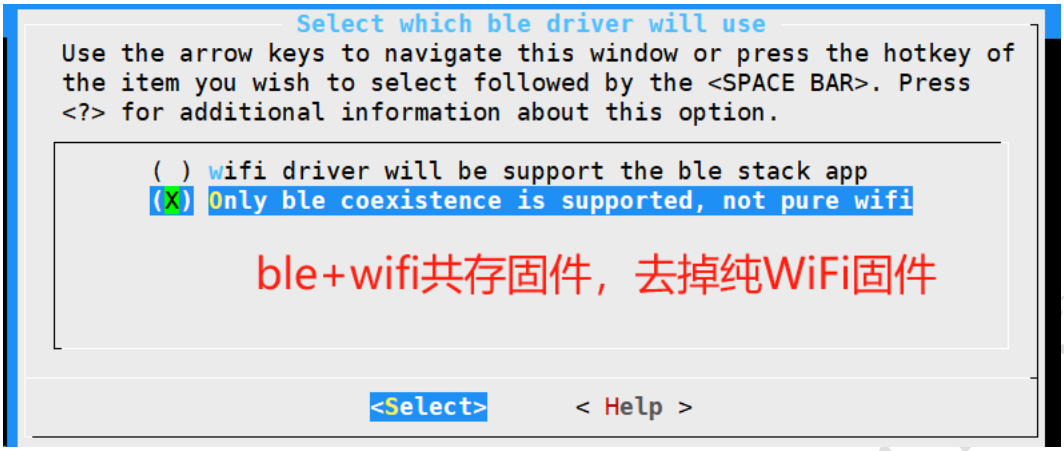
把该宏置 MYNEWT_VAL_SHELL_CMD (0)



(2) 用不到 SM 加密可以去掉，能省十几 k(6461 暂未支持)



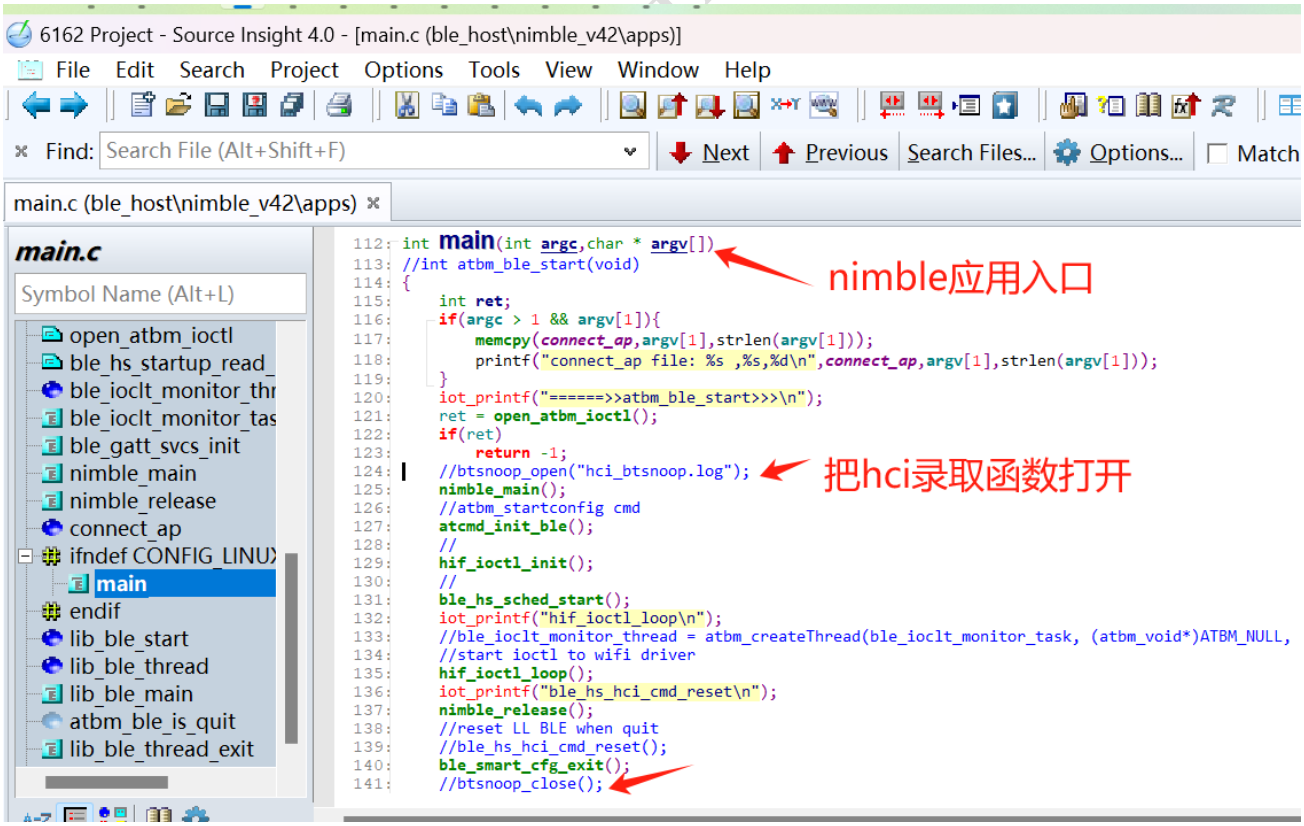
6.4 缩小驱动大小，单独使用 WIFI+BLE 固件，去掉完整 WiFi 固件。(linux 专用)



7 导出 HCI 数据分析

把 btsnoop_open("hci_btsnoop.log");和 btsnoop_close();屏蔽打开

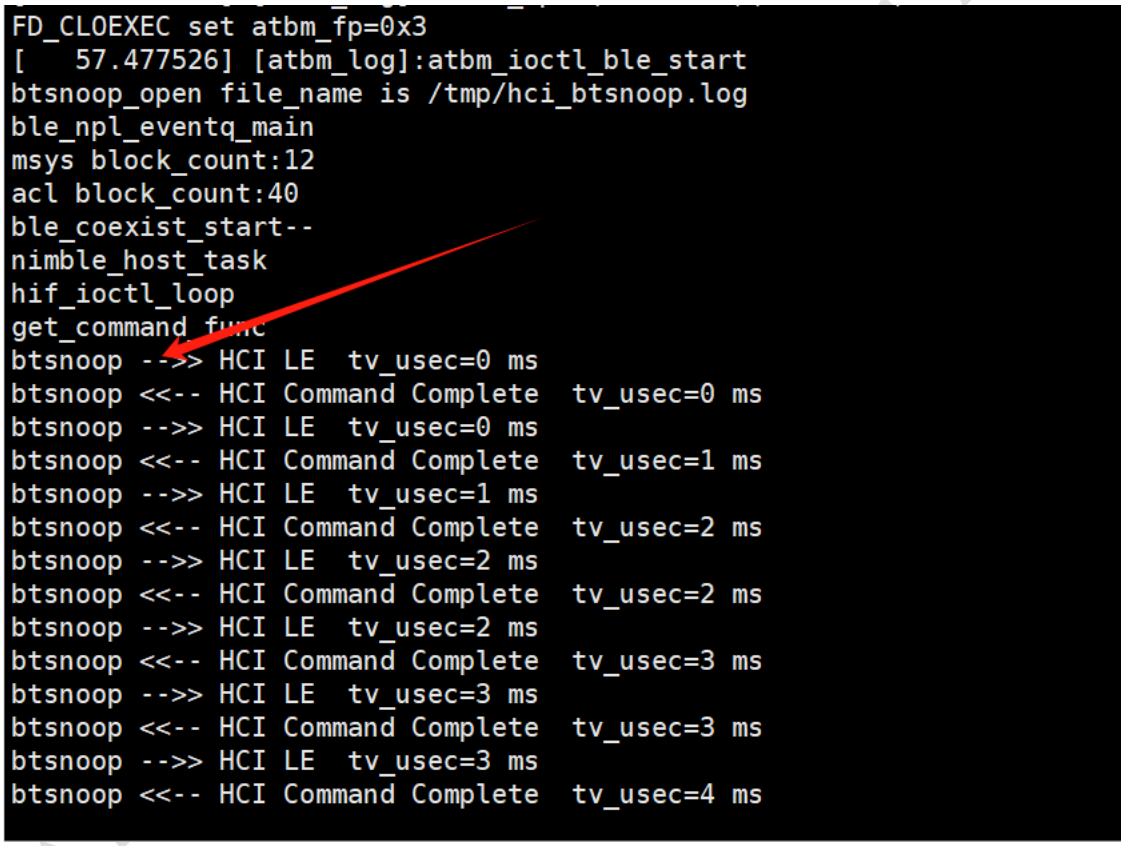
nimble 应用入口



打包成库函数：



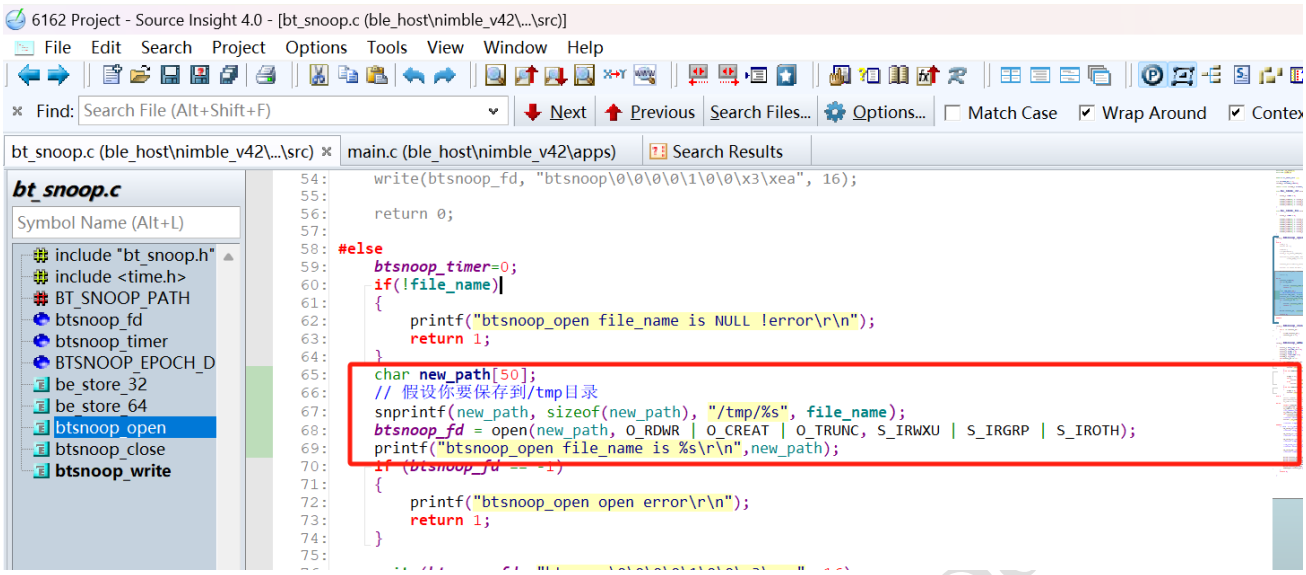
打开 hci log 后会有很多 log 打印



Hci 文件生成会在 nimble 文件的存储目录

hci_btsnoop.log //录 hci 文件名，请把该文件复制给我们分析

如果不想保存在当前目录请按照下图修改保存路径



char new_path[50];

// 假设你要保存到/tmp 目录

snprintf(new_path, sizeof(new_path), "/tmp/%s", file_name);

btsnoop_fd = open(new_path, O_RDWR | O_CREAT | O_TRUNC, S_IRWXU | S_IRGRP | S_IROTH);

8 错误码信息

8.1 以下是 API 返回值错误码原因

```
#define BLE_HS_EAGAIN            1
#define BLE_HS_EALREADY          2
#define BLE_HS_EINVAL            3
#define BLE_HS_EMSGSIZE          4
#define BLE_HS_ENOENT            5
#define BLE_HS_ENOMEM            6
#define BLE_HS_ENOTCONN          7
#define BLE_HS_ENOTSUP           8
#define BLE_HS_EAPP              9
#define BLE_HS_EBADDATA          10
#define BLE_HS_EOS               11
#define BLE_HS_ECONTROLLER       12
#define BLE_HS_ETIMEOUT          13
#define BLE_HS_EDONE             14
#define BLE_HS_EBUSY             15
#define BLE_HS_EREJECT           16
#define BLE_HS_EUNKNOWN          17
#define BLE_HS_EROLE             18
#define BLE_HS_ETIMEOUT_HCI      19
#define BLE_HS_ENOMEM_EVT        20
#define BLE_HS_ENOADDR           21
#define BLE_HS_ENOTSYNCED        22
#define BLE_HS_EAUTHEN           23
#define BLE_HS_EAUTHOR           24
#define BLE_HS_EENCRYPT          25
#define BLE_HS_EENCRYPT_KEY_SZ   26
#define BLE_HS_ESTORE_CAP        27
#define BLE_HS_ESTORE_FAIL       28
#define BLE_HS_EPREEMPTED        29
#define BLE_HS_EDISABLED         30
```

8.2 BLE 连接断开错误码解析

错误代码用于指示错误的原因，当出现一些异常情况时会出现一些对应的错误码。错误代码的大小为 8 个字节，下图代码低字节。



未知 HCI 命令(0x01)

未知 HCI 命令错误代码表示控制器不理解主机发送的 HCI 命令数据包操作码。所给出的操作码可能不对应于本文档中指定的任何操作码，也不对应于任何特定于供应商的操作码，或者该命令可能尚未实现。

未知连接标识符(0x02)

未知连接标识符错误代码表示从主机发送了一个命令，该命令应该识别一个连接，但该连接不存在或没有识别正确的连接类型。

硬件故障(0x03)

硬件故障错误代码向主机表明，控制器中的某些故障的方式无法用任何其他错误代码来描述。此错误代码所隐含的含义依赖于实现。

PAGE 超时(0x04)

页面超时错误代码表示由于页面超时配置参数而导致页面超时。

身份验证失败(0x05)

身份验证失败错误代码表示由于配对或身份验证过程中的结果不正确，导致配对或身份验证失败。这可能是由于一个错误的密码或链接键。

PIN 或密钥丢失(0x06)

当配对因 PIN 丢失而失败，或认证因密钥丢失而失败时，将使用 PIN 或密钥丢失的错误代码。

内存容量超过(0x07)

内存容量超过错误代码向主机表明控制器内存不足，无法存储新参数。

连接超时(0x08)

连接超时错误代码表示给定连接的链路监视超时已超时，或者给定广播的同步超时已过期。

AltoBeam Inc. Confidential Document

超过连接限制(0x09)

连接限制超过错误代码表示尝试创建另一个连接失败，因为控制器已经达到其可以支持的连接数量的限制。设备可以支持的连接数取决于实现。

对设备的同步连接限制超过(0x0A)

对设备的同步连接限制超过错误代码表示控制器已达到可以达到对设备的同步连接数量的限制。设备可以支持的同步连接数取决于实现。

连接已存在(0x0B)

连接已存在错误代码表明，当已经连接到该设备，且不允许对同一设备进行多个连接时，已尝试创建到该设备的新连接。

不允许命令(0x0C)

不允许命令的错误代码表示无法执行所请求的命令，因为控制器目前处于此时无法处理此命令的状态。错误代码未知 HCI 命令有效的命令操作码。

由于有限的资源而被拒绝的连接(0x0D)

由于资源有限而被拒绝的连接错误代码表明一个连接由于资源有限而被拒绝。

由于安全原因而被拒绝的连接(0x0E)

由于安全原因而被拒绝的连接错误代码表明一个连接由于未满足安全要求而被拒绝，如身份验证或配对。

连接被拒绝是由于不可接受的 BD_ADDR(0x0F)

由于不可接受的BD_ADDR 错误代码而被拒绝的连接表明一个连接被拒绝了，因为该设备不接受 BD_ADDR。这可能是因为该设备将只接受来自特定 BD_ADDRs 的连接。

连接接受超时超过(0x10)

连接接受超时超过错误代码表明此连接尝试已超过连接接受超时。

不支持的特性或参数值(0x11)

AltoBeam Inc. Confidential Document

不支持的特性或参数值错误代码表示不支持 HCI 命令中的特性或参数值。此错误代码不得在 LMP 中用于 PDU。

无效的 HCI 命令参数(0x12)

无效的 HCI 命令参数错误代码表明至少有一个 HCI 命令参数无效。

当参数总长度无效时，应使用。

•一个命令参数的类型无效。

•中的一个连接标识符与对应的事件不匹配。

•当一个参数需要为偶数时，它是奇数的。

•中的一个参数超出了指定的范围。

•中的两个或多个参数值的值不一致。

注意：例如，当需要 ACL 连接句柄时，使用 SCO 连接句柄时，可能是无效的类型。

远程用户终止连接(0x13)

远程用户终止连接错误代码表示远程设备上的用户要么终止了连接，要么停止了广播数据包。

由于低资源而导致的远程设备终止连接(0x14)

由于资源不足导致的远程设备终止连接错误代码表明远程设备由于资源不足而终止了连接。

远程设备终止连接(0x15)

由于关机错误而导致的远程设备终止连接代码表明远程设备由于设备即将关机而终止连接。

由本地主机终止的连接(0x16)

由本地主机终止的连接错误代码表示本地设备终止连接、终止与广播器的同步，或终止广播数据包。

重复尝试(0x17)

重复尝试错误代码表明控制器不允许身份验证或配对过程，因为自上次身份验证或配对尝试失败后经过的时间太少。

不允许配对（0x18）

AltoBeam Inc. Confidential Document

不允许配对的错误代码表示设备不允许配对。例如，当一个设备只允许在某些用户输入允许配对后的某个时间窗口内允许配对时。

未知 LMP PDU(0x19)

未知 LMP PDU 错误代码表示控制器接收到未知的 LMP 操作码。

不支持的远程功能(0x1A)

不支持的远程功能错误代码表明远程设备不支持与发出的命令、LMP PDU 或链接层控制 PDU 关联的功能。

SCO 偏移被拒绝(0x1B)

SCO 偏移被拒绝的错误代码表示在 LMP_SCO_LINK_REQ PDU 中请求的偏移已被拒绝。

SCO 时间间隔被拒绝(0x1C)

SCO 时间间隔被拒绝的错误代码表示在 LMP_SCO_LINK_REQ PDU 中请求的时间间隔已被拒绝。

SCO 空气模式被拒绝(0x1D)

SCO 空气模式被拒绝的错误代码表示在 LMP_SCO_LINK_REQ PDU 中请求的空气模式已被拒绝。

无效的 LMP 参数/无效的 LL 参数(0x1E)

无效的 LMP 参数/无效的 LL 参数错误代码表明某些 LMP PDU/LL 控制 PDU 参数无效。

当 PDU 长度无效时，应使用。

•当一个参数需要为偶数时，它是奇数的。

•中的一个参数超出了指定的范围。

•的两个或多个参数的值不一致。

未指定的错误(0x1F)

未指定的错误错误代码表示没有指定的其他错误代码适合使用。

不受支持的 LMP 参数值/不受支持的 LL 参数值(0x20)

AltoBeam Inc. Confidential Document

不受支持的 LMP 参数值/不受支持的 LL 参数值错误代码表示 LMP PDU 或 LL 控制 PDU 包含至少一个此时控制器不支持的参数值。这通常是在长时间的协商过程之后使用的，例如在 LMP_HOLD_REQ、LMP_SNIFF_REQ 和 LMP_ENCRYPTION_KEY_SIZE_-REQPDU 交换期间。这可以被链路层使用，例如在连接参数请求链路层控制过程中。

不允许角色更改(0x21)

不允许角色更改的错误代码表示控制器此时不允许角色更改。

LMP 响应超时/LL 响应超时(0x22)

LMP 响应超时/LL 响应超时错误代码表示 LMP 事务在 LMP 响应超时内未响应，或 LL 事务在 LL 响应超时内未响应。

LMP PDU 不允许(0x24)

LMP PDU 不允许的错误代码表示控制器发送了一个具有不允许的操作码的 LMP PDU。

加密模式不可接受（0x25）

加密模式不可接受的错误代码表示此时所请求的加密模式不可接受。

无法更改链接键(0x26)

无法更改链接密钥错误代码表示无法更改链接键，因为使用了固定的单位密钥。

请求的 QoS 不支持(0x27)

请求的 QoS 不支持的错误代码表明不支持请求的服务质量。

即时传递(0x28)

即时传递的错误代码表示无法执行包含瞬间的 LMP PDU 或 LL PDU，因为这将发生的瞬间已经传递。

不支持单元密钥配对(0x29)

不支持单元密钥配对错误代码表示无法请求单元密钥配对，不支持。

不同的事务冲突(0x2A)

AltoBeam Inc. Confidential Document

不同的事务冲突错误代码表示已启动了与正在进行的事务发生冲突的 LMP 事务或 LL 的过程。

QoS 不可接受参数(0x2C)

QoS 不可接受参数错误码表示此时不能接受指定的服务质量参数，但其他参数也可以接受。

QoS 拒绝(0x2D)

QoS 拒绝错误代码表示不能接受指定的服务质量参数，应终止 QoS 协商。

不支持通道评估（0x2E）

不支持通道评估的错误代码表明控制器无法执行通道评估，因为不支持它。

安全不足(0x2F)

安全不足错误代码表示所发送的 HCI 命令或 LMP PDU 仅可在加密的链路上执行。

参数超出强制范围（0x30）

参数超出强制范围错误代码表示请求的参数值超出了给定 HCI 命令或 LMP PDU 的强制参数范围，并且收件人不接受该值。

ROLE 切换挂起（0x32）

角色切换挂起错误代码表明角色切换正在挂起。当 HCI 命令或 LMP PDU 因角色切换而不能被接受时，可以使用此命令。这也可用于通知对等设备关于挂起的角色切换。

保留 SLOT 冲突(0x34)

保留插槽冲突错误代码表示当前同步协商已终止，协商状态设置为保留插槽冲突。

ROLE 开关失败(0x35)

角色开关失败的错误代码表示已经尝试了一个角色开关，但它失败了，并恢复了原始的图片网结构。开关可能已经故障，因为 TDD 开关或图片开关故障。

扩展查询响应太大(0x36)

AltoBeam Inc. Confidential Document

扩展查询响应太大的错误代码表明，扩展查询响应的要求太大，无法适应控制器支持的任何数据包类型。

主机不支持的安全简单配对(0x37)

主机不支持的安全简单配对错误代码表明 IO 功能请求或响应被拒绝，因为发送主机不支持安全简单配对，即使接收链接管理器不支持。

主机 Busy-配对(0x38)

主机忙配对错误代码表示主机忙于另一个配对操作，无法支持所请求的配对。接收设备稍后应再次重试配对。

连接拒绝到未找到合适的通道(0x39)

由于未找到合适的通道而拒绝的连接错误代码表明控制器无法计算通道选择操作的适当值。

控制器 BUSY(0x3A)

控制器繁忙的错误代码表示操作被拒绝，因为控制器繁忙，无法处理请求。

不可接受的连接参数(0x3B)

不可接受的连接参数错误代码表示远程设备由于一个或多个不可接受的连接参数而终止了连接或拒绝了一个请求。

广播超时(0x3C)

广播超时错误代码表示固定时间的广告完成，或者对于定向广播，广播在没有创建连接的情况下完成。

由于 MIC 故障而终止的连接终止(0x3D)

由于 MIC 故障而终止的连接错误代码表明，由于接收到的数据包上的消息完整性检查(MIC)失败，连接或同步已终止。

连接未能被建立起/同步超时(0x3E)

连接未能建立/同步超时错误代码表明，LL 启动了连接或启动了对定期广播的同步，但连接未能建立，或链接层未能与第一次尝试的 6 个周期广播事件内的定期广播同步。

AltoBeam Inc. Confidential Document

粗时钟调整被拒绝，但将尝试使用时钟拖动(0x40)

粗时钟调整被拒绝，但将尝试使用时钟拖动错误代码表明，此时无法使用提供的参数对微型时钟进行粗调整。相反，中央系统将尝试使用时钟拖动来移动时钟。

未定义类型 0 SUBMAP(0x41)

类型 0 子映射未定义的错误代码表示 LMP PDU 被拒绝，因为类型 0 子映射当前未定义。

未知广播标识符(0x42)

未知广播标识符错误代码表示从主机发送了一个应该标识广播或同步句柄的命令，但广播或同步句柄不存在。

达到限制（0x43）

达到限制的错误代码表示已达到请求的操作数量，并表示活动已完成（例如，广播或扫描）。

被主机取消的操作(0x44)

被主机取消的操作错误代码表示主机向控制器发出的且仍未完成的请求已成功取消。

包太长(0x45)

包太长的错误代码表示试图发送或接收超过允许的最大包长度的包。

9 单独 BLE 广播与扫描（无法连接）

9.1 启动广播共存宏

在 ble_host\os\linux_app\include\syscfg\syscfg.h 文件中把该宏

MYNEWT_VAL_USE_ADV_SCAN_ONLY 置 1

AltoBeam Inc. Confidential Document

syscfg.h

Symbol Name (Alt+L)

- ifndef INDIRECT_ROM_PAT
  - INDIRECT_ROM_PATCH
- endif
- ifndef NIMBLE_CFG_CONTR
  - NIMBLE_CFG_CONTROLI
- endif
- ifndef BT_BLE_ALL_SUPPOR
  - BT_BLE_ALL_SUPPORT
- endif
- ifndef SRAM_CODE
  - SRAM_CODE
- endif
- ifdef CONFIG_LINUX_BLE_S
  - MYNEWT_VAL_SHELL_CM
- endif
- MYNEWT_VAL_USE_ADV_S
- if MYNEWT_VAL_USE_ADV_
  - MYNEWT_VAL_SHELL_CM
  - MYNEWT_VAL_BLE_ROLE
  - MYNEWT_VAL_BLE_ROLE
  - MYNEWT_VAL_BLE_SM_L
  - MYNEWT_VAL_BLE_SM_S
  - MYNEWT_VAL_BLE_ATT_
  - MYNEWT_VAL_BLE_ATT_
  - MYNEWT_VAL_BLE_ATT_
  - MYNEWT_VAL_BLE_ATT_
  - MYNEWT_VAL_BLE_ATT_
  - MYNEWT_VAL_BLE_ATT_

```
55:
56: #ifdef CONFIG_LINUX_BLE_STACK_LIB
57: #define MYNEWT_VAL_SHELL_CMD                (0)
58: #endif
59:
60:
61: #define MYNEWT_VAL_USE_ADV_SCAN_ONLY        (1)
62:
63: #if MYNEWT_VAL_USE_ADV_SCAN_ONLY
64: #define MYNEWT_VAL_SHELL_CMD                (0)
65: #define MYNEWT_VAL_BLE_ROLE_CENTRAL         (0)
66: #define MYNEWT_VAL_BLE_ROLE_PERIPHERAL      (0)
67: #define MYNEWT_VAL_BLE_SM_LEGACY            (0)
68: #define MYNEWT_VAL_BLE_SM_SC                (0)
69: #define MYNEWT_VAL_BLE_ATT_SVR_FIND_INFO    (0)
70: #define MYNEWT_VAL_BLE_ATT_SVR_FIND_TYPE    (0)
71: #define MYNEWT_VAL_BLE_ATT_SVR_INDICATE     (0)
72: #define MYNEWT_VAL_BLE_ATT_SVR_NOTIFY       (0)
73: #define MYNEWT_VAL_BLE_ATT_SVR_QUEUED_WRITE (0)
74: #define MYNEWT_VAL_BLE_ATT_SVR_READ         (0)
75: #define MYNEWT_VAL_BLE_ATT_SVR_READ_BLOB    (0)
76: #define MYNEWT_VAL_BLE_ATT_SVR_READ_GROUP_TYPE (0)
77: #define MYNEWT_VAL_BLE_ATT_SVR_READ_MULT    (0)
78: #define MYNEWT_VAL_BLE_ATT_SVR_WRITE        (0)
79: #define MYNEWT_VAL_BLE_ATT_SVR_WRITE_NO_RSP (0)
80: #define MYNEWT_VAL_BLE_GATT_INDICATE        (0)
81: #define MYNEWT_VAL_BLE_GATT_NOTIFY          (0)
82: #define MYNEWT_VAL_BLE_L2CAP_JOIN_RX_FRAGS  (0)
83: #define MYNEWT_VAL_BLE_GATT_MAX_PROCS       (1)
84: #define MYNEWT_VAL_BLE_L2CAP_COC_MAX_NUM    (0)
85: #define MYNEWT_VAL_BLE_HCI_ACL_OUT_COUNT    (2)
86: #define MYNEWT_VAL_BLE_ACL_BUF_COUNT        (2)
87: #define MYNEWT_VAL_MSYS_1_BLOCK_COUNT       (4)
88: #endif
89:
90: /**
91:  * This macro exists to ensure code includes this header when needed.  If code
92:  * checks the existence of a setting directly via ifdef without including this
93:  * header, the setting macro will silently evaluate to 0.  In contrast, an
```

置1

Y:\6062\AtbmWifi6_ble_Driver_SVN3454_LMAC20712\ble_host\Makefile - Notepad++

文件(F) 编辑(E) 搜索(S) 视图(V) 编码(N) 语言(L) 设置(T) 工具(O) 宏(M) 运行(R) 插件(P) 窗口(W) ?

Serial-COM3_20240927_095101.log | 设备01压测_0925_145902.log | Makefile | Makefile_adv.include | main.c

```
24 AR       := $(CROSS_COMPILE)ar
25 AS       := $(CROSS_COMPILE)as
26 NM       := $(CROSS_COMPILE)nm
27 OBJDUMP  := $(CROSS_COMPILE)objdump
28 OBJCOPY  := $(CROSS_COMPILE)objcopy
29 SIZE     := $(CROSS_COMPILE)size
30 CONFIG_LINUX_BLE_STACK_APP = y
31 CONFIG_LINUX_BLE_STACK_LIB = n
32 CONFIG_BLE_ADV_CFG          = y
33 # Configure NimBLE variables
34 #NIMBLE_ROOT := nimble_v42
35 BLEHOST = $(shell pwd)/ble_host/
36 NIMBLE_ROOT = $(BLEHOST)/nimble_v42
```

9.2 启动 ble 广播共存

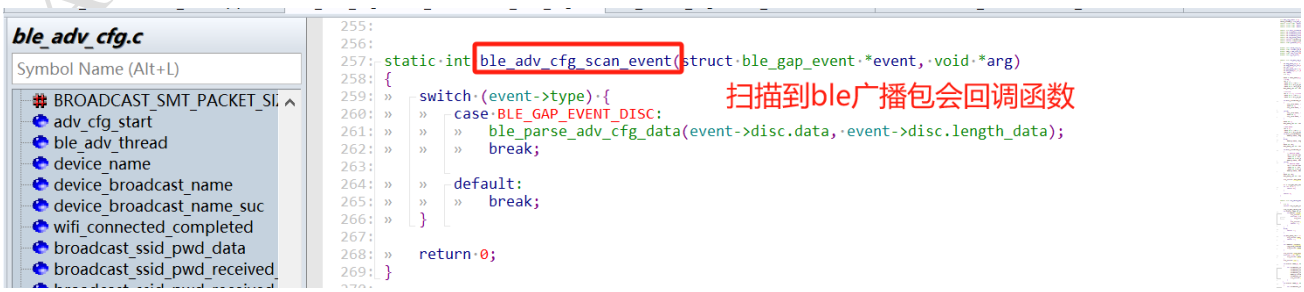
在 ble_host\nimble_v42\apps\ble.c 文件中，屏蔽连接共存 demo，启动广播共存 demo。该方式没办法连接 ble！



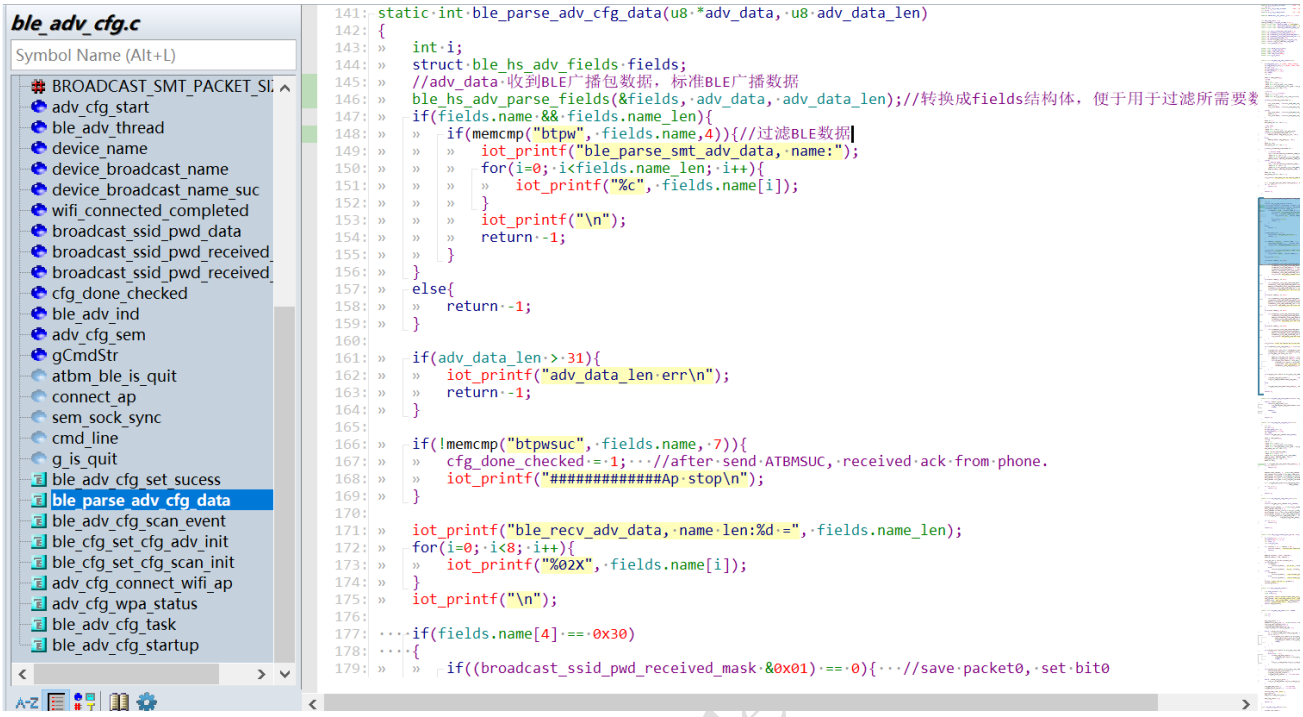
9.3 设置 ble 广播数据

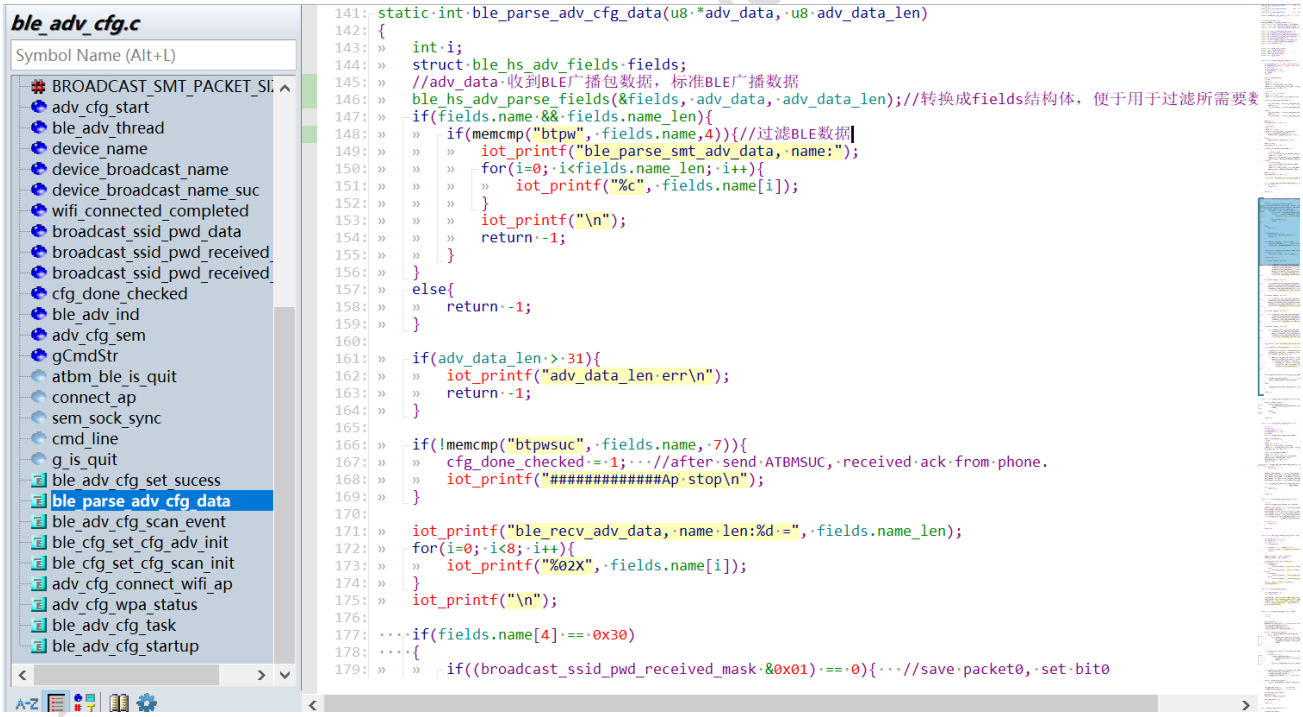


9.4 ble 扫描到数据进入回调函数



9.5 过滤 BLE 接收到广播数据





10 库方式编译 demo (6461 不需要看)

10.1 启动编译成静态库宏

在 ble_host 目录下打开 Makefile，把编译库方式打开

```
#  software distributed under the License is distribut
# "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF A
# KIND, either express or implied.  See the License fo
# specific language governing permissions and limitati
# under the License.
#

# Toolchain commands
CROSS_COMPILE ?= /wifi_prj/staff/zhouzhanchao/android4
CC       := $(CROSS_COMPILE)gcc
CXX      := $(CROSS_COMPILE)g++
LD       := $(CROSS_COMPILE)gcc
AR       := $(CROSS_COMPILE)ar
AS       := $(CROSS_COMPILE)as
NM       := $(CROSS_COMPILE)nm
OBJDUMP  := $(CROSS_COMPILE)objdump
OBJCOPY  := $(CROSS_COMPILE)objcopy
SIZE     := $(CROSS_COMPILE)size
CONFIG_LINUX_BLE_STACK_APP = y
CONFIG_LINUX_BLE_STACK_LIB = y
CONFIG_BLE_ADV_CFG          = n
# Configure NimBLE variables
#NIMBLE_ROOT := nimble_v42
```

10.2 动态库编译

```
nimble_linux_lib: $(OBJ) $(TINYCRYPT_OBJ)
        $(AR) rc libnimble_linux.a $(OBJ)
        cp -rf libnimble_linux.a  $(BLEHOST)/../driver_install
        #$(CROSS_COMPILE)gcc -shared -o libnimble_linux.so $(OBJ)
        #cp -rf libnimble_linux.so $(BLEHOST)/../driver_install
```

静态库

动态库

10.3 调用库编译应用

编译好库文件放在 driver_install 目录下，复制 ble_host\ble_smart_cfg_lib\



Confidential Document

在 ble_host\ble_smart_cfg_lib\目录下，执行 make 编译出 app_demo 可执行文件。

名称	修改日期	类型	大小
aes.c	6/11/2023 下午7:08	sourceinsight.c_file	20 KB
aes.h	6/11/2023 下午7:08	H 文件	1 KB
ble_smart_cfg.c	7/11/2023 下午6:36	sourceinsight.c_file	36 KB
ble_smart_cfg.h	6/11/2023 下午7:08	H 文件	2 KB
ble_smart_cfg.h.gch	6/11/2023 下午7:08	GCH 文件	3,521 KB
ble_smart_demo.c	6/11/2023 下午7:08	sourceinsight.c_file	10 KB
libnimble_linux.a	7/11/2023 下午6:50	A 文件	1,764 KB
Makefile	7/11/2023 下午6:02	文件	4 KB

10.4 协议栈初始化函数

调用 lib_ble_main_init 初始化整个协议栈

```
int main()
{
    printf("[%d] 11.14 aaa ble_smart_cfg_lib.c main fun enter!\n", __LINE__);
    iot_printf("======>>atbm_ble_start>>>lib_ble_reduce_mem\n");
    lib_ble_main_init();
    while(1)
        sleep(1);

}
```

初始化协议成功后，会自动执行到下面。需要主程序while1住。不然会退出进程

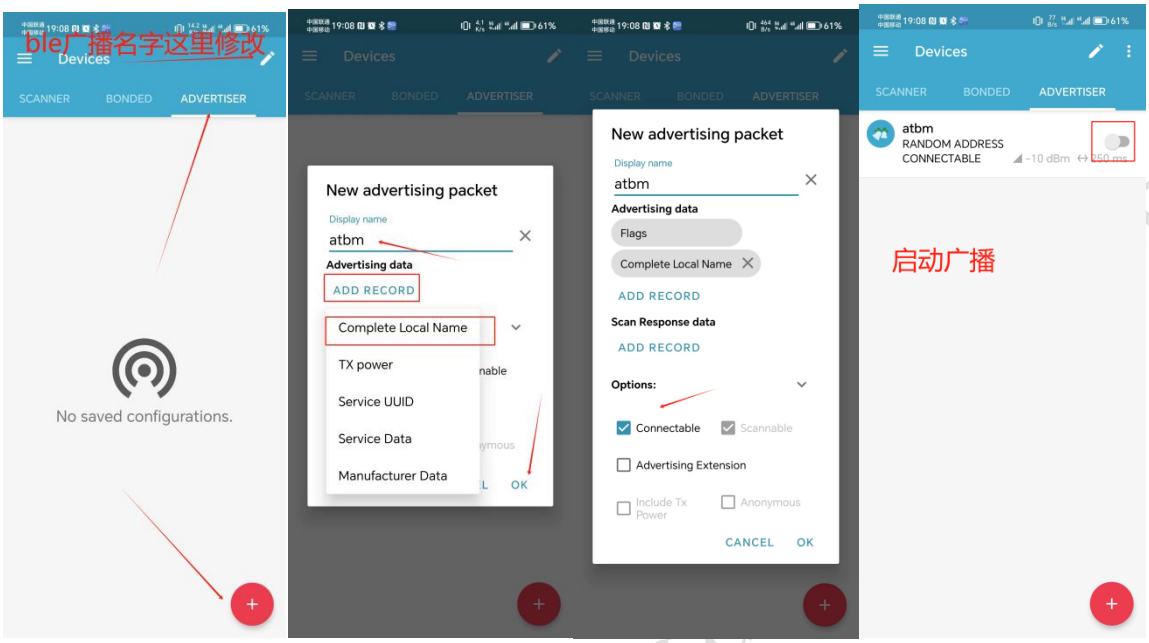
10.5 切换省内存版本 BLE

省内存版本请调用 lib_ble_main_init 初始化 ble 前把 lib_ble_reduce_mem 变量置 1，。切换正常版本就需要调用 lib_ble_thread_exit()退出 ble 线程再把 lib_ble_reduce_mem 清零后调用 lib_ble_main_init 函数。

AltoBeam Inc. Confi Docum

11 BLE master 指令使用介绍

11.1 Nrf connect 软件启动 ble slave 广播



11.2 扫描附近所有 BLE 设备

(6461 指令输入不需要 atbm_cli，请直接输入 btshell advertise stop，下面所有命令类似)

标准代码默认是启动 BLE 广播，需要用不到 slave 模式或者广播，请停止广播。

输入指令：./atbm_cli btshell advertise stop

输入指令：./atbm_cli btshell scan

BLE 地址类型：

/* Own Bluetooth Device address type */

#define BLE_OWN_ADDR_PUBLIC (0x00)

#define BLE_OWN_ADDR_RANDOM (0x01)

#define BLE_OWN_ADDR_RPA_PUBLIC_DEFAULT (0x02)

#define BLE_OWN_ADDR_RPA_RANDOM_DEFAULT (0x03)

AltoBeam Inc. Confidential Document

```
./atbm_cli btshell scan
cmd_line: btshell scan
GAP procedure initiated: discovery; own_addr_type=0 filter_policy=0 passive=0 limited=0 filter_duplica
tes=0 duration=forever
Anjoy# ===> fields:
    name(complete)=WBP1-69B
    mfg_data=0x18:0x2b:0x00:0xb8:0xb7:0x7d:0x0e:0xa6:0x9b:0x49:0x42:0x41:0x36:0x32:0x35:0x30:0x38:0x31
:0x39

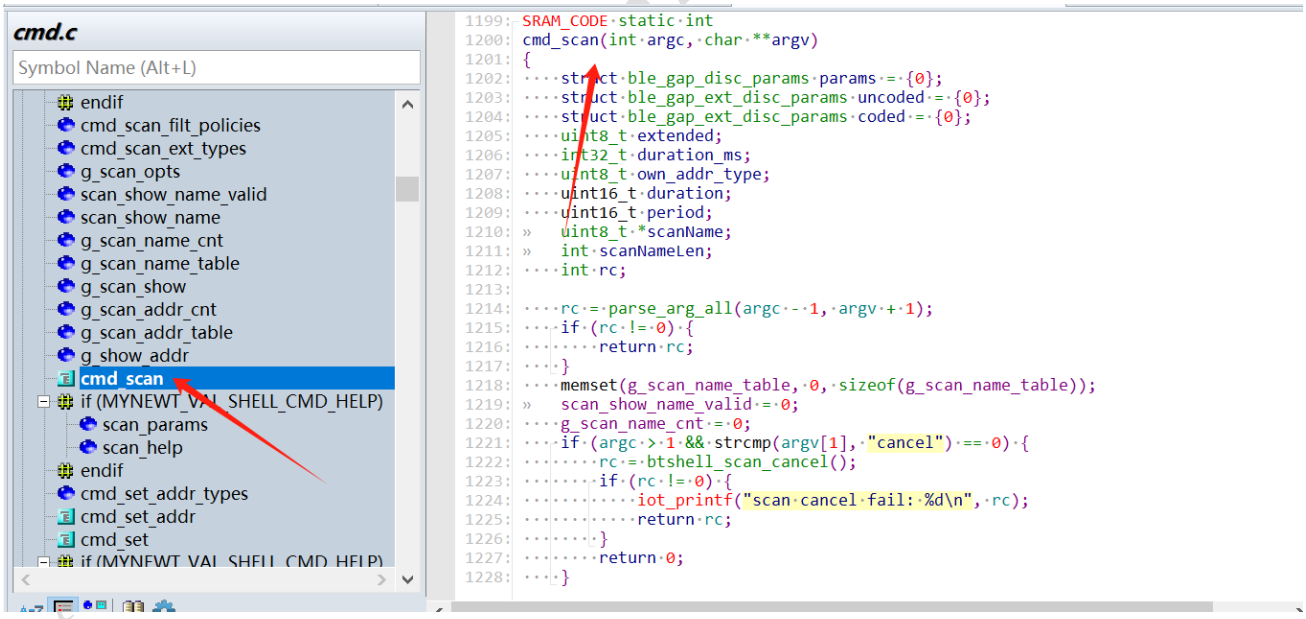
===> fields:
    flags=0x06:
        General discoverable mode
        BR/EDR not supported
    uuids16(complete)=0xBBB0 0xBBB1
    name(complete)=AltoBeam
    mfg_data=0xab:0xab:0xa0

===> fields:
    name(complete)=BB0759026
    mfg_data=0x18:0x2b:0x08:0x8a:0xc0:0x87:0x0c:0xff:0xa1:0x2b:0x65:0x7a:0x76:0x69:0x7a

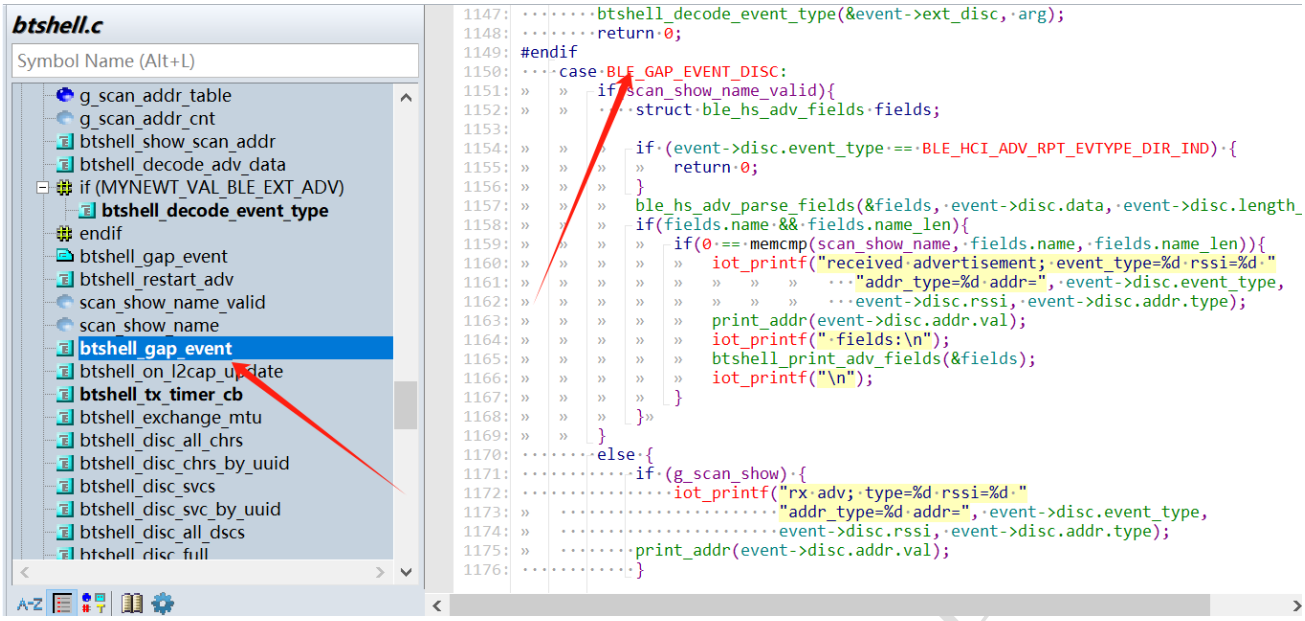
===> fields:
    name(complete)=BA9076037
    mfg_data=0x18:0x2b:0x08:0x4e:0xbf:0x87:0x0c:0xff:0xa1:0x04:0x65:0x7a:0x76:0x69:0x7a

===> fields:
    flags=0x06:
        General discoverable mode
```

命令函数：

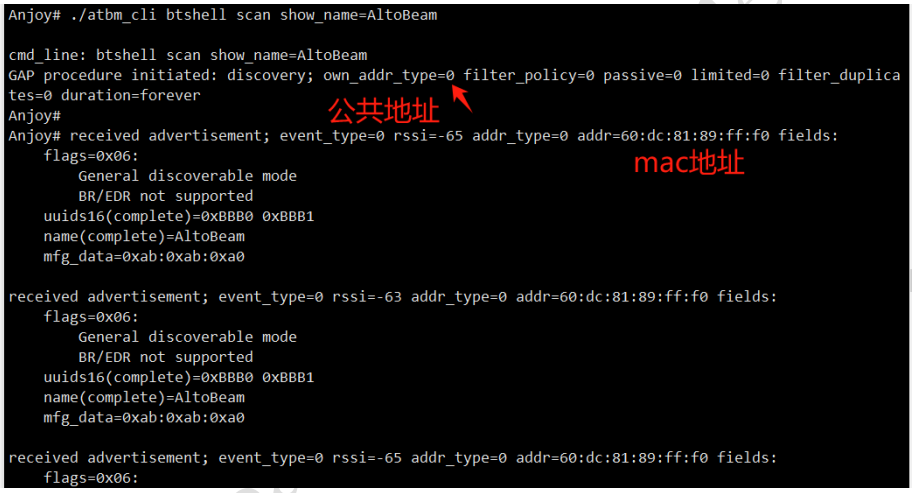


事件回调函数：



11.3 扫描指定 BLE 名字设备

输入指令：./atbm_cli btshell scan show_name=AltoBeam



11.4 停止扫描

指令输入：./atbm_cli btshell scan cancel

11.5 连接 slave 设备

连接公共地址：./atbm_cli btshell connect peer_addr=60:dc:81:89:ff:f0

连接随机地址：./atbm_cli btshell connect own_addr_type=public peer_addr_type=random peer_addr=60:dc:81:89:ff:f0

Mac 地址类型参考如下：

{"peer_addr_type", "usage: =[public|random|public_id|random_id], default: public"},

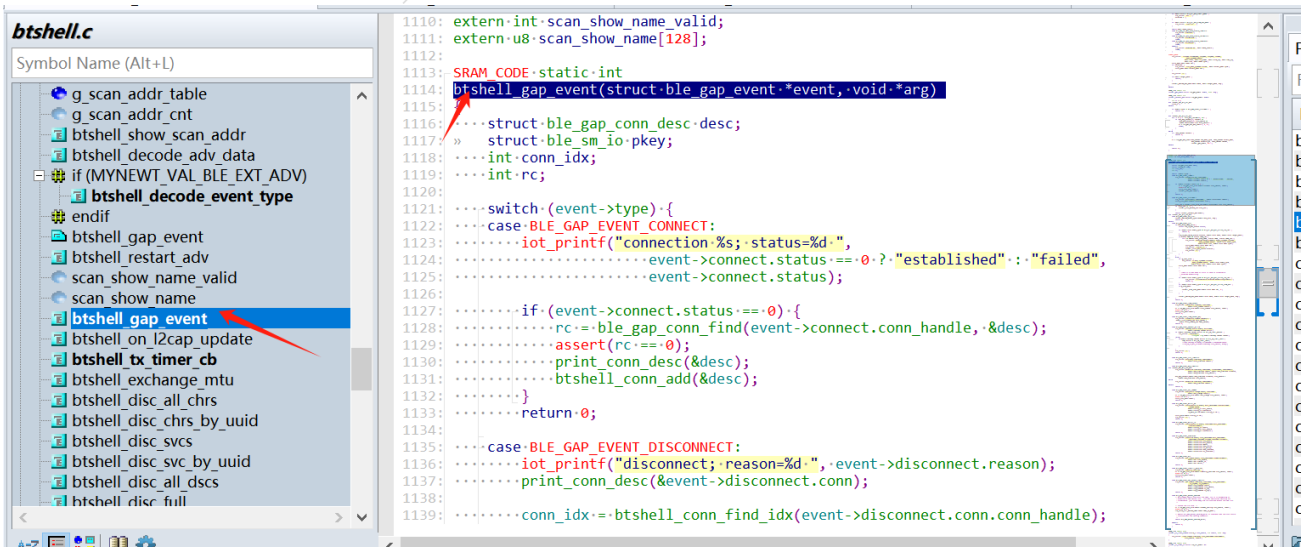
{"own_addr_type", "usage: =[public|random|rpa_pub|rpa_rnd], default: public"},

```
./atbm_cli btshell connect  peer_addr=60:dc:81:89:ff:f0
cmd_line: btshell connect peer_addr=60:dc:81:89:ff:f0
GAP procedure initiated: connect; peer_addr_type=0 peer_addr=60:dc:81:89:ff:f0 sc
an_itvl=16 scan_window=16 itvl_min=24 itvl_max=40 latency=0 super_tmo=256 min_ce_
len=16 max_ce_len=768 own_addr_type=0
error connecting; rc=0
Anjoy# connection established; status=0 handle=1 our_ota_addr_type=0 our_ota_addr
=60:95:ce:69:3d:cb our_id_addr_type=0 our_id_addr=60:95:ce:69:3d:cb peer_ota_addr
_type=0 peer_ota_addr=60:dc:81:89:ff:f0 peer_id_addr_type=0 peer_id_addr=60:dc:81
:89:ff:f0 conn_itvl=40 conn_latency=0 supervision_timeout=256 encrypted=0 authent
icated=0 bonded=0
mtu update event; conn_handle=1 cid=4 mtu=255
```

连接 api



回调函数



11.6 显示连接设备的 GAP 与 GATT

./atbm_cli btshell gatt-discover-full conn=1 //获取手机支持的 GATT 服务 conn=1 是第一个从机设备

./atbm_cli btshell gatt-show conn=1 //显示手机支持的 GATT 服务

代码如下：





Properties	值	Properties	值	Properties	值
BROADCAST	0x0001	AUTH_SIGN_WRITE	0x0040	WRITE_ENC	0x1000
READ	0x0002	RELIABLE_WRITE	0x0080	WRITE_AUTHEN	0x2000

WRITE_NO_RSP	0x0004	AUX_WRITE	0x0100	WRITE_AUTHOR	0x4000
WRITE	0x0008	READ_ENC	0x0200		
NOTIFY	0x0010	READ_AUTHEN	0x0400		
INDICATE	0x0020	READ_AUTHOR	0x0800		

11.7 BLE master 通过 WRITE 属性发数据

通过 WRITE 属性发数据给 UUID：0x2A8A：

./atbm_cli btshell gatt-write conn=1 attr=21 value=ff:ff:89:99:00:77:ff:89:99:00 //attr 上面图片 val_handle 值。

11.8 断开连接 slave 设备

./atbm_cli btshell disconnect conn=1 //conn 值为第几台 slave 设备

12 BLE MASTER DEMO 使用介绍

12.1 根据名字连接 slave 设备

根据名字连接 slave 设备，启动 ble 扫描。然后过滤名字后再去连接。

#define BLE_CONN_NAME "altobeam"

ble_master_connect(BLE_CONN_NAME);

12.2 根据 BLE 设备地址连接 slave 设备

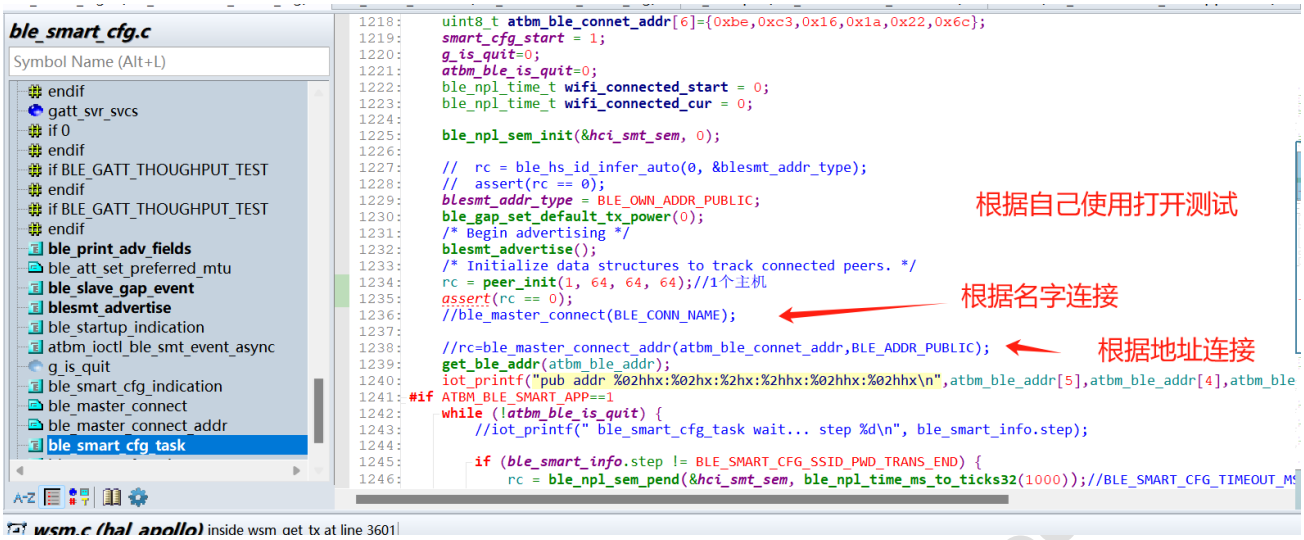
根据 ble mac 地址去连接，需要知道 slave 设备是公共地址还是随机地址。

Ble mac 地址是小端格式。

uint8_t atbm_ble_connet_addr[6]={0xbe,0xc3,0x16,0x1a,0x22,0x6c};

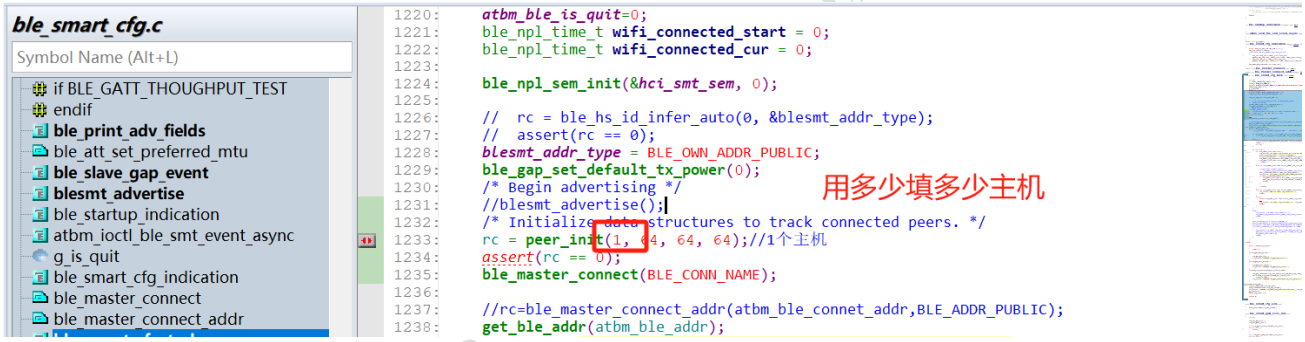
ble_master_connect_addr(atbm_ble_connet_addr,BLE_ADDR_PUBLIC);

AltoBeam Inc. Confidential Document



12.3 初始化 peer 属性队列

需要连接前初始化 peer 队列，按照需求申请 buff



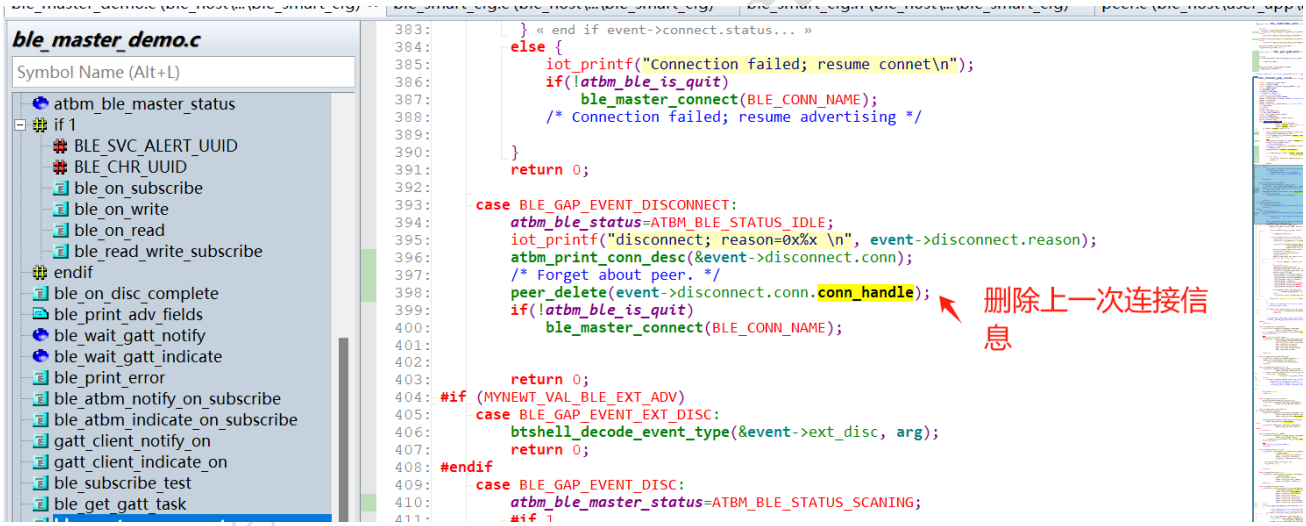
12.4 连接成功后获取 slave 端 Service 和 Characteristic、Descriptors

在连接成功后回调函数 BLE_GAP_EVENT_CONNECT 中进行获取属性设置 MTU

AltoBeam Inc. Confidential Document

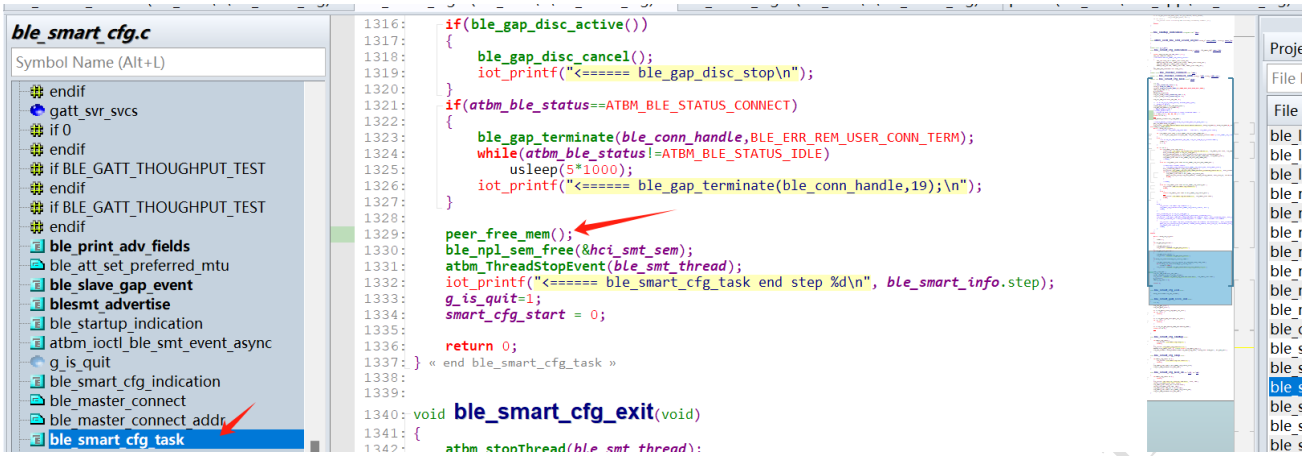


12.5 断开连接释放该连接内存占用



12.6 释放 peer 属性队列

退出协议栈，释放内存

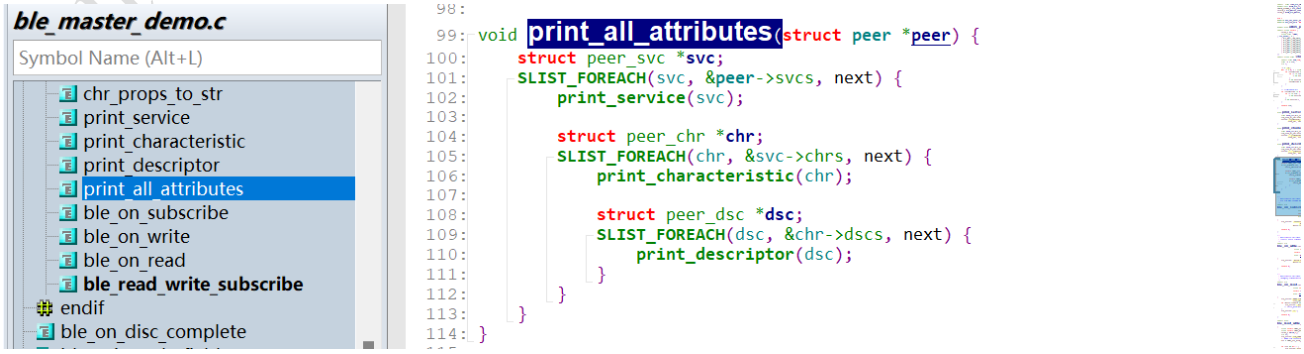


12.7 获取完所有属性

读完所有属性后，会到回调这个函数



12.8 打印已获取属性列表



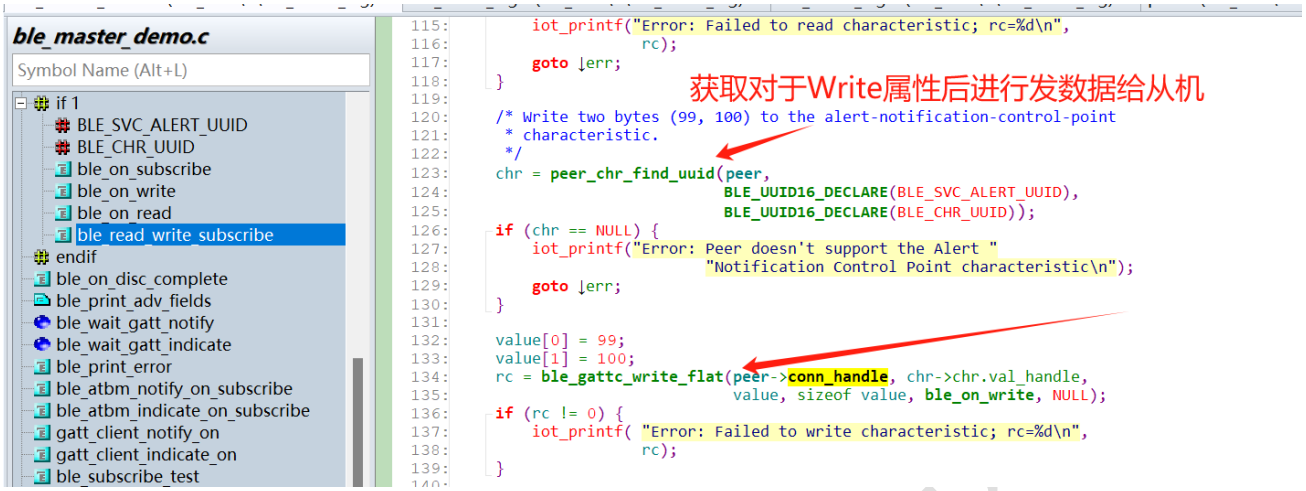



```
[26/3/2025 下午11:21:56.%N] Service discovery complete; status=0 conn_handle=1
[26/3/2025 下午11:21:56.%N]
[26/3/2025 下午11:21:57.%N] [Service] UUID: 0x1801, Handles: 1-9
[26/3/2025 下午11:21:57.%N]   [Characteristic] UUID: 0x2a05, Handle: 2, Properties: INDICATE (Props: 0x20)
[26/3/2025 下午11:21:57.%N]   [Characteristic] UUID: 0x2b3a, Handle: 4, Properties: READ (Props: 0x02)
[26/3/2025 下午11:21:57.%N]   [Characteristic] UUID: 0x2b29, Handle: 6, Properties: READ|WRITE (Props: 0x0a)
[26/3/2025 下午11:21:57.%N]   [Characteristic] UUID: 0x2b2a, Handle: 8, Properties: READ (Props: 0x02)
[26/3/2025 下午11:21:57.%N]
[26/3/2025 下午11:21:57.%N] [Service] UUID: 0x1800, Handles: 20-26
[26/3/2025 下午11:21:57.%N]   [Characteristic] UUID: 0x2a00, Handle: 21, Properties: READ (Props: 0x02)
[26/3/2025 下午11:21:57.%N]   [Characteristic] UUID: 0x2a01, Handle: 23, Properties: READ (Props: 0x02)
[26/3/2025 下午11:21:57.%N]   [Characteristic] UUID: 0x2aa6, Handle: 25, Properties: READ (Props: 0x02)
[26/3/2025 下午11:21:57.%N]
[26/3/2025 下午11:21:57.%N] [Service] UUID: 0xfe35, Handles: 134-144
[26/3/2025 下午11:21:57.%N]   [Characteristic] UUID: 0x2a00, Handle: 135, Properties: READ|WRITE (Props: 0x0a)
[26/3/2025 下午11:21:57.%N]   [Characteristic] UUID: 0x2a01, Handle: 137, Properties: NOTIFY|INDICATE (Props: 0x30)
[26/3/2025 下午11:21:57.%N]     [Descriptor] UUID: 0x2902, Handle: 139
[26/3/2025 下午11:21:57.%N]   [Characteristic] UUID: 0x2a02, Handle: 140, Properties: WRITE (Props: 0x08)
[26/3/2025 下午11:21:57.%N]   [Characteristic] UUID: 0x2a03, Handle: 142, Properties: NOTIFY|INDICATE (Props: 0x30)
[26/3/2025 下午11:21:57.%N]     [Descriptor] UUID: 0x2902, Handle: 144
[26/3/2025 下午11:21:57.%N]
[26/3/2025 下午11:21:57.%N] [Service] UUID: 0x046a, Handles: 145-147
[26/3/2025 下午11:21:57.%N]   [Characteristic] UUID: 0x046c, Handle: 146, Properties: READ|WRITE (Props: 0x0a)
[26/3/2025 下午11:21:57.%N]
[26/3/2025 下午11:21:57.%N] [Service] UUID: 11c8b310-80e4-4276-afc0-f81590b2177f, Handles: 148-154
[26/3/2025 下午11:21:57.%N]   [Characteristic] UUID: 0x2b01, Handle: 149, Properties: READ|WRITE|NOTIFY (Props: 0x1a)
[26/3/2025 下午11:21:57.%N]     [Descriptor] UUID: 0x2902, Handle: 151
```

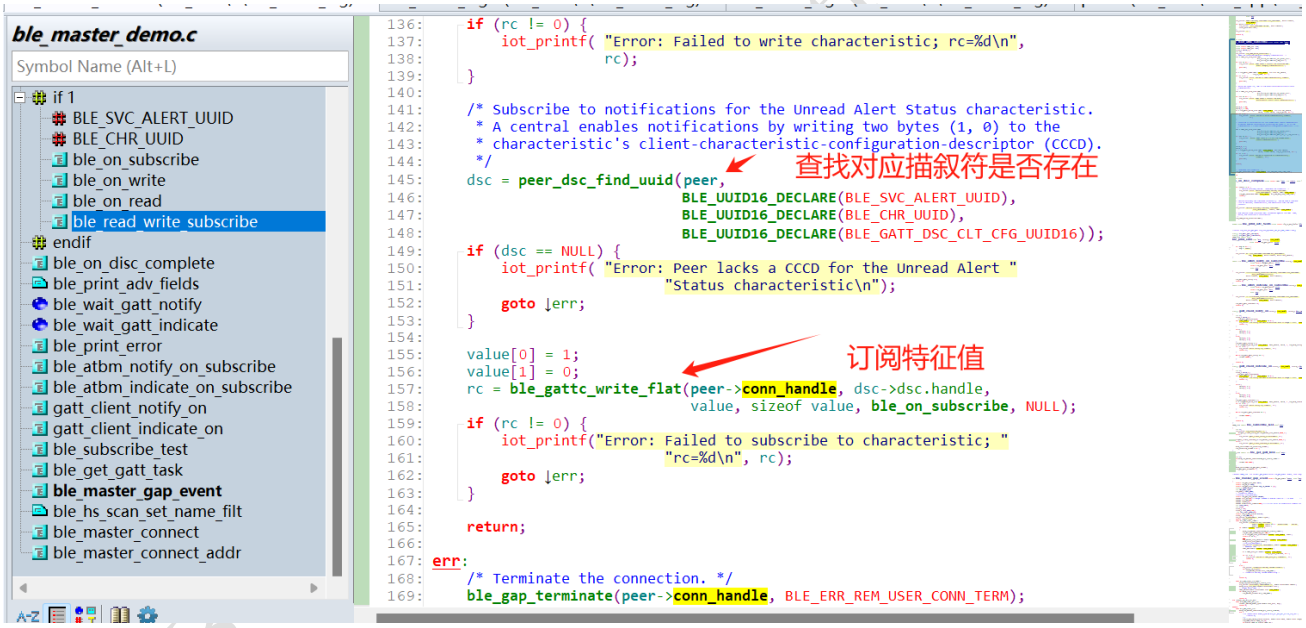
12.9 获取对应服务与特征值 uuid 是否存在，并发起 read 请求获取数据



12.10 获取对应 Write 属性并发数据给从机



12.11 获取对于描叙符特征值后，并订阅



12.12 NOTIFY 订阅 API 使用

int32_t gatt_client_notify_on(uint32_t conn_handle, uint16_t desc_handle, bool on)

参数：

conn_handle=连接 slave handle

desc_handle=打开对于 Characteristic uuid 对于的 Descriptors handle

gatt_client_notify_on(ble_conn_handle,dsc_handle,1);开启订阅

gatt_client_notify_on(ble_conn_handle,dsc_handle,0);关闭订阅

12.13 INDICATE 订阅 API 使用

int32_t gatt_client_indicate_on(uint32_t conn_handle, uint16_t desc_handle, bool on)

参数：

conn_handle=连接 slave handle

desc_handle=打开对于 Characteristic uuid 对于的 Descriptors handle

gatt_client_indicate_on(ble_conn_handle,dsc_handle,1); 开启订阅

gatt_client_indicate_on(ble_conn_handle,dsc_handle,0); 关闭订阅

12.14 断开 slave 设备

Int ble_gap_terminate(uint16_t conn_handle, uint8_t hci_reason)
conn_handle：第几台设备
Reason ：断开原因可以填 0x16

13 BLE 主从机 SMP 加密使用

13.1 BLE Slave SMP 初始化

配对请求的数据格式

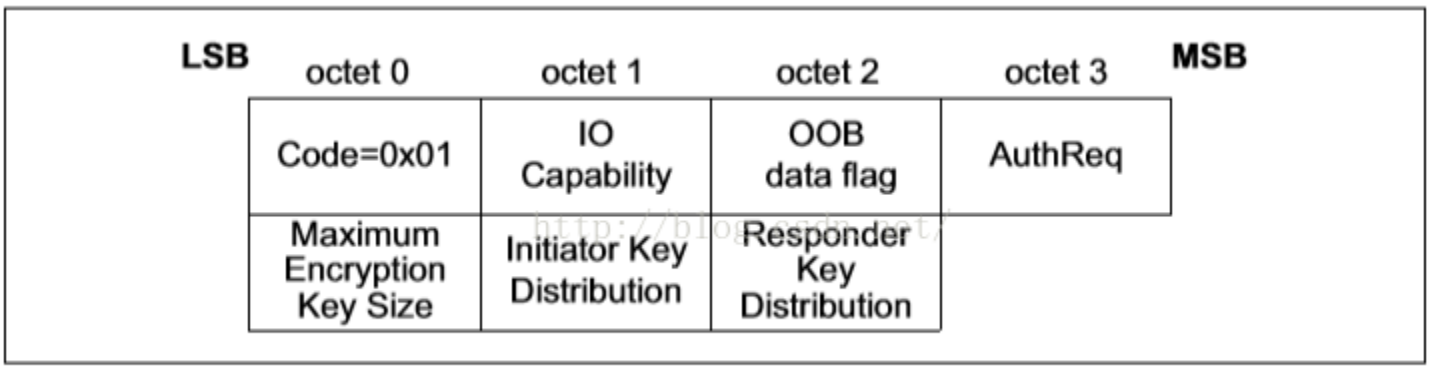


Figure 3.2: Pairing Request Packet

1. IO capabilities 表明输入，输出的能力

输入是按键、键盘，输出是显示数字用的界面。

- 0x00 DisplayOnly 只能是显示 000000～999999 的数字
- 0x01 DisplayYesNo 显示 Yes/No 的按钮
- 0x02 KeyboardOnly 只能是输入 000000～999999 的数字
- 0x03 NoinputNoOutput 没有输入也没有显示，只能用 Just work 工作方式
- 0x04 KeyboardDisplay 能输入 000000～999999 的数字和输出

2. OOB data flag

- 0x00 OOB 数据没有发送
- 0x01 OOB 数据通过远端设备发送（如 IR）
- 0x02-0xFF 保留

3. 身份验证请求

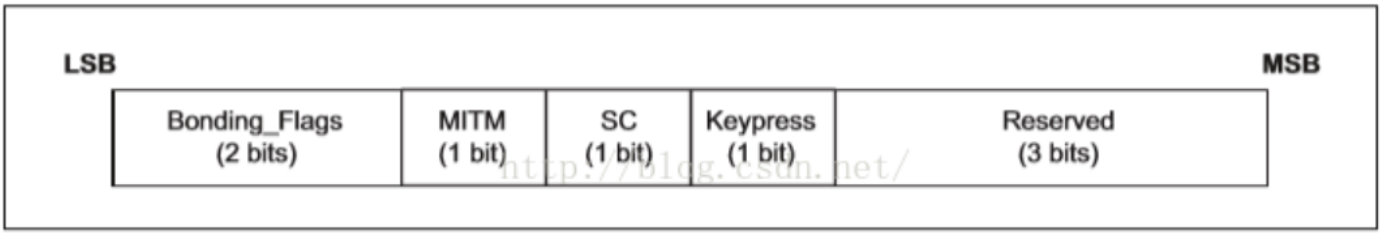


Figure 3.3: Authentication Requirements Flags

Bonding_Flags b1b0 Bonding Type

- 00 No Bonding
- 01 Bonding
- 10 Reserved
- 11 Reserved

MITM

MITM 域设置为 1 为请求 MITM（中间人介入）保护，否则设置为 0. 设备将标志设置为 1 为 STK 请求认证的安全属性。

选择 Key 生成的方法

如果 auth Req 中 MITM 没有，则说明不需要人参与中间，所以 IO capabilities 会被忽略，只用 Just Works 就 OK 了。

如果有 OOB data，auth Req 将可直接忽略，会直接选择 OOB 的方式了。

- SC

SC 字段是一个 1 位标志，设置为 1 以请求 LE 安全连接配对，否则应根据发起方和响应方支持的功能将其设置为 0，可能的结果配对机制为：如果两个设备均支持 LE 安全连接，使用 LE 安全连接； 否则，请使用 LE 旧式配对。

- Keypress

keypress 字段是一个 1 位标志，仅在 Passkey Entry 协议中使用，而在其他协议中将被忽略。 当双方将该字段设置为 1 时，应使用 SMP 配对按键通知 PDU 生成并发送按键通知。



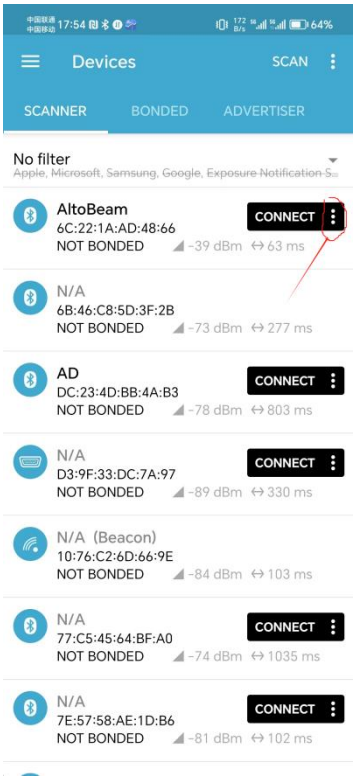
加密主从机初值:

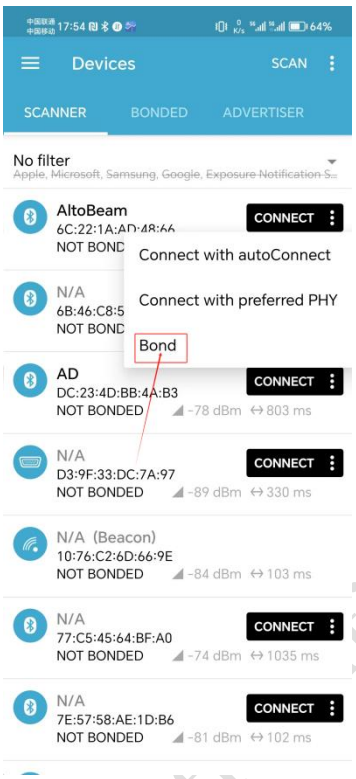
sm_our_key_dist：brief Security Manager Local Key Distribution Mask（可以为 0 或者自己设置）

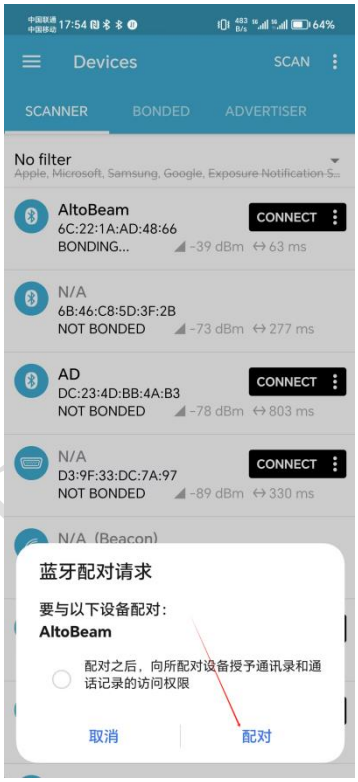
sm_their_key_dist：rief Security Manager Remote Key Distribution Mask（可以为 0 或者自己设置）

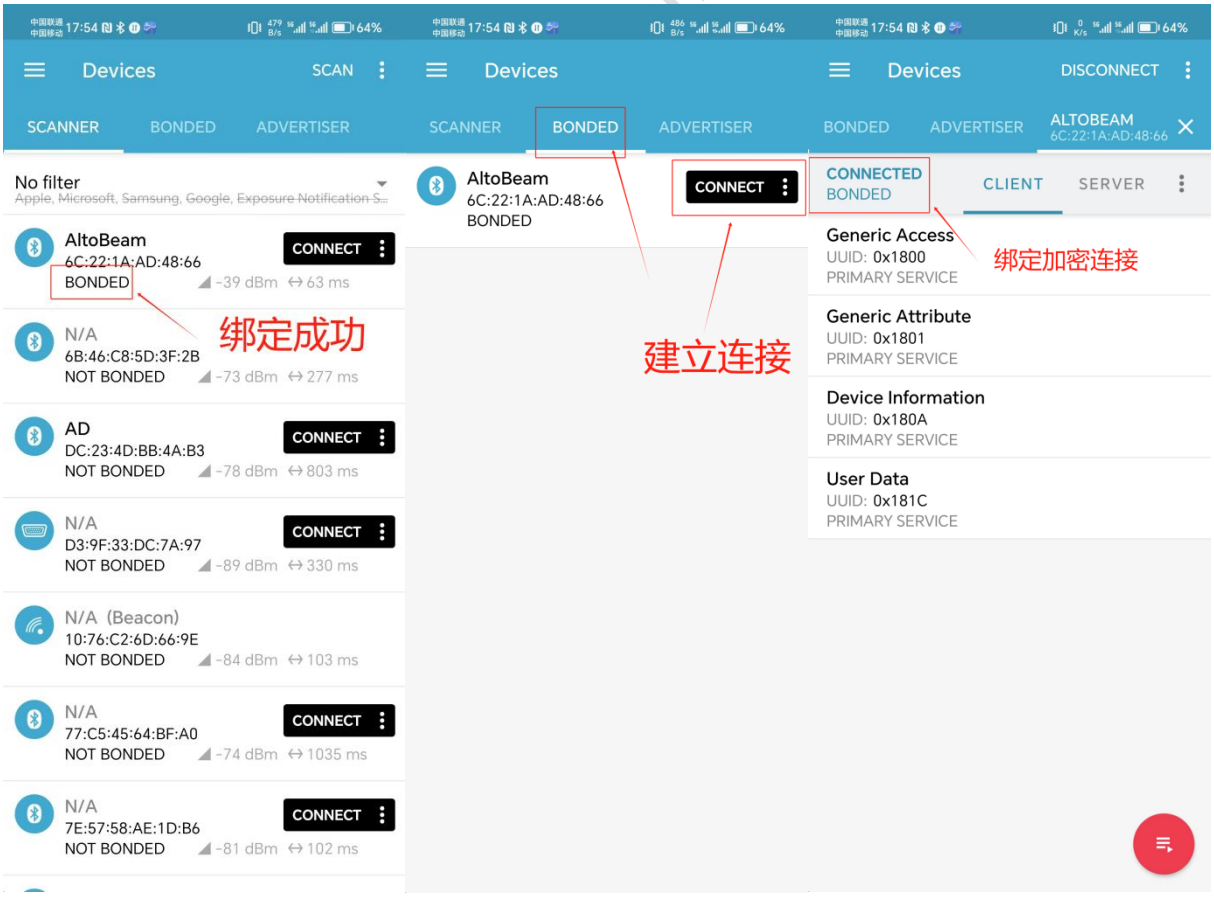
AltoBeam Inc. Confident

13.2 手机 nrf connect 软件建立加密连接：









13.3 BLE Slave 建立连接 log



13.4 BLE MASTER 连接成功进行加密绑定 api

int rc;

rc = ble_gap_pair_initiate(conn_handle);//conn 参数是连接第几台连接设备,需要连接成功后调用

if (rc != 0)

{

iot_printf("error initiating pairing; rc=%d\n", rc);

}

13.5 BLE 主从机指令连接设备并建立加密方式

Slave 端输入指令： 或者参数 12.1 章节启动广播

./atbm_cli btshell security-set-data oob_flag=0 mitm_flag=0 io_capabilities=3 our_key_dist=0 their_key_dist=0 bonding=1 sc=1 //设置配对参数

./atbm_cli btshell set-adv-data name=ble_fpga btshell advertise //启动广播

Master 端输入指令：

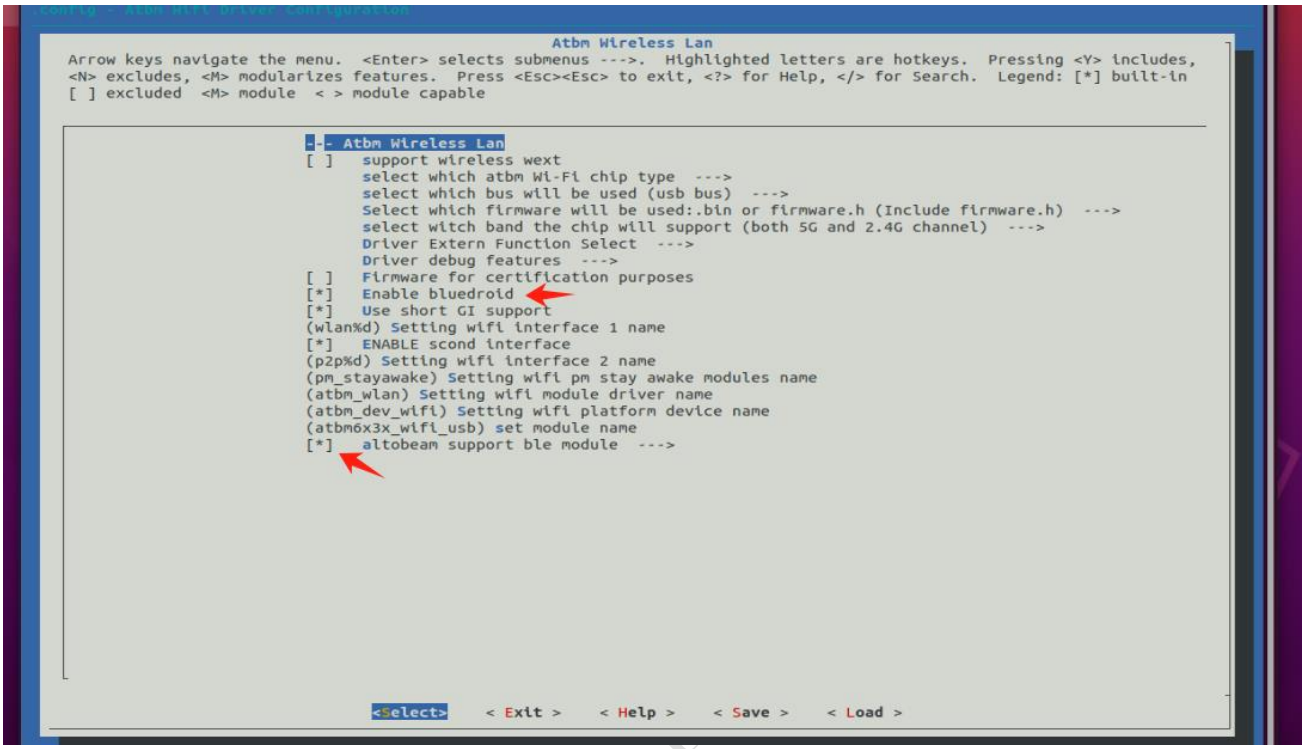
./atbm_cli btshell connect own_addr_type=public peer_addr_type=public peer_addr=08:88:88:00:00:00

//可以参考 10.5 章节

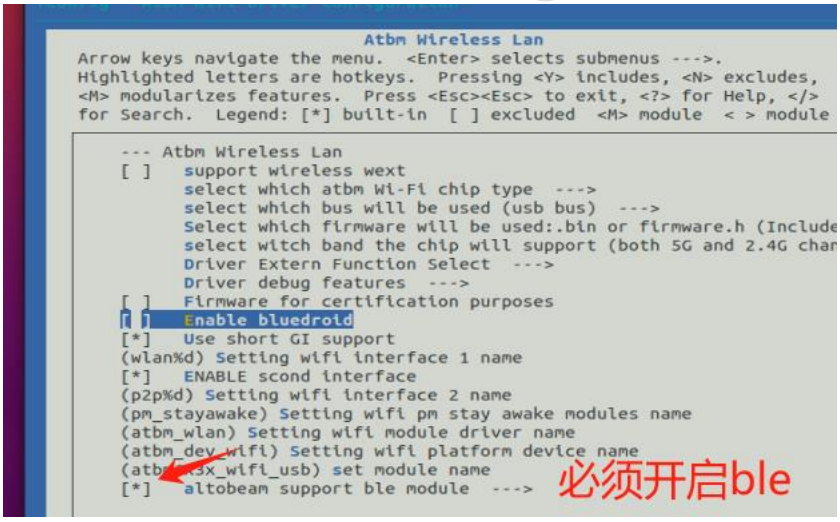
AltoBeam Inc. Confidential Document

./atbm_cli btshell security-pair conn=1 //进行加密绑定，conn 参数是连接第几台连接设备

14 支持安卓 bluedroid(6461 不需要看)



15 支持 linux BLUEZ(6461 不需要看)



增加下图编译项就支持 BLUEZ，在 hal_apollo 目录下 Makefile

ccflags-y += -DCONFIG_ATBM_SUPPORT_BLUEZ

V:\ubuntu\ATBM_WIFI_SVN3407_AsmLite19040_ARESM18826_Meru19963_20240418\hal_apollo\Makefile - Notepad++

文件(F) 编辑(E) 搜索(S) 视图(V) 编码(N) 语言(L) 设置(T) 工具(O) 宏(M) 运行(R) 插件(P) 窗口(W) ?

```
190 # use the deltagain value in file :"$(ATBM_CONFIG_FILE_5G)",when file exist
191 ATBM_RATE_5G_POWER_CONFIG_FILE="/tmp/set_rate_power_5g.txt"
192 ccflags-y += -DCONFIG_RATE_TXPOWER_5G=\"$(ATBM_RATE_5G_POWER_CONFIG_FILE)\"
193
194 #ccflags-y += -DATBM_USE_FASTLINK
195 ccflags-y += -DATBM_SAVE_BSS=\"/data/fastlink.txt\"
196
197 ifneq ($(IF1NAME),)
198 ccflags-y += -DCONFIG_IF1NAME=\"$(IF1NAME)\"
199 endif
200 ifneq ($(IF2NAME),)
201 ccflags-y += -DCONFIG_IF2NAME=\"$(IF2NAME)\"
202 endif
203 ifneq ($(USBVID),)
204 ccflags-y += -DCONFIG_USBVID=$(USBVID)
205 endif
206 ccflags-y += -DCONFIG_ATBM_SUPPORT_BLUEZ
207 ifneq ($(USBPID),)
208 ccflags-y += -DCONFIG_USBPID=$(USBPID)
```

#加载 bluez 依赖库

sudo modprobe bluetooth

加载 cfg80211 模块

insmod /lib/modules/5.15.0-131-generic/kernel/net/wireless/cfg80211.ko

加载 atbm6x3x_wifi_usb 模块，并传递参数

insmod atbm6x3x_wifi_usb.ko wifi_bt_comb=1

hciconfig -a

```
root@lubancat-vm:/mnt/hgfs/shinework/ubuntu/ATBM_WIFI_SVN3407_AsmLite19040_ARESM18826_Meru19963_20240418/driver_install# hciconfig
-a
hci0:   Type: Primary  Bus: USB
        BD Address: 6C:22:1A:B2:FF:22  ACL MTU: 255:4  SCO MTU: 0:0
        UP RUNNING
        RX bytes:0 acl:0 sco:0 events:18 errors:0
        TX bytes:0 acl:0 sco:0 commands:18 errors:0
        Features: 0x00 0x00 0x00 0x00 0x60 0x00 0x00 0x00
        Packet type: DM1 DH1 HV1
        Link policy:
        Link mode: SLAVE ACCEPT
```

bluetoothctl

list

```
        Link mode: SLAVE ACCEPT
Can't read local name on hci0: Input/output error (5)
root@lubancat-vm:/mnt/hgfs/shinework/ubuntu/ATBM_WIFI_SVN3407_AsmLite19040_ARESM18826_Meru19963_20240418/driver_install# bluetoothc
tl
Agent registered
[bluetooth]# list
Controller 6C:22:1A:B2:FF:22 lubancat-vm [default]
[bluetooth]# scan on
Discovery started
[CHG] Controller 6C:22:1A:B2:FF:22 Discovering: yes
[NEW] Device 75:78:C8:14:D6:AD 75-78-C8-14-D6-AD
[NEW] Device 72:9B:6C:4F:46:1F 72-9B-6C-4F-46-1F
[NEW] Device 55:CF:29:5F:B2:7F 55-CF-29-5F-B2-7F
[NEW] Device 6B:5C:6F:96:CE:40 6B-5C-6F-96-CE-40
```

抓 hci 数据：

AltoBeam Confidential D

sudo btmon -w btsnoop.log &

15.1 修改 BLE 公共地址

只支持 bluez 通过 iwpriv 修改！

iwpriv wlan0 common setble_pub_mac,10:02:03:04:05:01

必须在 ble 广播开启前，修改才会生效。Bluez 内部获取不到该公共地址，

只能启动 ble 广播看到地址已经修改。

16 编译 os 层报错修改汇总(6461 不需要看)

16.1 Os 平台适配报错

C:\Users\13420\Desktop\ble_host\os\linux_app\src\os_task.c
2024/8/14 14:14:46 3,973 字节 C,C++,C#,ObjC 源代码 ANSI UNIX

```
106 {
107     return true;
108 }
109
110 void ble_npl_task_yield(void)
111 {
112     sched_yield();//pthread_yield();
113 }
114
115
116
117 pAtbm_thread_t atbm_createThread(atbm_int32(*task)(atbm_void* p_arg), atbm_
118 {
```

Y:\...\AtbmWifi6_ble_Driver_SVN3454_LMAC20712\ble_host\os\linux_app\src\os_task.c
2024/5/8 11:48:13 3,957 字节 C,C++,C#,ObjC 源代码 ANSI UNIX

```
106 {
107     return true;
108 }
109
110 void ble_npl_task_yield(void)
111 {
112     pthread_yield();
113 }
114
115
116
117 pAtbm_thread_t atbm_createThread(atbm_int32(*task)(atbm_void* p_arg), atbm_
118 {
```

C:\Users\13420\Desktop\ble_host\os\linux_app\src\os_mutex.c
2024/8/14 14:09:28 2,290 字节 C,C++,C#,ObjC 源代码 ANSI UNIX

```
28 {
29     if (!mu) {
30         return BLE_NPL_INVALID_PARAM;
31     }
32
33     pthread_mutexattr_init(&mu->attr);
34     pthread_mutexattr_settype(&mu->attr, PTHREAD_MUTEX_RECURSIVE);
35     pthread_mutex_init(&mu->lock, &mu->attr);
36
37     return BLE_NPL_OK;
38 }
39 ble_npl_error_t
40 ble_npl_mutex_free(struct ble_npl_mutex* mu)
```

Y:\...\AtbmWifi6_ble_Driver_SVN3454_LMAC20712\ble_host\os\linux_app\src\os_mutex.c
2024/5/8 11:32:29 2,293 字节 C,C++,C#,ObjC 源代码 ANSI UNIX

```
28 {
29     if (!mu) {
30         return BLE_NPL_INVALID_PARAM;
31     }
32
33     pthread_mutexattr_init(&mu->attr);
34     pthread_mutexattr_settype(&mu->attr, PTHREAD_MUTEX_RECURSIVE_NP);
35     pthread_mutex_init(&mu->lock, &mu->attr);
36
37     return BLE_NPL_OK;
38 }
39 ble_npl_error_t
40 ble_npl_mutex_free(struct ble_npl_mutex* mu)
```

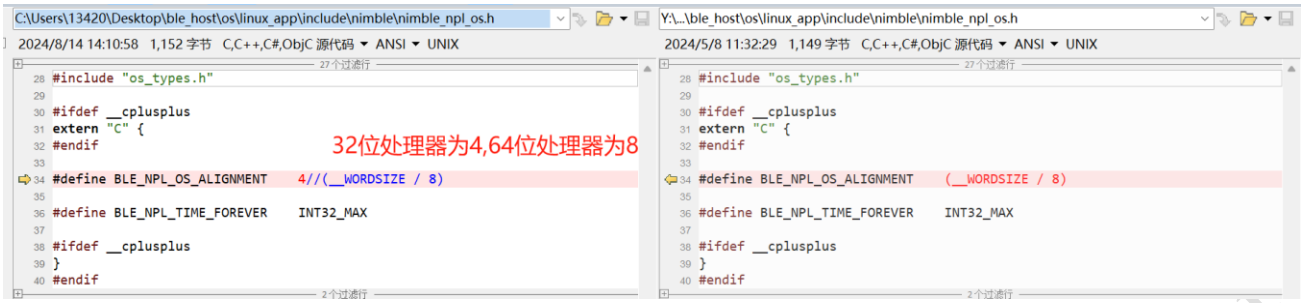
C:\Users\13420\Desktop\ble_host\os\linux_app\src\os_atomic.c
2024/8/14 14:07:38 1,121 字节 C,C++,C#,ObjC 源代码 ANSI UNIX

```
19
20 #include <stdint.h>
21 #include <pthread.h>
22
23 #include "nimble/nimble_npl.h"
24
25 static pthread_mutex_t s_mutex = PTHREAD_MUTEX_INITIALIZER;
26
27 uint32_t ble_npl_hw_enter_critical(void)
28 {
29     pthread_mutex_lock(&s_mutex);
30     return 0;
31 }
```

Y:\...\ble_host\os\linux_app\src\os_atomic.c
2024/5/8 11:32:29 1,134 字节 C,C++,C#,ObjC 源代码 ANSI UNIX

```
19
20 #include <stdint.h>
21 #include <pthread.h>
22
23 #include "nimble/nimble_npl.h"
24
25 static pthread_mutex_t s_mutex = PTHREAD_RECURSIVE_MUTEX_INITIALIZER_NP;
26
27 uint32_t ble_npl_hw_enter_critical(void)
28 {
29     pthread_mutex_lock(&s_mutex);
30     return 0;
31 }
```

Preliminary Document



16.2 atbm_cli 输入 btshell 命令无法识别

无法识别 cmd log

```
S_MB013E#
S_MB013E# ./atbm_cli btshell advertise stop
cmd_line:  stop
Unknown cmd: stop
S_MB013E# ./atbm_cli btshell dtm-tx chan=0 len=37 payload=0 phyMode=1
cmd_line:  phyMode=1
Unknown cmd: phyMode=1
```

代码修改图



Altobeam Confidential Document

CONTACT INFORMATION

AltoBeam (China) Inc.
Address: B808, Tsinghua Tongfang Hi-Tech Plaza, Haidian, Beijing, China 100083
Tel: (8610) 6270 1811
Fax: (8610) 6270 1830
Website: www.altobeam.com
Email: support@altobeam.com

AltoBeam Inc. Confidential Document

DISCLAIMER

Information in this document is provided in connection with AltoBeam products. No license, express or implied, by estoppels or otherwise, to any intellectual property rights is granted by this document. Except as provided in AltoBeam's terms and conditions of sale for such products, AltoBeam assumes no liability whatsoever, and AltoBeam disclaims any express or implied warranty, relating to sale and/or use of AltoBeam products including liability or warranties relating to fitness for a particular purpose, merchantability, or infringement of any patent, copyright or other intellectual property right.

AltoBeam may make changes to specifications and product descriptions at any time, without notice.

Designers must not rely on the absence or characteristics of any features or instructions marked "reserved" or "undefined." AltoBeam reserves these for future definition and shall have no responsibility whatsoever for conflicts or incompatibilities arising from future changes to them.

Unauthorized use of information contained herein, disclosure or distribution to any third party without written permission of AltoBeam is prohibited.

AltoBeam™ is the trademark of AltoBeam. All other trademarks and product names are properties of their respective owners.

Copyright © 2007~2025 AltoBeam, all rights reserved